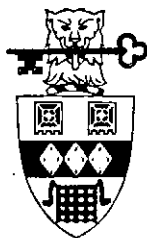


VECTOR

APII at Helsingør

• 16-page Educational Supplement	17
• Piper and Small on Namespaces	50
• APL90: News and Analysis	76
• Iverson's Dictionary of J	99
• De Kerf on Parentheses	125
• Norman Thomson on $\square CT$	135



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.7 No.2 October 1990

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from APL*PLUS/PC, IBM APL2/PC or Dyalog APL.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1990-91

Category	Fee	Vectors	Passes
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5
Non-voting Student	£5	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

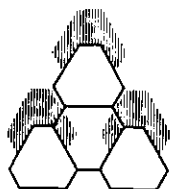
Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (There is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given on the inside back cover.

CONTENTS

EDITORIAL: Namespaces	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
APL Training Courses 1990-91		7
British APL Association News:		
Outstanding Achievement Award	David Eastwood	9
News from Sustaining Members	Alison Chatterton	11
The Education VECTOR	Alan Sykes	17
REVIEWS		
APL Product Guide	Alison Chatterton	33
APL Makes the Front Page in the USSR	Anthony Camacho	43
RECENT MEETINGS		
Packaged Workspaces in APL2	Nick Small	50
Packaged Workspaces and their Implications for Application Design	Dave Piper	63
APL90: Conference News and Analysis		
APL 90 and the Ideal APL Conference	Anthony Camacho	76
A Personal View on APL 90	Ray Cannon	80
APL in The Soviet Union	John Searle	81
Reports on the Presentations	The Vector Team	83
APL90 Reflections	John Searle	95
Pictures at an Exhibition		97
A Dictionary of J	Ken Iverson	99
GENERAL ARTICLES		
Densities Well Suited for Statistics Problems	Dr Dietrich Trenkler	118
What's Wrong with Parentheses?	Joseph De Kerf	125
TECHNICAL SECTION		
Hackers Corner: Unlocking in Dyalog APL	Ray Cannon	132
Technical Correspondence		133
Comparison Tolerance Explained	Norman Thomson	135
Namespace Play	Jonathan Barman	138
Index to Advertisers		143



APL PEOPLE

Serving the International APL Community

CONSULTANCY

Advice and assistance with all
aspects of APL systems development

RECRUITMENT

Unique placement service for
companies and individuals

SOFTWARE

Competitive prices on all APL
software tools and interpreters

Contact: Jill Moss, APL People Ltd.,
The Old Malthouse, Clarence Street,
Bath BA1 5NS

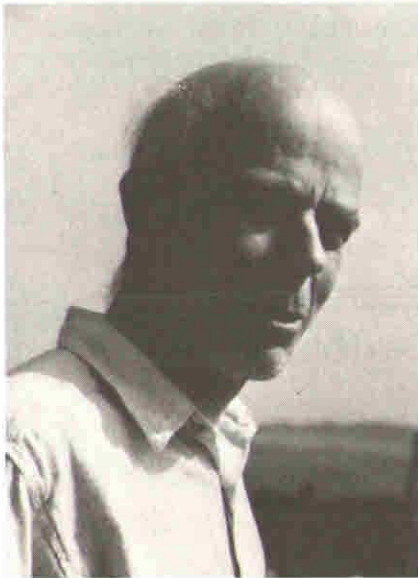
Tel: 0225 462602 during office hours
0225 333618 evenings and weekends

Editorial: Namespaces

by Jonathan Barman

Namespaces and Packaged Workspaces have been available with IBM's APL2 for a number of years but surprisingly little has been published about their concepts and uses. This issue of Vector contains reports of the June meeting which was devoted to packaged workspaces, plus some additional tutorial material.

In traditional APL, a Workspace has a Symbol Table, which records all the names used, together with special internal information such as the location of the objects in memory. A Namespace is the term used for a set of names defined by a Symbol Table, which in traditional APL is not very interesting because this is exactly the same as a Workspace. However, in APL2 one can have more than one Namespace available, each with its own Symbol Table. The system function `⌈NA` (Name Association) can be used to access objects in different Namespaces. Workspaces are packaged up into a standard load module format to make them available for use as a Namespace, hence the term 'Packaged Workspace'.



Jonathan Barman

Namespaces solve many of the problems that have beset the commercial application of APL systems, the main ones being the distribution and maintenance of standard utility functions and the management of global variables in large applications.

The management of utility functions is now very straightforward. All that is needed is a simple name association, using `⌈NA`, to a central packaged workspace. Should a utility function need maintenance, then there is only one copy to be changed, and everyone automatically gets the latest copy when they next load their workspace. File based function libraries, or complex copying procedures, can be thrown out of the window.

Global variables are always a worry, it is so easy to alter them inadvertently or to mask them with a local name. Namespaces make it possible to ensure that global variables are kept in a separate watertight compartment. For example, it is usually necessary to keep printing information such as page size and destination in global variables. Keeping all the printing functions in a separate Namespace means that these globals can be completely protected from the outside world.

Workspace listing functions are notorious for their bizarre local variable names and monolithic structure. They are deliberately written that way in an attempt to avoid any conflict with the names used in the workspace being documented. With $\square NA$ a suite of workspace documenting functions can be written in a sensible, easily maintainable way with no name clash problems.

Understanding the exact way in which namespaces can be used and manipulated takes rather longer than expected. I think that the reason for this is that there are virtually no sample applications available for study. The documentation supplied by IBM accurately describes what is available, but the worked examples tend to concentrate on the more complex aspects so that the elegance and simplicity of the underlying concepts tend to get lost.

I think that the namespace concepts can make a material contribution to writing systems which are both secure and easily maintainable. I hope that vendors of other flavours of APL will consider implementing equivalent facilities. This is a major new feature for APL which should help to promote its use for commercial systems.

Overseas APL User Groups

The Vector working group are anxious to maintain contact with as many overseas associations as possible. If you are running an APL group and would like to receive Vector in exchange for your own newsletter or journal, please contact Alison Chatterton at the address on the inside back cover. We will endeavour to compile and maintain an accurate list of contacts of overseas groups, and publish this regularly with our APL Product Guide.

Quick Reference Diary

Date	Venue	Event
19 October	York	BAA Field Trip to Rowntrees
23 November	London	BAA Meeting

... remainder of programme to be confirmed.

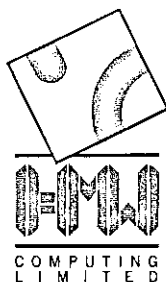
4th - 8th August 1991	Stanford	APL91: The Next 25 Years
-----------------------	----------	--------------------------

British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

Meetings are normally held on the 3rd Friday of the month throughout the autumn and spring. Dates for the 1991 programme will be confirmed at the September committee meeting.

Dates for Future Issues of VECTOR

	Vol.7 No.3	Vol.7 No.4	Vol.8 No.1
Copy date	1st Dec 90	1st March 91	4th June 91
Ad booking	8th Dec 90	12th March 91	15th June 91
Ad Copy	23rd Dec 90	21st March 91	23rd June 91
Distribution	January 91	April 91	July 91

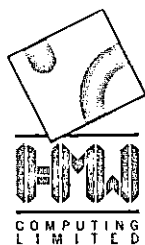


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:- deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

APL Training Courses for 1990-91

compiled by Alison Chatterton

Dates shown cover the period from November 1990 to March 1991. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	9 Oct, 30 Oct, 27 Nov, 11 Dec
	MicroAPL	1	4 Oct, 29 Nov, 7 Feb, 4 April
	Mercia Software	3	14 Nov
APL*PLUS/PC	MicroAPL	1	1 Nov, 10 Jan, 7 Mar
	Mercia Software	2	17 Jan
Intro to APL2	MicroAPL	1	25 Oct, 15 Nov, 24 Jan
Intermediate	MicroAPL	1	11 Oct, 6 Dec, 14 Feb
APL*PLUS II conversion	MicroAPL	1	8 Nov, 17 Jan, 14 Mar
Advanced (Nested Arrays)	MicroAPL	1	18 Oct, 13 Dec, 21 Feb
	Cocking & Drury	2	13 Nov
System Design	Cocking & Drury	4	17 Sept
Statgraphics	Cocking & Drury	2	2 Oct

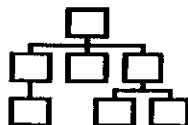
Welcome back Mercia Software! They have a new training room equipped with the latest overhead PC projection system as well as PCs for hands-on work.

A single source for APL



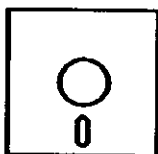
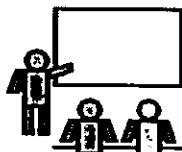
Advice

Design



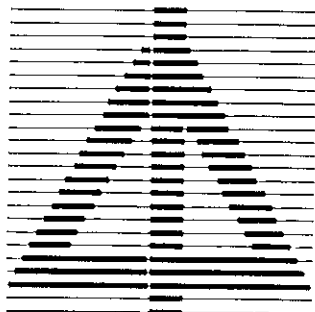
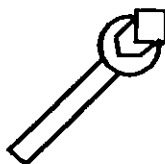
**Application
Development**

Education



APL software

Support



COCKING & DRURY LTD.
THE APL PROFESSIONALS
180 Tottenham Court Road
London W1P 9LE
Tel: 01 - 436 9481

(From May 1990, our STD code will be 071)

British APL Association News: 1990 Outstanding Achievement Award

from David Eastwood

Background

Each year the British APL Association makes an award for what is, in the committee's estimation, an Outstanding Achievement within the APL community. The terms which govern the award are fairly loose and the Committee are looking for achievements which assist the overall objectives of the BAPLA - be they technical achievements in APL; APL interpreter design; services to the APL community; favourable publicity for APL and so on. The previous Awards have been to John Scholes for his work on the Dyalog APL interpreter and to British Airways for their in-house APL MIS systems.

THE 1990 AWARD

This year, the Awards subcommittee was faced with the widest choice that has been presented to them in the three years since the award was instituted. After much consideration, and a number of visits to some of the contenders, the subcommittee decided unanimously to make the award to Metapraxis for the Resolve package.

Metapraxis, based in Kingston Surrey, was founded in 1984 to supply Executive Information Systems and supporting consultancy services.

In its first 6 years of existence, Metapraxis has been highly successful - with over a hundred major corporate and governmental clients around the world. A subsidiary company was established in the USA in 1988.

A major element of Metapraxis' success has been the Resolve software package. Resolve is one of the leading Executive Information Systems currently commercially available and has been widely acclaimed in numerous review articles in the computer and general press. In 1989 Resolve won a British Design Award.

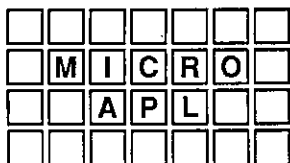
Resolve is designed for operation on PCs and is written in APL*PLUS/PC.

The Outstanding Achievement Award subcommittee felt that Resolve merited the 1990 award on the following grounds:

1. Technical excellence of the product.
2. Public recognition in the shape of the 1989 Design Award.
3. Demonstration of the use of APL in the commercial packaged software environment.

Most APL programmers and users are well aware of the power of APL, but as a community APLers are rather inward looking. Opportunities to show that, given the right application, APL can hold its own with other programming languages are rarely taken. The Award subcommittee were firstly impressed with the quality of the work that Resolve embodies, but more importantly were pleased to note the Resolve had demonstrably met a real need in the market and had proved its worth against a number of non-APL rivals.

APL.68000 Level II



APL.68000 Level II is a full specification APL2 for the 68000 family of processors. It incorporates all the user-oriented enhancements of APL.68000 Level I as well as MicroAPL's acclaimed windowing interface. APL.68000 Level II is available for computers such as the NCR Tower, Sun, HP, Bull, Nixdorf, Wicat, Mac, Amiga and ST. Features include:

- Nested and mixed arrays
- User-defined operators
- Selective specification
- Upwards compatibility with APL.68000 Level I

For more details contact:

MicroAPL Ltd.,
South Bank Technopark,
90 London Road, London, SE1 6LN.
071-922-8866

News from Sustaining Members

Compiled by Alison Chatterton

HMW Computing Limited

HMW Computing Limited continue to develop and market "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System.

We have been extremely busy over the last six months and have released yesterday (Monday 3rd September) the latest release of 4XTRA into a major UK Bank. The system is supporting about 130 traders and is fully integrated into the banks existing systems.

Excellent teamwork and professionalism has been displayed by the HMW personnel involved in this project and we are happy to report a successful conclusion of this phase of work.

Following on from this we are about to start major developments of the product and hope to release news to Vector soon.

If you would like to see the latest 4XTRA system, we are exhibiting at a number of exhibitions and would be please to offer invitations or if you prefer a demonstration can be arrange at our offices or on your own premises.

Our general APL consultancy business is doing well given the general recession in the industry and a number of our clients have had major system delivered over the last few months. However, maintenance and development of existing system continues to be our major on going work load. We encourage APL users who are looking for help to use our services as we can supply consultants with proven track records of success. Naturally our Employment Agency can supply permanent people if you prefer.

APL People Limited

Efforts continue to build up our overseas business in readiness for the coming of the European 'open market' in 1992. We have already successfully placed people

in jobs in Spain, Sweden, Luxembourg, France, Toronto and Belgium and were looking for APLers keen to work in Saudi just before the 'Gulf crisis' blew up!

We currently have a number of excellent APL consultants available for assignments either in the UK or abroad, and would like to hear from any companies who are having difficulty in finding people of a high enough calibre for their needs. I can invariably find someone with the right skills, no matter what the problem (or location!)

We also plan to set up a 'help-line' offering free advice to companies experiencing problems with the development or conversion of systems in APL. Between them, our consultants have worked in all industry sectors - some have over 20 years' experience in the use of the APL language - and are able to advise on any aspect of systems development, large or small.

Dyadic Systems Ltd

Dyadic is pleased to announce Version 6.1 of Dyalog APL which will initially be available for DOS/386 but will also be ported to the Unix implementations of the product. Version 6.1 contains the following enhancements:

Browsing/Editing Variables

You can now open a display/edit window for a variable. The window is opened in the same way as for functions and operators, i.e. by using `)ED` or by positioning the cursor over the name and pressing the `<EDIT>` key. The appearance of the variable in the window is exactly as if it were displayed using standard output ignoring `⎕PW`. If you subsequently change the value of the variable, the display changes accordingly. This is particularly useful for debugging when using the TRACE facility because you can watch the value of one or more variables change as you step through a program.

If you skip to the window using `<TAB>` you can scroll the data vertically or horizontally to browse a large array. ANY array can be displayed, but in this release only certain types of array can be edited. This restriction will be lifted in later releases. Additional colour parameters may be specified to distinguish between variables and functions and between "editable" and "non-editable" arrays.

□ED

A new system function, □ED, has been introduced to permit the browsing and/or editing of variables and functions/operators under program control.

New Colour Scheme

In response to criticism in this journal and elsewhere, a new default colour scheme has been introduced for Release 6.1. However, in the sure knowledge that "you cannot please all of the people all of the time", Dyadic expects the new scheme to be as much a source of amusement and debate as the old one.

Cocking and Drury Ltd

At the beginning of August Cocking & Drury became STSC's PLUS*DEALER for the UK. This is a new programme launched by STSC for their major dealers - there are only 5 world-wide.

One of the benefits of this new status is that it gives us access to consistently better discount rates on APL*PLUS PC purchases and this has helped us REDUCE PRICES on all the APL*PLUS PC products - see Product News for details.

STSC itself has gone through a major reorganisation which has separated APL*PLUS and STATGRAPHICS into an 'Analytical Software Division' which will enable STSC to continue to commit itself to the further development of APL*PLUS products.

The dealers at STSC's International Dealer Conference held in Windsor at the end of August put forward a number of suggestions about what these developments should be!

Products announced at the Dealer Conference for release in the near future included APL*PLUS UNIX Version 5 which will include a Version for the IBM RS6000; APL*PLUS PC Version 10; Statgraphics Version 5 and a new Statgraphics 386 product.

Here at Cocking & Drury we have agreed with one of our clients that we will develop and market a product originally written for their internal use. The UTILITY DELIVERY SYSTEM is a set of facilities for the management of APL utilities in the APL*PLUS mainframe environment (and in the networked

APL*PLUS II System Version 3). It allows developers of APL utilities to create and maintain Utility Files and APL application developers to deliver Utility File based utilities into running applications. For further information contact us on 071 436 9481.

We have also said 'Au-revoir' to Dave Ziemann who - after 13 years work - has decided to take a year out to see the world. We look forward to his return in 1991.

MicroAPL Ltd

In the last issue of Vector we announced the launch of our new APL - APL.68000 Level II. The official launch took place at APL90, and we have prepared a special issue of MicroAPL news which covers the development and features of APL.68000 Level II. Most readers of Vector Should have received their copy of MicroAPL news issue 13 by now - if you haven't, contact us for a free copy.

We made use of the gap between publication deadlines for Vector 7.1 and the launch of APL.68000 Level II to improve the specification of the product by including user-defined operators in the first full release - version 1.1. Publishing the full specification here would take more space than we are allowed, so I will confine myself to saying that we have now implemented all the key features of APL2 - nested and mixed arrays; user-defined operators; selective specification. In addition we have added a whole host of new system functions and system variables.

At APL90 we showed APL.68000 Level II running on the NCR Tower, the Apple Macintosh, Commodore Amiga and the Atari ST. By the time we went to Copenhagen we had produced versions of APL.68000 Level II for all the major systems that run APL.68000 Level I (as the original product is now called) - the multi-user Unix and Mirage versions and the single-user Mac, Amiga and ST versions.

We have now begun to ship the first sets of APL.68000 Level II. Prices are published elsewhere in this issue of Vector. Users of APL.68000 Level I (as the original product is now called) - can upgrade to Level II at a special price.

Now that we are publicly involved with APL2, we have extended the scope of our training programme to include an introduction to APL2. We have already

been running APL*PLUS II training courses for some time now but we feel that our new APL2 courses should be of particular interest to anyone using APL2 in any of its implementations (APL2/370, APL2/PC and APL68000 Level II). In common with our APL*PLUS II courses, we assume some basic APL knowledge on the part of attendees (for example as might be obtained on our Beginners and Intermediate courses). The dates of the next few scheduled courses are printed in Vector and if you would like more details please contact us. We are finding that more and more clients are asking us to prepare 'custom' training courses for them which are tailored to reflect their own in-house needs.

Finally, our APL*PLUS users will be delighted to hear that we are now shipping APL*PLUS II version 3.0 and upgrades from earlier versions are also available.

I.P. Sharp Associates Limited

I.P. Sharp Associates Limited has been renamed REUTER:FILE Ltd with effect from July 1, 1990. I.P. Sharp Associates, headquartered in Toronto, Canada was acquired by Reuters Holdings PLC of London, England in June, 1987. The Reuter:file companies form part of the newly established Reuters Historical Information Division which provides high quality software products, and text and numeric information products to the financial and business communities throughout the Reuter worldwide organization.

Compass R&D Ltd (née MetaTechnics Systems Ltd)

Last year MetaTechnics merged with the COMPASS group of companies, a world-wide group with offices in Scandinavia, Switzerland, UK, USA and West Germany. The group provides efficiency and cost analysis studies to IT and senior management.

The group started in 1980 and MetaTechnics has been connected with COMPASS since 1985 having designed and implemented the present generation of COMPASS software - mostly in APL*PLUS/PC.

The main product of the group is DC COMPASS which is a management tool for improving the overall efficiency of clients Data Centre operations, based on comparative analysis. The system uses a model which contains over 2000 separate quality and cost measures and a database of over two hundred companies.

Since merging with COMPASS and in addition to software and product development we have become responsible for all central support and services and worldwide database administration.

To reflect our changing role we changed our name in June 1990 to COMPASS R&D Ltd and because of more staff have moved premises to the Surrey Research Park adjacent to Surrey University.

We are constantly developing new products and have recently announced Network COMPASS which performs comparative analysis on the cost and efficiency of SNA Networks. Current projects include development of new models, simulation techniques and merging the data centre and network projects. We are planning to move to a second generation APL and are currently evaluating the contenders.

We will continue to support the BAPLA and help to promulgate the use of APL to the data processing world at large.

If you require further information about COMPASS services please contact John Stembridge or David Toop at the address on the back of Vector.

THE EDUCATION VECTOR

October 1990

Editor Alan Sykes

This Education Vector has been reprinted from VECTOR Vol.7 No.2. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727 860130.

Contents

Editorial	Alan Sykes	18
Puzzle Corner		19
Letters		20
Searching for Strings in Strings	Alan Sykes	27
An Even Sneakier String Search!	Adrian Smith	31
APL Booklist	Ed Shaw	32

Dr Alan Sykes,
c/o European Business Management School
Swansea University
Singleton Park, Swansea SA2 8PP
Wales, UK

Editorial

Another Year Ahead

Another academic year looms ahead as I write this editorial: buckets and spades have been packed away. My teaching courses will probably involve the supervision of over 150 students either using APL statistical functions or actually learning to program in APL. So, writing this edition has helped me to get in the mood.

Since the last edition (JULY's not APRIL's!) I have had quite a few letters and I have included as many as I can. Once again, I invite you to send your contributions. It is helpful if you are able to present them as near as possible in a form suitable for inclusion in Education Vector as I have limited time to spend actually editing material.

Once again I have included a puzzle corner which I hope will provoke some interesting communications from readers. Thank you to all readers who have taken the trouble to write to me, either with puzzle solutions, or with general comments on what they have been doing with APL in education. If I have not included your contributions so far, please forgive me for the omission as it is not always easy to do all you want to do in a severely limited amount of spare time. Might I suggest a gentle reminder in the form of another copy?

Help wanted urgently

Greg Morse has just completed the OS9 port of I-APL.
Do you have an OS9 computer, a little skill in
handling its operating system and some time to help
turn a completed port into a stock of disks ready for
the mailing house to supply against cash with order?
If so please telephone Anthony Camacho on 0727-860130.

Puzzle Corner

Chess Board Pattern

On the basis that APL can be (and usually is) fun, previous Education Vectors have included a puzzle or challenge section. Although I don't get inundated with responses, a significant number of readers do send in their thoughts. Indeed, I am still receiving solutions to the original puzzle I included (converting marks into grades).

In this edition I decided to continue the theme of strings and look at the sort of patterns that can be created from a simple vector of two characters, say `□` and `•`. The following 9 by 9 squares can be generated easily by APL expressions which end with the string `'□•'`.

```

□□□•••□□□  □□□•••□□□  □□□•••□□□
•••□□□•••  •••□□□•••  •••□□□□□□
□□□•••□□□  □□□□□□•••  •••□□□•••
•••□□□•••  □□□•••□□□  □□□□□□•••
□□□•••□□□  •••□□□□□□  □□□•••□□□
•••□□□•••  •••□□□•••  □□□•••□□□
□□□•••□□□  □□□•••□□□  •••□□□□□□
•••□□□•••  □□□•••□□□  •••□□□•••
□□□•••□□□  •••□□□•••  □□□□□□•••

```

```

□□□•••□□□  □□□•••□□□
•••□□□□□□  □□□•••□□□
•••□□□•••□  □□□•••□□□
□□□□•••□□□  •••□□□•••
□•••□□□□□□  •••□□□•••
•••□□□•••  •••□□□•••
□□□□□•••□  □□□•••□□□
□□•••□□□□  □□□•••□□□
□•••□□□•••  □□□•••□□□

```

This edition's puzzle corner is to identify simple APL expressions that will produce some of these patterns; in particular the last pattern which is basically a chess-board. Having produced this, then production of a full size chess board should be easy. Please send your solution(s) or patterns to the editorial address.

COMMUNICATIONS

'DAYOFWEEK': Education Vector 7.1

From: C.E.Williams

Sept 1990

I hope the following may help to shed light to those who are unaware of the method used in the function 'DAYOFWEEK'

Turner's function is simply the APL coding of the method due to Gauss, devised to determine the day of the week on which a given date will occur. Since in a leap year an additional day is added at the end of February, it is easier to assume that the year starts in March. Then March, April and May will be the first, second and third months of the year N, January and February will become the eleventh and twelfth months of the year N-1. The days of the week begin on Sunday by 0,1,2,...6. Then f, the number of the day of the week corresponding to the kth day of the mth month of the year N, is determined as follows:

$$f = (k + (2.6m - 0.2) + D + (1/4 D) + (1/4 C) - 2C) \text{ Modulo } 7$$

Thus the example in Vector: August 1st 1990

```
k=1, m=6, D=90, C=19. C=number of the century
D= last two digits of year N Then

f=(1+15+90+22+4-38) Modulo 7

= 94 Modulo 7

= 3 i.e. Wednesday.
```

With this in mind:

Line 1 of DAYOFWEEK sets up M

Line 2 decodes M to 23888 using $0 \ 12 \ 1 \ M[3 \ 1];$

this is then encoded to 1990 5, having subtracted 3, with the code $0 \ 12 \ 1 \ 3 + 0 \ 12 \ 1 \ M[3 \ 1];$

1990 5 is rotated and one added to become 6 1 1990;
this is assigned to M, which now is 6 1 1990;

finally on line 2, Q becomes 90 using $100 \mid M[2]$.

Line 3 collects terms as per Gauss, and ends with MODULO 7 giving the day number 3.

(N.B., Turner has combined the last two terms $+(1/4C) - 2C$ into one: $-1.75C$.)

If I may be allowed to criticise (*sure!* - Ed) line two seems too complicated. Why not test the value of the month, if 3 to 12, subtract 2, and the year will be MODULO 100 assigned to q . If month is 1 or 2, add 10 to month and the year will be MODULO 100 -1, again assigned to Q . It would be much easier to follow and no doubt speedier.

Bearing in mind k is the day of the month, substitution of a day into M , the second term allows the day of the week for any date, if $M+2 \ 29 \ 1952$ then 29th February 1952 fell on a FRIDAY (3).

Thanks a lot for Education Vector; please keep up the good work!

Yours sincerely

C.E.Williams

Cold Keld
Loweswater
Cockermouth
Cumbria

Reading ASCII files into I-APL

From: J.C.Hines

Sept 1990

In the April 1990 issue of 'The Education Vector', you appeal for any information on reading ASCII files into IAPL. I have been away for some time and have not been able to write to you. I realise that you may be flooded with responses (*Ed's comment = No!*), but, in case not, here are a few points.

I can read ASCII files in without trouble and have written a little function called Getfile, which reads

```
Z←Getfile filename
DFOPEN filename
Z←DREAD DFSIZE
DFCLOSE
```

There are a few things to take care of when using this function, all quite simple, but it is often the simple things we overlook, especially when working alone and without someone to act as a cardboard Indian.

One thing is to remember that the filename is a string, so that the call must have the argument in quotes, i.e. be on the lines of

```
TextVar ← GetFile 'a:thislet.doc'
```

Another depends on the way the ASCII file was produced in the first place. I-APL reads the file as a vector, unless the shape is specifically changed in the reading function. This means that line ends may not be marked as specific characters, because the input may have been created in a word processor which has re-formatted the document on the screen but without inserting actual carriage returns at the end of each line. Many of the line ends will have only virtual carriage returns. There are several approaches to this. One is to look at the file as received for carriage returns, possibly on the lines of

```
(TextVar=⊡AV[12])/1ρTextVar
```

to find out where the real carriage returns are. If they are not tidy, reformatting can be done by finding the length of the longest line and then padding the shorter ones with blanks to create a matrix. I found it easy to ensure that the text in my wordprocessor had real carriage returns at the end of each line.

I hope these thoughts are sufficient to help Mr Sargeant to trace the source of his problem.

While writing to you, I am taking the opportunity to include another little function. This was written because of an article in 'Program Now' on ways of listing all the ways in which *m* objects could be chosen from *m*, allowing duplicates, but without using recursion. The discussion assumed non-APL languages, without recognising that this was a self-imposed limitation. The problem was therefore now to write something on the lines of:

```
for i=1 to num
  for j=1 to num
    for k=1 to num
      .
      .
      .
    write i,j,k .....
```

... but without knowing the depth of the nesting at compile time.

Various solutions were produced, using pointers, stacks or just choosing an arbitrary maximum depth of nesting and then providing a drop-out if this was not-reached.

It has amused me to write a version in APL which shows how much more powerful it is for problems of this kind. My code is:

```
Z←ALLCOMS;N;⊞IO
⊞IO←0
R←(NρN)⊔N*N
Z←(⊞AV[(49+19),65+126])(1N))[qR]
```

The problem can be rephrased as one of counting up to $N*N$ in base N . Line 1 calculates the number of terms and uses encode to get the answer but written in decimal arithmetic. Line 2 then creates a symbol list, consisting of as many characters as required from the list 1..9,A..Z, rotates the answer from line 1 so that the display is a neat list, and then indexes on these symbols. This does have a limit when N reaches 35, but $35*35$ is big enough that most of us will not want to wait around for this number of cases to be calculated. I think this is a nice demonstration of the value of APL. I hope you do too.

J.C.Hines

2 Lockerly Close
Lymington
Hants
(SO41 BER)

*Editor's comment: Yes, I agree it's a nice approach: unfortunately its use is somewhat limited by the size of the array that is created in line 2. This is an $N*N$ integer array, and each integer requires four bytes of storage. Hence for this program to work up to $N=35$ would require a computer with approximately $10*51$ Kbytes of random access memory! On a computer with 640K RAM, ALLCOMS is bound to fail for $N>6$.*

Dbase and Lotus files into I-APL

From: W J Smith

Sept 1990

Your appeal on behalf of E.J.Sargeant in the July Vector re retrieving ASCII files into an IAPL workspace struck a chord. Some time ago, on the grounds that it is usually possible to transfer data from one package to another, I experimented with transferring files from Lotus and dBaseIII into an IAPL workspace.

dBaseIII files, output with a COPY TO <filename> TYPE SDF command, yield a series of fixed length records in ASCII format. These are only fixed if the rightmost field of each row is of the same size but this is easily managed even if you have to create a dummy column to achieve it! A further problem with Lotus files is that they are preceded by a number of carriage returns/line feed characters which need to be disposed of in the first stages of transfer; actually in the *DFREAD* function.

The *DFREAD* function in the IAPL DFILE workspace reads foreign files into the workspace as a VECTOR which includes all the CR's and LF's of the original file.

Having acquired the vector, a count of the CR's can be used to determine the number of records (as long as you haven't got the leading ones from LOTUS!). Then, after stripping out the unwanted CR and LF characters, the vector can be reshaped into an appropriate character matrix.

This matrix can then be separated into character matrices corresponding to the columns/fields of the original files; effectively generating an inverted file in data processing jargon.

I enclose a couple of 'partly-baked' functions I wrote while experimenting with this problem; they may be of use to your enquirer.

```
▽ Z←GETSDF F
[1]  R OPENS AND READS AN ASCII FILE 'F' IN THE \APL DIRECTORY
[2]  Z←'FILE ',F,' NOT FOUND'
[3]  →(0=DFOPEN F)/0
[4]  Z←DFREAD DFSIZE
▽
```

```

V Z+MSDFM A
[1]  A CONVERTS AN SDF FILE INTO A MATRIX OF
[2]  A ITS CONSTITUENT RECORDS. B CALCULATED, IS THE NUMBER OF
[3]  A RECORDS IN THE VECTOR, A IS THE NAME OF THE RAW VECTOR
[4]  A [7] COUNTS RECORDS, [8] REMOVES THE CR'S AND THE LF'S,
[5]  A [9] CALCULATES THE RECORD LENGTH AND RESHAPES THE
[6]  A CLEANED VECTOR
[7]  B++/A=⌈AV[14]
[8]  A+((A*⌈AV[14])^(A*⌈AV[11]))/A
[9]  Z+(B,⌈((ρA)+B))ρA
[10] A
[11] A WORKS WITH .PRN FILES GIVEN THAT: (1) THE LAST LOTUS
[12] A COLUMN HAS ALL DATA OF EQUAL LENGTH; (2) THE FILE IS
[13] A READ WITH '9 DFREAD DFSIZE' TO SKIP BLANK HEADING ROWS
V

```

W J Smith

Plymard
The Barrow
Boddington
Cheltenham

I-APL Constraints

From: Walter G Spunde

July 1990

Thank you for running my article FUNCTION SAMPLING WITH I-APL in the July issue of Vector.

I appreciate your comment on ways of clearing the screen without the need to create a screenfull of blanks (25 1ρ ' '), although a stubborn streak in me is tempted to argue that closing down a screen from the top is more gentle on the user than shooting it out through the top like a sheet ripped from a typewriter. There is also the machine code call:

```
CL: ap11519 ⌈MC 1 2 1ρ 'AB' ZZ A+203 0 0 25 80 ZZ B+ZZ ⌈FX 'ZZ:α'
```

... which achieves the result I prefer, and which one could not expect the average maths teacher to know. All this, however, is beside the point I was driving at in my article, which is, that teachers WILL want to focus attention on key areas without the distraction of their having to create certain essential tools for this study; that 32K workspaces are really too small for even simple tasks like graph-

plotting and that the lack of such elementary facilities as built-in screen handling in I-APL does consequently push one's patience to the limit.

Teachers who have to use I-APL, because of cost constraints, are not in the market for a site license for a commercial APL at this point in time. At the same time their students are often accustomed to playing with sophisticated games software. I do believe that the constraints imposed on I-APL are too severe to allow teachers to come to a happy balance between their needs and student expectations or even to exploit the full potential, especially in the graphics area, of I-APL.

How many teachers will feel that it is worthwhile developing a software library (of minuscule pieces) for I-APL? The result may well be that teachers conclude that APL would be wonderful if you could afford a commercial version, but otherwise you may as well forget about it, i.e. the same situation as before the I-APL project was initiated.

I hope I have not bored you with these comments and that you may pass them on to those more closely associated with the I-APL project, whose efforts I do most certainly appreciate and commend despite my criticisms.

Walter G Spunde

University College of Southern Queensland
School of Information Technology

Editor's comment: I appreciate the argument, but your article showed just how much can be done in the way of producing excellent teaching material using I-APL despite the real problems discussed above.

Searching for Strings in Strings

by Alan Sykes

No, this not an introduction to String Theory which I'm told is the latest attempt of Mathematicians to provide the mathematical foundations necessary for Physicists to explain the fundamentals of Nature. Rather just a short article continuing my theme of string handling in APL.

Perhaps the most frequent use of a computer is for word-processing. One of the essential tasks that a word-processor must perform is that of searching for the occurrence of target strings in a text. Facilities are then provided for replacing that string with a preferred string; this can be very useful where a complicated name or phrase occurs often throughout a particular text. Text editors for computing, such as the one I am using now, also provide a 'search and replace' facility, so that variable names can be changed through a program.

I was reminded of this topic in a wider context by a letter and newspaper article sent to me by Bill McClean of Dunoon. He sent me an article concerning the work of Andrew Morton and a Computer Science Colleague of mine Mike Farrington who have recently developed new techniques of examining texts (e.g. novels) to test hypotheses concerning disputed authorship. Specific examples of this are parts of the Bible whose authorship is questionable and the famous case of the Federalist Papers. Their technique depends crucially on being able to search through the text for the occurrences of particular words, or types of words, or phrases, hence the connection with this article.

Let's start with a simple example.

```
STRING←'CAT'
TEXT←'THE CAT WAS A CAT NO MORE'
```

First I will try to construct 'an APL solution' to the problem of string searching (i.e. one without looping). Clearly it is necessary to use = to find the occurrences of each letter of 'CAT' in TEXT. The outer-product ∘.= clearly is what we need.

```
STRING ∘.= TEXT
0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
```

Now the problem is to look for the diagonal sets of 1's. One way to do this is by rotation of the second row one place to the left, and the third row 2 places to the left, so that the diagonal lines become columns. This can be done using ϕ with a left argument. Here are some examples that show you how ϕ works:

```

      M+4 4p14
      M
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

      0 1 2 3φM
1 2 3 4
2 3 4 1
3 4 1 2
4 1 2 3

```

Each number of the left argument of ϕ tells you how many places to the left the equivalent line is to be rotated. (A negative number indicates rotation to the right.)

This is clearly what we need to align the series of 1's. We can then test that the aligned columns are all 1's by using $\wedge$ ('and') applied to the columns, i.e. $\wedge \neq$. To achieve the required rotation, we create the vector 0 1 2 (in this case) using the general formula $\neg 1 + \text{ip} \text{STRING}$ (assuming the index origin is 1). Hence,

```

      ^/ (¬ 1 + ip STRING) φ STRING * . = TEXT
0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0

```

The position of the 1's in the resulting vector indicate the required occurrences of 'CAT'.

Having discovered the essential method, and programmed it successfully, now starts the problem of trying to make the method apply in all sensible cases. What happens, for example, if we use the above formula when $\text{TEXT} + 'A'?$ Can you anticipate the problem? Basically, instead of a Boolean matrix, we obtain a vector, hence $\wedge \neq$ will fail as there are no columns to reduce on. This problem can be easily circumvented by making sure that STRING is a vector, i.e. $\text{STRING} + , \text{STRING}$.

A second problem occurs because of the rotation. Suppose $TEXT \leftarrow 'THIS\ CAT\ IS\ CALLED\ ICA'$. Then a matching will occur on the penultimate place because the last two letters are CA and the first is T . One easy way to prevent this is by appending to $TEXT$, a 'character' that does not/will not occur in the text, say $AV[1]$. Piecing all this together, we get the function SS .

```

▽ R←STRING SS TEXT
[1] STRING←,STRING ∘ TEXT←TEXT,AV[1]
[2] R←(⌊(-1+⌊ρSTRING)⌋STRING ∘.=TEXT)/⌊ρTEXT
▽

```

One drawback of the APL solution, is that for large $TEXT$ and moderate $STRING$ sizes, the $∘.=$ construct creates a huge array, making this very infeasible. What alternative strategies might be appropriate, still with the aim of achieving efficient APL code?

When thinking about this, it occurred to me that the 'Sieve of Erasthenes' method could be adopted to this situation. This method is used to derive the prime numbers in the set of the first N natural numbers, proceeding by deleting all numbers divisible by 2, then all divisible by 3 and so on. For example,

```

W←150
X←X+(0≠2|W)/W
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41
  43 45 47 49

```

```

Y←Y+(0≠3|X)/X
1 5 7 11 13 17 19 23 25 29 31 35 37 41 43 47 49

```

```

Z←Z+(0≠5|Y)/Y
1 7 11 13 17 19 23 29 31 37 41 43 47 49

```

```

Z←Z+(0≠7|Z)/Z
1 7 11 13 17 19 23 29 31 37 41 43 47

```

In the context of searching for strings, suppose we searched for all the occurrences of the first letter of $STRING$; then search amongst those positions+1 for the occurrence of the second letter of $STRING$. At each stage the size of the APL problem is considerably reduced, and looping only has to occur for $(\rho STRING)$ times. The program $SS2$ executes the new algorithm. It is

convenient to append to *TEXT* some dummy characters so that simple indexing is easy. Once again we coerce *STRING* to be a vector.

```

      V R+STRING SS2 TEXT;L;N;C
[1]   STRING+,STRING ◊ L+ρSTRING ◊ C+1
[2]   TEXT+TEXT,(L-1)ρ□AV[3]
[3]   R+1ρTEXT
[4]   LP:R+(STRING[C]=TEXT[R+C-1])/R
[5]   →(L≥C+C+1)/LP
      V

```

Now let's test the two algorithms on this article which at present is 5479 characters long.

```

      'STRING' SS2 ARTICLE
1477 1788 2786 2815 2823 3414 3439 3447 3835 3856 3864 3907
      3915 4014 4912 5001 5112 5271 5313 5341 5349 5361 5425

```

... tells me that the first occurrence of (uppercase) *STRING* is in the 1477th position etcetera. The response is more or less immediate, whereas for *SS*, it takes about 4 seconds to produce the equivalent result (running on an IBM PS/2 Model 70).

Of course, the next step is to be able to replace *STRING* with an alternative string. This doesn't appear to be straightforward, unless the length of the alternative string is the same as the original. If the new string is greater in length, then the original *TEXT* vector has to be padded out to make room for the extra characters at appropriate places. This can be done using 'replicate' (e.g. 1 2 1 2 / 'ABCD' yields 'ABBCDD'). Then, substitution can take place. If, on the other hand the new string is smaller in length, then appropriate parts of the original text must be dropped e.g. 1 0 1 0 / 'ABCD' gives 'AC'. Any offers for an APL function *REPLACEIN* with the syntax as indicated by the example below?

```

      'CAT/DOG/' REPLACEIN TEXT+'THE CAT SAT ON THE MAT'
THE DOG SAT ON THE MAT

```

An Even Sneakier String Search!

by Adrian Smith

Alan should have known better than to leave me a blank page to fill! A couple of timings on the *entire* Education Vector (22130↔pQQQ) showed that Alan's SS2 is indeed rather fast, but can be beaten by the APL*PLUS 'find' primitive $\underline{\epsilon}$...

```
Z+ 'STRING' SS2 QQQ ... 2.3 sec
ZZ+ ('STRING'  $\underline{\epsilon}$  QQQ)/1pQQQ ... 1.6 sec
ZZ^.=Z
```

1

Clearly, this is not good enough for the true APL hacker! The function below takes the additional step of arranging the substring so that the least frequent letters are matched first! This cuts down the size of the potential targets much faster, and will hit the 'STRING' test in 1.46 sec. I believe the APL*PLUS $\square$ SS system function is simply an assembler coding of this algorithm; one wonders why they bothered!

∇ POS+SUB Δ SS TARGET;GR;LEN;CT

```
[1]      R Returns index of all locations in TARGET where
[2]      a substring SUB is found.
[3]      a
[4]      R From: APL Design Handbook (Adrian Smith) Wiley
[5]      a Algorithm adapted from Bob Smith's
[6]      a "Collected Whizzbangs" circa 1978 by STSC.
[7]      a

[8]      TARGET+,TARGET  $\diamond$  SUB+,SUB
[9]       $\rightarrow$ (2 1 0  $\leq$ pSUB)/OK,Single,Empty
[10]     Empty:POS+1pTARGET  $\diamond$   $\rightarrow$ 0
[11]     Single:POS+(TARGET=SUB)/1pTARGET  $\diamond$   $\rightarrow$ 0
[12]     OK:LEN+0[(pTARGET)-(pSUB)-1
[13]     SUB+SUB[GR+ $\nabla$ ' ETARINOS',SUB]
[14]     GR+GR-1
[15]     POS+(LEN+GR[1]+TARGET=1+SUB)/1LEN
[16]      $\rightarrow$ (pPOS)+0  $\diamond$  GR+1+GR  $\diamond$  SUB+1+SUB  $\diamond$  CT+1
[17]     Lp: $\rightarrow$ (pPOS+(TARGET[POS+GR[CT]]=SUB[CT])/POS)+0
[18]      $\rightarrow$ ((pSUB) $\geq$ CT+CT+1)+Lp
```

∇



Renaissance Data Systems

Enlightenment Thru Information Processing

ALL APL BOOKS IN PRINT

Catalog excerpts as of September 1, 1990

ACCOUNTING STRUCTURED IN APL , Ijiri, 1984, 491p.	\$10.00
The basic principles of accounting with numerous APL expressions to model them.	
COMPUTING IN STATISTICAL SCIENCE THRU APL , Anscombe, 1981, 426p.	\$42.00
Lots of statistics and APL. His workspaces are available from Yale University. A classic.	
MATHEMATICAL EXPERIMENTS ON THE COMPUTER , Grenander, 1982, 525p.	\$64.00
Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed.	
PROBABILITY IN APL , Alvord, 1984, 142p.	\$17.50
A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required.	
DISKETTE FOR ABOVE containing functions covered in book.	\$22.00
APL WITH A MATHEMATICAL ACCENT , Jones, Reiter, 1990, 200p.	\$41.95
Specially written for use in advanced high school or college math courses. By a student of Alvord's made good!	
APL - THE LANGUAGE AND ITS ACTUARIAL APPLICATIONS , de Kerf, Goovaerts, Stiers, 1987, 223p.	\$89.00
Introduction to APL. Loss reserves, credibility, probability, numerical analysis, forecasting, with APL functions.	
COMPUTATION FOR THE ANALYSIS OF DESIGNED EXPERIMENTS , Heiberger, 1989, 683p.	\$59.95
Analysis of the construction of ANOVA programs using least squares techniques, including the parsing algorithms by which a language specification is interpreted. Emphasizes the geometry as well as the algebra of the methodology. All programs in the book are included on 5 1/4" diskettes in APL, BASIC, C, and FORTRAN.	
APL2 AT A GLANCE , Brown, Pakin, Polyka, 1988, 444 p.	\$35.00
Solid, unimimidating, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started.	
LEARNING APL: AN ARRAY PROCESSING LANGUAGE , Mason, 1985, 259p.	\$21.50
Emphasis on arrays, naturally. A readable, detailed introduction with thorough examples.	
APL - ADVANCED TECHNIQUES AND UTILITIES , Berquist, 1986, 450p.	\$44.95
Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques. Assumes knowledge of the basics. Many idioms	
DISKETTE FOR ABOVE ,	\$15.00
Contains all functions described in text. Formatted for APL*PLUS/PC, uploadable to Sharp APL, VS/APL, APL2.	
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL: The first 5 years.	\$55.00
A wide range of topics in education and business. Logic, insurance, statistics, A. I., accounting, fractals, teachers toolbox, computer science, biology, graphics, engineering, data bases, and much more. 1983 - 1987, approximately 600p.	
APL89 - STATISTICS TUTORIAL , Alvord, Traberman, et al, 72p.	\$10.00
A unique collection of papers on statistics and its exposition in APL.	

Name: _____ Date: _____
Street: _____
City: _____ State: _____ Postal Code: _____
Country: _____ Telephone Day: _____ Evening: _____

Payment in check or money order payable in \$U. S. must accompany order. Send to:
Renaissance Data Systems
P. O. Box 20023
Park West Finance Station
New York, New York 10025-1510
(212) 864-3078

We regret that we cannot accept purchase orders or credit cards.

Title	Price	Quantity	Total
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

Subtotal: _____

Postage and handling - U.S. and Canada (\$2.50 minimum; \$2.50 each item over \$25.00): _____
- International (surface mail add 15%, airmail add 40%): _____

Total: _____

Thank You For Your Order!

Please send complete CATALOG of APL books and software ☐

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	IBM PCs & compatibles	poa	Includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,798	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
M.T.I.C.	386sx AT	2600+	Entry level 32-bit compact AT with DIALOG APL. 2MB ram, VGA, 40MB hard disk, 1.44MB floppy disk, 102-key keyboard, monitor, mouse, MS-DOS 4.01, WORKS and WINDOWS-386. Many expansion options available.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES (£)	DETAILS
REUTER:FILE	SHARP APL	poa	International Network application systems and public databases.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL
	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	378	IBM's APL 2 for the PC.

Cooking/Drury	APL*PLUS PC Rel 9	477	STSC's full featured APL for IBM's and compatibles - Version 9 offers the potential for larger workspaces using extended/expanded memory; full screen editing of numeric arrays; user command processor; point & edit etc. Upgrades to version 9 are available from Version 8 and earlier releases.
	APL*PLUS PC Run-Time	poa	Closed version of the Interpreter for developers, prevents user exposure to APL
	APL*PLUS II System	1600	High powered APL Interpreter for the 80386 chip. Price includes one years maintenance and free upgrades - volume discounts VERSION 3 NOW AVAILABLE.
	APL*PLUS II Developer System	poa	All the features of the APL*PLUS II System plus unlimited free run-times to enable developers to distribute their applications.
	APL*PLUS UNX	poa	STSC's 2nd generation APL for all major Unix computers and workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and oSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL for Unix Systems	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000 Level I		
	Mac, ST, Amiga, QL	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II		
	ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 9	450	
	APL*PLUS II V 3.0	1395	
REUTER:FILE	SHARP APL	poa	for IBM mainframes

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
General Software	Mellordata	poa	
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows. 80/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen. 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/5C APL	1395	14" colour monitor. 75 Hz refresh. 8 or 16 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Queen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC), 125(mf)	Including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
APL IMPETUS	Impetus	poa	Hierarchical Planning System

Cooking/Drury (for VSAPL)	ELEMENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRESII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
(for PC's)	FILEMANAGER	poa	Extends APL primitives to database management
	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)
	UCF	1800	Inter-user data transfer for VM users via IUVIC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercia	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC and 386. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.

	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management
	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 4	590	
REUTER:FILE			
	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	75	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - info centre product
Warwick University			
	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL8800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-160 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	150-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics In APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
REUTER:FILE	Consultancy	poa	Consultancy & support service world-wide.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.

Wickdiffe Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems
---------------------------	-------------	-----	--

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	228pp by Helzer
	APL in Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual (2nd Edn)	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	14.50	185pp by Thomson
	** Please add one pound packing charge per order **		
REUTER:FILE	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.
ISI	Tangible Math	\$19	An APL Approach to Math - Shareware, includes base Sharp APL
	J	\$24	Dictionary APL simplified and enhanced - Shareware (Mac,PC)
	SharpAPL/PC	\$74	Registered Shareware and Reference Manual
	ISI APL	\$99	Improved APL/PC - enhancements, performance, large workspaces
Renaisance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-482602
APL Software	David Allis Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-482602
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 860130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 081-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524

Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256 811125 Fax: 0256 811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
General Software Ltd	M.E.Martin	22 Russell Road, Northolt, Middx, UB5 4QS. Tel:081-864-9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 071-353 4212 Telex: 926604 HAMHSEG Fax: 071-353 3325
HRH Systems	Dick Holt	Box 4498, Silver Spring, Maryland 20904
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190
I-APL Ltd	Anthony Camacho	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-860130 for queries, order forms, bulk orders
	J C Business Services	58 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 081-747 0747
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131 Tel: (404) 992-8400
Intelligent Programs Ltd	Mike Bucknall	Unit 7, Hermitage Court, 6-10 Sampson Street, London E1 9NA Tel:071- 481-4813
ISI	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096
	Orders	3512 Cameron Mills Road, Alexandria, Virginia, USA 22305-1103 Tel:(703) 548-1799
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5098
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
M.T.I.C.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: Sunbury(09327) 80848
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW8 3ER Tel: 071-524 7013 (Answerphone) 0860-377953 (Mobile)
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0278-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
REUTER:FILE	Paul Jackson	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
The Rochester Group	Robert Pullman	50 S.Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716- 454-4841
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (608) 261-7538
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel:203-869-0131 or 212- 242-5816
Wickliffe Computer Ltd	Nick Teller	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-692588
Warwick Univ.	Prof.Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523369



ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА

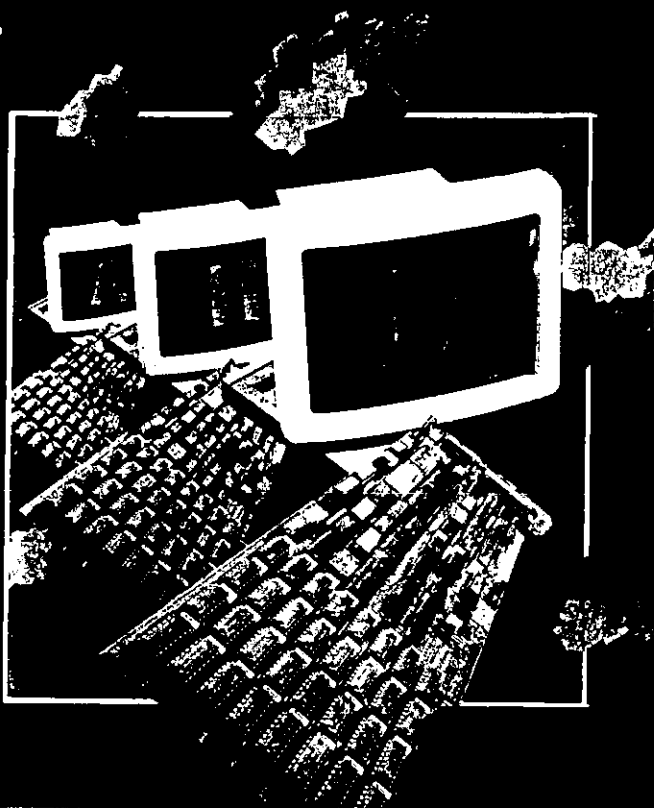
И ЕЁ ПРИМЕНЕНИЕ

Новое
в жизни,
науке,
технике

Подписная
научно-
популярная
серия

Издается
ежемесячно
с 1988 г.

Язык персональных вычислений



1990

4

Russian Magazine Devotes an Issue to APL

Translations by Peter Saunders

At APL 90 Alexei Miroshnikov and Andrei Kondrashev gave away copies of the Russian magazine that is nearest to Personal Computer World or Byte. The April issue is devoted to APL (40 or so of the 48 pages). We thought Vector readers would be interested to read some of this in English, and to see a couple of pages reproduced from the original.

Extracts from:

Computer Technology and its Application No.4, 1990

[Editorial] The World of the APL Language

No programming language appears to inspire as many disputes and mutually exclusive views as APL. The outstanding programming theorist, J.Martin, has pronounced on APL, "This is a super high level language, the only candidate so far as a fifth generation language". Another no less authoritative scholar, E. Dijkstra has called it "error brought to a state of perfection". In his book "The Mythical Man Month" F. Brooks remarked that if he had to rewrite the IBM 360 operating system he would start by modelling it on APL. In another publication someone miscalled APL "Chinese Basic".

APL is practically unknown in the USSR although the number putting it to practical use world wide has been estimated as hundreds of thousands. Following several years active and successful work with APL the authors of the article decided to write the brochure presented here so that Soviet Computer users could also discover and, we hope, come to love the amazing world of APL.

To love it - means to understand it...

Languages, Programmers and Programs

by V.M. Kuprianov [Reproduced in full]

"How poor our language! My ambition foiled,
The limpid storm cloud in my soul
Remains opaque to friend or foe." A.Fet

Machine languages are numbered in hundreds and more are written every day. Most widely used ones - FORTRAN, BASIC, COBOL - are neither user friendly nor linguistically structured.

The main cause of programming 'Tower of Babel' is the limitation of each language to a fairly small task set. Each new demand/application calls forth a new language so arguments about their respective merits are not well founded.

There are no good or bad languages: a programmer just picks the one most favourable to writing a good program in his field.

To 'What is a good program?' professional programmers answer 'one that works as fast as possible while using the least processing resources'. At the present stage of development that answer is unsound. When computing technology was expensive and limited, merit lay in using hardware almost as economically as possible with machine code. Today's situation quite different. The ratio of hardware productivity to expenditure on achieving it has multiplied a million times whereas programming costs rise continually. Parameters for assessing program quality have changed.

A good program is quick to write, easy to polish, reliable (no bugs), transparent to the customer, easy to modify. No traditional language scores many points. Some - PASCAL & MODULA - deliver reliability but not the paramount requirements - speed of writing and cheapness.

The programming backlog is growing faster than the number of programmers. Cheaper & faster hardware only increases the problem. With prevalent methods no conceivable number of programmers could soak up the work. If nothing is done [at current rate of growth] by the year 2000 the WHOLE labour force of the developed world will be programming. IT specialists estimate programming productivity must grow 50-100 times but in the whole history of computing it has grown 4-fold.

Must use more powerful languages and automated programming where possible. That doesn't mean end of PASCAL, SI[sic], & COBOL as high grade programmers (systems programmers) and traditional languages will be essential for writing the new languages' operating systems & data base systems etc. Applications programs are quite another matter to which mechanised automated programming & more powerful languages are ideally suited.

Lion's share of work will fall to end-user able ideally to program own applications. Machine language 'competence threshold' too high so languages for non-professional must become more powerful.

APL has all properties mentioned. Simple, easy for beginner, not at all primitive, ten times the productivity of FORTRAN or PL/1. APL programs an order or two shorter so that much cheaper.

Historical Perspective

by A.I. Miroshnikov

[An extended article giving the full history of APL. The following extracts cover the history in the USSR]

Late 70's Prof.P. I. Meadow of Toronto University brought two MCM/70's to Soviet Union and many took up study of APL.

End 70'S: impressed by APL on trip to Canada Academician Moiseev initiated development of first Soviet system, APL/BESM-6 by S.M.Prokhorov in the AN computing Centre of the USSR (BTsAN) and in 1978 in an Obrninsk institute the APL/VIDEO system for an ES EVM was introduced - adapted version of APL/360. Lead by E.T.Semyonov group of programmers started work on Soviet APL system in Moscow Energetics Inst.

1979: first use in Leningrad on powerful Cyber 172 machine by AN USSR computing centre.

1984: working on a system for processing images the Novosibirsk Computing Centre developed one of first APL parallel interpreters; IBM and IPISA presently working actively on APL systems for multiprocessor complexes.

1985: the AN USSR Computing Centre produced the first Soviet APL system for a minicomputer, the SM-4, which has since spawned a family of systems for minis, micros and PC's with the general name APL/VTsAN.

1986: first annual All-Union APL conference. A hitherto informal working group is being formalised to popularise APL in the USSR and coordinate developments using it.

1987: first Soviet APL machine built on basis of DVK and Elektronika 60M micros (v. Microprocessors and Systems 1989. No.4).

1988: First Soviet organisation devoted exclusively to production and distribution of APL products (the 'Dialog' scientific and technical cooperative); its latest product is the powerful APL/85 system for the 'Elektronika 85' PC.

Despite its 25 years existence APL is still considered a new language. The unique property of easy expansion conferred on it by Iverson enables it stay up to date and popular.

What is APL?

By A.V. Kondrashev

"There are many things that we don't understand, not because our understanding is weak but, because they don't enter the circle of our understanding."

K.Prutkov

APL allows one to code the specific structures of algorithms with the same ease as one can represent mathematical ideas with algebra. It is strikingly compact - average APL program length is 10 lines! Programming productivity is much higher than with compiled languages like PL\1 or PASCAL and other interpreted languages like BASIC. The programmer can concentrate on getting doing what he wants with the computer's help instead of on the problem of how to program it.

The APL Language and the APL System

The language is a formal notation for writing Information processing algorithms - pen and paper, chalk and blackboard can be used.

The system is a special program permitting communication with a computer in APL, conventionally called a dialogue integrated medium including all the computer controls a user needs, its own concept of dialogue organisation, and techniques for developing, polishing, saving and using programs.

Imagine that you have a clean piece of paper, a pencil, and a calculator. In the drawers of your desk are the notes, calculations, or drawings you have prepared earlier. On the bookshelf are textbooks and reference books. The daily environment in other words of the engineer, the scientific worker, the student and university or school. The APL system is constructed in just the same way with just one difference: the role of the piece of paper is played by a computer's memory and the sheet, called in APL the working area, can reach several megabytes. The keyboard fulfils the role of pencil and the screen is the communication link between system and man showing what is stored in the working area or the answer to a question. etc. etc... The APL user can always copy any program or array of data into the working area just as he would copy references, formulae and quotations onto his paper.

The APL system working area can hold as many program and blocks of data as its dimensions allow. Programs and data are quite independent of one another just as the same variable can form part of different mathematical formulae.

The laconic language makes communication with the system very productive as the time spent entering information is practically nil, very different from BASIC say which requires the programmer to expend no little effort on 'stuffing' even the simplest of programs. With the entry of only one line the APL system allows one to accomplish tasks that would demand tens or hundreds of lines in traditional languages.

..... [A 12 page tutorial on APL follows]

Here we finish this section where we hope to have given you the most general presentation of the APL language. The vast majority of its powerful tools have not been mentioned but one of the advantages of the language is that no matter what you do not know you can still work in it. No one uses all APL's facilities - it is inexhaustible...

APL in the Analysis of Data

by O.P. Lushka

[The article deals with use of APL in statistics ... here is a page from the original!]

чайных факторов можно применить прием группировки. Разобьем, например, массив скоростей на 3 группы по 8 чисел в каждой, а затем вычислим средние значения скорости и тормозного пути для каждой из этих групп. Процедура вычислений для первой группы такова:

```
(+/СКОРОСТЬ1[1 2 3 4 5 6 7 8])÷8
9.375
(+/ПУТЬ1[1 2 3 4 5 6 7 8])÷8
11
```

— На самом деле, мы могли бы не перечислять явно все восемь номеров, — продолжал Джонс, — а воспользоваться генератором индексов. Давайте подсчитаем и запоем все нужные нам средние значения.

```
C1←(+/СКОРОСТЬ1[8])÷8
P1←(+/ПУТЬ1[8])÷8
C2←(+/СКОРОСТЬ1[8+8])÷8
P2←(+/ПУТЬ1[8+8])÷8
C3←(+/СКОРОСТЬ1[16+8])÷8
P3←(+/ПУТЬ1[16+8])÷8
```

— Теперь мы можем построить сводную таблицу результатов, — сказал программист, закончив работу.

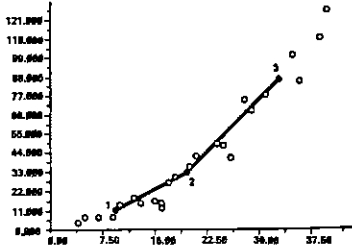
	(C1, C2, C3)	и P1, P2, P3
9.375	11	
19.625	32.5	
33	86.125	

— А если нас интересует средний тормозной путь автомобиля, имеющего скорость 11? — спросил Смит. — Как его подсчитать?

— Простейшее решение этой задачи — графическое, — ответил Джонс. — Нужно соединить прямыми линиями три точки, координаты которых указаны в строках полученной нами таблицы. Это легко можно сделать с помощью программы ЛИНИЯ.

линия (P1, P2, P3) от C1, C2, C3

— Эта ломаная линия показывает уже непрерывную зависимость между скоростью автомобиля и его тормозным путем. Да и тех противоречивых результатов, что были раньше, здесь уже нет, — закончил программист.



RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Thanks to Nic Small and Dave Piper for such comprehensive notes on their talks at the last British APL Association meeting of the 1989/90 season!

Thanks to the 'Vector Team' of Anthony Camacho, Sylvia Camacho, John Searle, and Ray Cannon for their sterling work at APL90. As usual the notes, analysis, and photography are first-class.

Packaged Workspaces in APL2

by Nicholas Small, National Westminster Bank

Introduction

Packaged workspaces were introduced into APL2 in release 1.3, which became available about the beginning of 1988. This release was only installed at NatWest in the middle of 1989 - which explains why this paper was not written earlier.

For the benefit of those who have heard of packaged workspaces in APL2/PC, it should be pointed out that this is something completely different! Rather than being a run module, as are packaged workspaces in APL2/PC, packaged workspaces in APL2 closely resemble in functionality the subroutine libraries that are so useful in other languages.

In many ways, the most important property of the packaged workspace facility is that it provides a way of creating libraries of utilities that can be made available to all workspaces without the need to copy functions into the workspace at run time, and without having to worry about name conflicts for global variables or secondary functions. Packaged workspaces also offer a means of concealing from users all but the objects that they need to know about, and can produce savings in CPU time and in workspace requirements.

Certain details in what follows are specific to the MVS/TSO environment, though the general principles apply equally to VM/CMS.

Name Association

Access to objects in a packaged workspace is via the `⌈NA` function which was introduced in APL2 release 1.2 to provide a means of using code written in languages other than APL via (Associated) Processor 11. When a workspace is packaged, variables, functions and operators may all be made accessible to other workspaces, as required. Once accessed, such objects, which may be referred to as external objects, can be used in the normal way.

There are three external functions supplied for use with packaged workspaces, viz. `PACKAGE`, `EXP` and `QNS`. To get hold of these (or other supplied external functions) one needs first to have allocated the distributed NAMES file `APL2.NAMES.AP2TN011` to the ddname `AP2TN011`. Then, to access, say, `PACKAGE` requires the command

```
3 11 □NA 'PACKAGE'
```

which should return the value 1.

In the right argument of □NA, it is possible to supply an alias, by which the object will be known in the associating workspace, to avoid a name conflict. For example, there is a supplied routine *OPTION* which can be used to query or set invocation options; if it is desired to use this function in a workspace that already has an object with this name, then it can be associated as, say, *IOPT* by using the command

```
3 11 □NA 'IOPT OPTION'
```

To find out what the name association of an object is, use □NA monadically:

```
□NA 'IOPT'  
3 11
```

The result is the left argument used with □NA when activating the object, if it is an external object. For objects which are not external, the first element of the result is the name class of the object and the second element is 0.

When a workspace is saved, sufficient information relating to any name associations is stored to enable re-accessing of the respective objects when they are first referenced after the workspace is reloaded.

Packaging Workspaces

Suppose that one wishes to package a workspace called *MONTHLY* which is an end-user application with only two functions that need to be accessed from the command line: *START* and *RESTART*. The workspace must be saved in a SAM library - the technique does not work for workspaces stored on VSAM files. One must first allocate a file, *TEST.OBJ*, say, to ddname *SYSPUNCH* with a command like:

```
)HOST ALLOC DD(SYSPUNCH) DS(TEST.OBJ) NEW SP(5 5) TRA LRECL(80)  
BLKSIZE(3200) RECFM(F B)
```

If the external function *PACKAGE* has been associated as described above, then the command

```
'START' 'RESTART' PACKAGE 'MONTHLY'
```

will cause an object deck to be placed in the file TEST.OBJ; this filename is returned as the result of *PACKAGE*. Finally, the object deck must be link-edited into a load library using a command of the form

```
)HOST LINK TEST.OBJ LOAD(LOADLIB(MONTHLY)) RMODE(ANY)
```

which will create a member MONTHLY in the partitioned dataset LOADLIB. (The parameter RMODE(ANY) should only be used under MVS/XA or ESA.)

In the present example, the list of items to be accessible, which forms the left argument to *PACKAGE*, has only two items, which is quite typical when packaging an entire application that is menu driven. In other cases, for example when one is packaging a utility workspace, the list of items to be accessible may be quite long. In any case, it is good practice to have this list stored as a variable, *NAME_LIST*, say, in the workspace. (If all objects are to be accessible then the left argument to *PACKAGE* may be omitted.)

For best results, before packaging a workspace, do a *)COPY* of the saved workspace into a clear workspace and *)SAVE* again. This causes the workspace to be compacted, clears the state indicator, and removes any traces and stops.

Accessing Objects in Packaged Workspaces

Objects in packaged workspaces are accessed by using *□NA*, which takes as right argument the list of objects to be accessed. The left argument of *□NA* has a number of possible forms, allowing a variety of ways of specifying how and where the system should search for the specified objects.

Continuing with the example above, one convenient way to access the objects in the packaged workspace requires that one first allocate the load library LOADLIB, with ddname, say, AP2LOAD. Then to access *START* one may use the command

```
'AP2LOAD.MONTHLY' 11 □NA 'START'
```

MONTHLY being the name of our object module in LOADLIB. Both functions may be accessed in one command by using

```
(<'AP2LOAD.MONTHLY' 11) □NA" 'START' 'RESTART'
```

The value 1 will be returned by `□NA` for each object found and associated, 0 for those not associated. Assuming that the load library has been properly allocated and that the correct object module has been specified, the most likely reason for failure of the association request is that the specified name is already in use in the current workspace for an object that is not associated, or was associated using a different left argument to `□NA`.

Another form for the left argument of `□NA` is that described in section 2, i.e. the left argument is: 3 11. In this case, one indicates where the code is to be found using a NAMES file; entries are as used when employing `□NA` to access code written in another language, except that one must add the tag link:APL. See the manual APL2 Programming: System Services Reference for further details.

The use of a NAMES file is perhaps somewhat more robust than locating code by allocating the load modules to a particular ddname, but it is often less convenient and is certainly more restrictive, as object names are required to be alphanumeric, and, in MVS, names are limited to eight characters; also, object names must be unique.

Name Scopes

From what has already been said about the avoidance of name conflicts, it will be apparent that the active workspace and each packaged workspace knows about only those objects which are local to itself, i.e. are within the name scope of the particular workspace; for an external object, i.e. one that has been associated via `□NA`, this knowledge is restricted to pointers to the object's "home" name scope.

The active workspace is referred to as the primary name scope, because, of course, this is where any APL session will start. As soon as an object declared via `□NA` is encountered during execution, the system changes to the name scope from which that object comes and takes from there the value or definition of both the object and any objects that it may reference. If the object is a variable then its value is returned (or, if it is a shared variable, the corresponding auxiliary processor is accessed) and the calling name scope becomes current again. If the object is a function or operator then this secondary name scope will remain active either until execution of the function or operator terminates or until an object in another name scope is encountered. If execution suspends in a secondary name scope, then that name scope remains current and all and only those objects in that name scope are directly available to the user. To revert to the primary name scope use `)RESET`.

A change of name scope that occurs as a direct result of a call to an external function or operator (i.e. one declared using `□NA`) is referred to as an explicit change of name scope. This contrasts with the implicit change of name scope that occurs during the execution of an externally defined operator called with a defined function operand: the call to the operator causes an explicit change of name scope, but if the operator references its operand then there will be an implicit change to the name scope from which the operator was called, for that is where the function is defined.

For example, suppose we have an external optimizing operator *MINIMAX*, which takes as left argument the function to be optimized and as right argument various parameters. A call to *MINIMAX* will be of the form

```
R ← FUN1 MINIMAX PARMS
```

Initially, execution will be in the name scope from which *MINIMAX* comes, but as soon as the function operand is encountered in the definition of *MINIMAX*, *FUN1* will be invoked in its own name scope, causing an implicit change of name scope.

Accessing Objects in Other Active Name Scopes

As soon as one thinks of packaging utility functions or operators one discovers the need to access objects in other active name scopes. For example, an external workspace documentation function needs to work on the objects in the calling name scope, not in its own name scope; also, some utilities call others, e.g. for screen display and error handling. It is apparent that, unless one packages all utilities in a single workspace, it is quite possible for there to be objects from a number of name scopes on the stack, separating the primary name scope from the current name scope.

There are three ways of getting hold of objects in another name scope.

The obvious way is by using `□NA`. An object *X* in the primary name scope can be made accessible to the current name scope (assumed different from the primary name scope) by using the command

```
' ' 11 □NA 'X'
```

where the null string indicates the primary name scope. This should return the value 1.

To assist in using `UNA` like this, there is a supplied function `QNS` (Query Name Scope) which when executed in a name scope will return as its result the appropriate left argument for `UNA` to associate with an object in that namespace. `QNS` is always used with a right argument of 0.

For example, suppose we have a packaged utility `UTIL` which will perform some function on objects not necessarily in the primary name scope, and that `UTIL` takes as its right argument a variable `NAMESCOPE` which identifies the calling name scope. Then `UTIL` can be called by

```
UTIL (QNS 0)
```

and within `UTIL` there might be the lines

```
NAMESCOPE UNA 'NL UNL'  
VARS + NL 2
```

The first line makes use of the alias facility of `UNA` to get hold of the system function `NL` in the referenced name scope and the second will put into `VARS` a list of the variables there.

A second way to access objects in another active name scope is via the use of an externally defined operator, as described in the explanation of an implicit change of name scope in the previous section. The function operand (or operands) will execute in the name scope that called the operator. For example, if `NL` is used as such an operand, it will provide lists of objects in the calling name scope. It is apparent that this technique cannot readily be employed to get hold of objects in name scopes further away than the calling name scope.

The third technique is to use the supplied external function, `EXP`, which allows one to execute various forms of expression in the previous name scope. (Perhaps there should be a Vector competition to provide an explanation why `EXP` is a better acronym than `EPNS`.) `EXP` is called with a nested vector right argument and usually returns a result. The simplest form of usage is

```
RESULT + EXP c'string'
```

which will cause "string" to be executed in the previous name scope; any result (including the result of an assignment) will be returned as `RESULT`. For example, consider

```

EXP < 'VAR'
EXP < '[]CR' 'FUN1'
EXP < 'FUN2'
EXP < 'TEMP + 0'
EXP < 'QNS 0'

```

The first of these will return the value of the variable *VAR*; the second will return the canonical representation of *FUN1*; the third will execute the niladic function *FUN2* and return the result, if any; the fourth will assign the value 0 to *TEMP* and return this as result; the last example will return the identity of the previous name scope (assuming that *QNS* has been associated there).

To use *EXP* to execute a function in the previous name scope using arguments from the active name scope the syntax is

```

EXP 'FUN' RARG
or EXP LARG 'FUN' RARG

```

according as the function *FUN* is monadic or dyadic, where *LARG* and *RARG* are values in the current name scope. For example, the second of the previous examples can be replaced by

```
EXP '[]CR' 'FUN1'
```

Finally, *EXP* can be used to assign a value to a variable in the previous name scope using the form

```
EXP 'VAR1' '+' VAR2
```

This will give to the variable *VAR1* in the previous name scope the value of the variable *VAR2* in the current name scope. Thus the fourth of the earlier examples can be replaced by

```
EXP 'TEMP' '+' 0
```

It is best to shun the first form of usage of *EXP* if one of the others can be employed, in order to avoid a proliferation of quote marks.

A call to *EXP* in the primary name scope will be executed there, so most functions that have been prepared for use from a packaged workspace will work properly when used with their definition in the active workspace.

Note that *EXP* causes an implicit change of name scope.

Error Trapping Example

In this section we shall consider what has to be done to package as utilities the functions used in a simple error trapping application. The purpose of the trapping procedure considered is not to provide any fancy corrective action; it simply displays on the screen an appropriate message when, say, a syntax error is encountered or the user presses attention, and then allows the program to restart at the main menu. Similar methods may be used when more sophisticated error trapping techniques are being employed.

The application workspaces in which the error trapping is used may be considered to have a latent expression which calls the function *START*. This will perform all initialization and then call *RESTART*, which brings up the main menu. Three other functions are also important in the present context:

PFK, which when activated (in immediate execution mode) by the pressing of PF key 4 causes *RESTART* to execute;

STACKSET, which creates an input stack for use when the keyboard opens for immediate execution after an interrupt has occurred;

RAWAPL, which retrieves and displays any error message.

In the original version of the workspace, the last two functions were as follows:

```

V STACKSET;A;STACK
[1]  A Prepare stack for attention exit
[2]  STACK+'(FIFO'
[3]  A+101 □SVO 'STACK'
[4]  STACK+0 A Clear the stack
[5]  STACK+')SI'
[6]  STACK+'ΔERR+□ET □EM' A Save event details
[7]  STACK+')RESET'
[8]  STACK+'RAWAPL 1'
[9]  STACK+'PFK 4'
V

```

An execution stack is set so that if an interrupt occurs then (starting with line 5): the state indicator is displayed so that there is a record in the session log; the event type and event message are saved in variable *ΔERR*; the state indicator is cleared; *RAWAPL* is called to display the event message; finally, the program is restarted by the call to *PFK*.

```

V RAWAPL X;M;T;ΔEM;ΔET
[1]  A Entered if we get into raw APL by any mischance
[2]  A Global variable ΔERR holds event type and message
[3]  T←'An abnormal event has occurred'
[4]  (ΔET ΔEM)+ΔERR
[5]  →('ATTENTION' 'INTERRUPT'=2+(' '*ΔEM[1;])<ΔEM[1;])Δ0
[6]  →Δ1,pM←c'You hit attention.'
[7]  Δ0:M←c'Something strange has happened'
[8]  →(0εpΔEM)+Δ2
[9]  ΔEM→Δ' ((' (Error type is ',(ΔET),') ')) '
[10] Δ2:M←M,c[2]' ',[1]ΔEM
[11] M←M,(c''),c'Please report this to IT Systems Support'
[12] Δ1:M←M,(c''),c'Hit PF 3 to return to the primary panel'
[13] A←T SCREENΔDISPLAY ΔM
V

```

Function *SCREENΔDISPLAY* is a utility which displays its matrix right argument under the title string in the left argument. At line 4 the event type and message are retrieved. The message to be displayed is then built up as a vector of vectors, with special cases if Attention was pressed or if the event message is empty.

The whole of the original application workspace can be packaged as a single unit without changes, as described earlier. We now consider what alterations have to be made to the functions just described if it is desired to package the utility functions from the application in a number of separate workspaces. At the very least we shall have separate packages for screen-handling, file-handling and miscellaneous utility functions. *RAWAPL* will go into the last of these and we shall consider that first.

The only reference in *RAWAPL* to the environment that called it is at line 4 where the variable *ΔERR* is referenced. If we replace this line by

```
[4] (ΔET ΔEM)+EXP c'ΔERR'
```

then all will be well: the value of *ΔERR* will be retrieved from the calling name scope.

Consider now what is put on the stack in *STACKSET*. The *)RESET* is performed before the call to *RAWAPL* in case the interrupt was caused by *WORKSPACE FULL*. Unfortunately, if the interrupt occurs in a packaged workspace, the variable *ΔERR*, in which we save the event type and message, will be lost when the *)RESET* returns us to the primary name scope. What is required is a function

to repeatedly place the variable *ERR* in the previous name scope, until it finishes up in the primary name scope.

Such a function is the following:

```

      V TNS MOLE PARS;X
[1]  A Do an EXP on PARS, and burrow further down if this is
[2]  A not the target name scope TNS
[3]  A Requires MOLE, EXP and QNS at all levels
[4]  X+EXP PARS
[5]  +(' 11=X+EXP 'QNS' 0)/0 A Quit if bottom or
[6]  +(TNS=X)/0 A target level accessed
[7]  EXP TNS 'MOLE' PARS
      V

```

We will need to make the following call (to be put on the stack at line 6.5 of *STACKSET*):

```
( ' 11) MOLE 'ΔERR' '+' ΔERR
```

Then, when *MOLE* executes, line 4 will put a copy of *ΔERR* in the next earlier name scope, lines 5 and 6 check to see if we have finished, and line 7 repeats the call to *MOLE*, in the previous name scope. As indicated in the comment in line 3, *MOLE* has to exist already in each name scope that it accesses, as do *EXP* and *QNS*. It is possible to write the function to perform from each name scope a *□NA* in the previous name scope to ensure that all required functions are there, but that seems an unnecessary complication. It certainly appears sensible to have *EXP* and *QNS* available in all workspaces that make use of packaged functions.

Having got this far, the next step is to package *MOLE* as, say, *MOLE2*:

```

      V TNS MOLE2 PARS;X
[1]  A Do an EXP on PARS, and burrow further down if this is
[2]  A not the target name scope TNS - packaged version
[3]  A Requires MOLE2, EXP and QNS at all levels
[4]  X+EXP 'EXP' PARS
[5]  +(' 11=X+EXP 'EXP' ('QNS' 0))/0 A Quit if bottom or
[6]  +(TNS=X)/0 A target level accessed
[7]  EXP 'EXP' (TNS 'MOLE2' PARS)
      V

```

All that is required is another layer of *EXPs*, so that what was executed in the current name scope from *MOLE* is now executed in the previous name scope from *MOLE2*. The name of the function called in the extra line of *STACKSET*

should be changed to *MOLE2*, and *MOLE2* should be associated as an external function in all packaged and application workspaces.

Finally in this section, we consider the function *PFK*. The chief feature of *PFK* is that it causes *RESTART* to run when function key 4 is pressed; the effect of other function keys may vary from application to application. When testing with error trapping not activated, it is useful to be able to restart the application by pressing PF4, even when one is not in the primary name scope. This objective can be achieved by having, in each utility packaged workspace, a function *PFK* which resets the state indicator and then runs *PFK* in the primary name scope with the same parameter (key number) as that with which it was called. This is done by the following:

```

      ▽ PFK N;A;STACK
[1]  * For use in packaged utility w/s only
[2]  * Reset SI and run PFK N in the original active w/s
[3]  +(' ' 11=QNS 0)/0 * Quit if in primary name scope
[4]  STACK+('FIFO'
[5]  A+101 ▯SVO 'STACK'
[6]  STACK+0
[7]  STACK+(')RESET'
[8]  STACK+('PFK ',▽N
      ▽

```

Line 3 is there to avoid accidents when the utility workspace is the active workspace. This function can be packaged without modification.

Details of Operation

On page 51, it was explained how to put into a load library the load module resulting from the packaging of a workspace. It is also possible, with the assistance of a system administrator, to have the module placed in the MVS link pack area, from where the code can be shared by a number of APL users. When the code is not shared, on the first occasion that an object in a packaged workspace is activated, the packaged workspace is loaded by Processor 11 into the free space in the APL user's address space. If a saved workspace containing name associations is loaded, then the corresponding packaged workspaces will not be loaded until the respective objects are first accessed.

Functions or operators in the packaged workspace will not consume space in the user's workspace unless their definitions are modified during the session. However, any variables in the packaged workspace that are referenced or specified during the session will take up space in the user's workspace, as will

any newly defined objects in the name scope of the packaged workspace. If a `)SAVE` command is issued, then such variables and objects will be saved with the user's workspace (the packaged workspace itself is not saved). Consequently, to avoid loss of storage space, loss of workspace, and slower loading, the following sequence of commands is recommended when saving an application workspace:

```
)RESET  
)SAVE TEMP  
)CLEAR  
)COPY TEMP  
)WSID xxxx  
□LX ← 'yyyy'  
)SAVE
```

where the workspace identity and the latent expression are set appropriately. Performing a simple `)SAVE` may be convenient during testing, but to use such a method to save changes made to objects in a packaged workspace as part of an application is clearly a very poor technique from a theoretical point of view. It is also bad in practice, because if a packaged workspace is modified then the next time that an application accesses it all such changed objects will be lost.

If a packaged workspace has been changed since an application workspace was last saved, then an informatory message is displayed the first time that each associated object is accessed.

Debugging

The chief difficulty in debugging is that, for some reason, stop controls are not honoured in packaged workspaces. Therefore, if tracing does not provide the necessary clues, it may be necessary to introduce a deliberate syntax or value error in the code to persuade the packaged workspace to stop at the desired point. In many applications it will be possible to use Attention during execution to gain access to the name scope of a packaged workspace; in more difficult cases it may be most convenient to have a special function, containing an error, that may be called to force access to the name scope of the packaged workspace.

Using such techniques it is possible to produce local modifications to functions in the packaged workspace, which can be stored using `)SAVE`. However, because of the way that `)COPY` works, it is not possible to transfer objects saved in this way to the utility workspace for repackaging, unless one resorts to trickery to define the desired objects in the primary name scope before saving the workspace.

Frequently, when some use of *EXP* does not have the desired effect, the resulting error message is not very illuminating (as is usual with external routines) as errors in any functions executed are reported against *EXP*. A bit of head-scratching will often be more effective than trying variations on the original theme.

The best piece of advice for dealing with packaged workspaces is probably: Keep it simple.

Advantages and Disadvantages of Packaged Workspaces

The disadvantages of packaged workspaces are relatively minor:

- (i) There will be a need to keep two copies of functions and operators that require modification when they are packaged.
- (ii) Debugging is more difficult, but problems with this at the program development stage can be avoided by getting things working first in unpackaged mode, and then making any modifications necessary for packaging.
- (iii) When a utility workspace is repackaged, unless accessing workspaces are resaved, there will be informatory messages displayed when objects are accessed.

On the other hand:

- (i) Packaging workspaces is an excellent way of providing a library of utility functions.
- (ii) The writing of workspace documentation functions, and other utilities that need global variables and secondary functions, is greatly simplified by the absence of the need to avoid name conflicts.
- (iii) The number of objects in most application workspaces will be reduced because secondary functions for screen handling, file handling, etc., will be hidden in packaged workspaces.
- (iv) Packaged workspaces may also prove a useful place to store such items as printer identifiers, that have previously been kept on a file.
- (v) Considerable savings in workspace requirements can be achieved, especially if packaged workspaces are placed in the MVS link pack area; it appears also that there are some cpu savings to be had, independent of any resulting from a larger free workspace size.

Packaged Workspaces and their Implications for Application Design

by Dave Piper (Cocking & Drury)

Abstract

This paper concentrates on the implications that the use of packaged workspaces have for the design of APL2 applications. It begins by describing what packaged workspaces are, how they are produced and how they can be used. The paper then considers what a packaged workspace is in terms of application structure, and the impact on the design of APL2 applications.

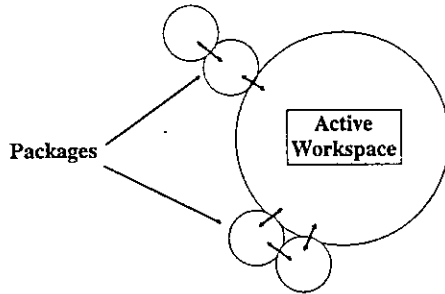
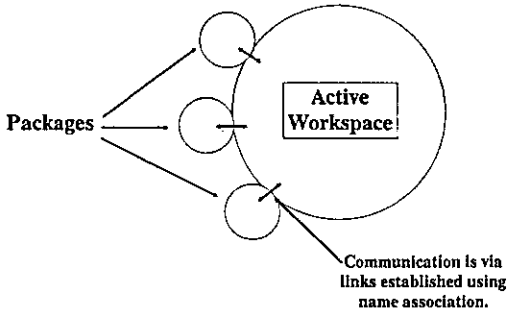
Acknowledgement

I would like to acknowledge the opportunity presented by the Life Division of Eagle Star in facilitating the research for this paper; also for subsequent permission to publish the results to the APL community at large in the form of this paper and a presentation at the June 90 meeting of the BAA.

Introduction

Packaged workspaces are a new feature of APL2, introduced in APL2 Version 1, Release 3. The concept of the packaged workspace allows us to invoke APL2 functions (or APL2 objects in general) without having them reside in the active workspace. Of course, the objects have to live somewhere - they live inside the package. Packages can be used in a variety of "configurations", some of which are illustrated on the following page.

The links to a packaged workspace can be tightly controlled by establishing a limited number of access points - all other objects in the package are inaccessible to the active workspace. This adds a degree of security and integrity to the application code. Objects within the package are accessed using the name association (`⌈NA`) facility of APL2.



There are a number of advantages in the use of packaged workspaces:

- APL2 code can be shared easily amongst many users.
- Code can be loaded in shared memory, saving virtual storage.
- Utilities can be distributed without the usual difficulties.
- Utilities can be updated without the need for) *COPY*.
- Applications can be made more modular.
- Code is relatively secure.
- Access to a package is faster than use of) *COPY* or) *LOAD*.

All these features make the use of packaged workspaces quite an attractive proposition. IBM have done a good job of making the packaging process as

simple as possible; in order to encourage the use of packages, access to packaged code is as transparent as possible.

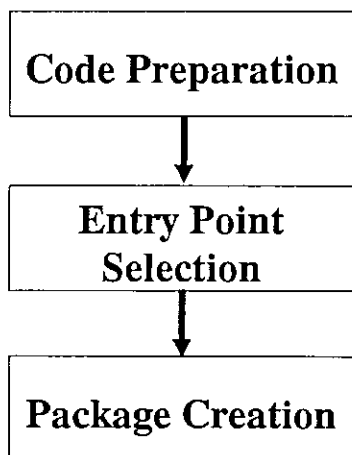
Of course, as with all powerful new facilities, in order to make really effective use of packages and to obtain many of the resulting benefits a significant amount of work may be needed - there is no such thing as a free lunch.

**There is no such thing as
"A Free Lunch"
or even
"A Free Packaged Lunch"**

The Packaging Process

The process of creating a package occurs in three stages:

- code preparation
- selection of "entry point" functions (objects)
- creation of the package



Each of these steps is described in more detail below. Once the package has been created, it remains for the programmer to arrange access to the package before it can be used.

Code Preparation

It is necessary to undertake a series of code checks before commencing the packaging process. Typically these are the same as the actions needed to ensure code is "productionised" before it is distributed as an application:

- removal of all stops and traces
- de-commenting to save space
- localisation of all possible variables
- locking of functions

These actions will ensure that the execution of code is never halted (for example by an error) within the packaged portion of the application.

Selection of "Entry Points"

The number of linkage points between the packaged workspace and the active workspace can be strictly limited. This allows the designer to ensure that the facilities provided by the package are accessed and used only through the correct channels. In order to make use of this capability, he has to decide in advance which objects in the package will form the entry points.

The entry points are included in the specification of the package during the packaging process. Only these objects are available to the user of the packaged workspace - all other objects in the package are invisible to the active workspace and cannot be accessed.

Package Creation

Once these matters have been dealt with, the actual packaging process can proceed. This happens in two stages:

- production of an object module
- link editing of the object module to create a load module

Object Module Creation

IBM supply an associated function (*PACKAGE*) to assist with the first stage - this associated function is actually a packaged workspace in its own right. The object module is created quite simply; the contents of the workspace are re-written into the object module practically unchanged - a few header records are inserted and record headers are also added. The essential (binary) contents of the workspace are left unchanged.

The Package function is ambivalent. For its right argument, the name of the workspace dataset is specified. Under MVS/TSO, the package facility will not process workspaces stored in VSAM libraries. The need to specify the name of the dataset rather than the workspace identity seems rather strange since (in the TSO environment at least) it is possible to deduce the dataset name from the workspace identity.

The left argument, if supplied, gives the names of the "entry point" objects for the packaged workspace. If this argument is elided, all objects in the package are accessible from the active workspace. As long as some care is taken implementing the package, it is best to name specific entry points rather than allowing all objects to be accessible.

Link Editing

Once the object module has been created, it can be link edited using the standard link editor to create a load module. Be careful at this stage - the MVS link editor will not accept input files with a blocking factor greater than 40; this is not documented in the APL2 documentation, only in the documentation for the link editor. The package facility does not detect this error - so you may end up having to re-create your package from scratch. An IBM internal publication [IBM89] gives lots of useful hints about optimising the load module; especially in the VM/CMS environment.

Characteristics of Packaged Applications

It seems apposite at this point to ask the question:

"What should we package?"

We can start to answer this question if we look at the characteristics of the packaged environment.

Packages live in the freespace allocated to the APL2 session, but are not limited to the amount of freespace memory requested at invocation. If necessary, a package can use the active workspace for its work. However, it is probably a good idea to ensure that things we package are conservative in their use of memory.

Secondly, links between the active workspace and the package have to be set up (using `⌈NA`) in order to be able to use the packaged objects. To simplify this initialisation we should try and minimise the number of "entry points" in the package. This has the beneficial side effect of improving the security and integrity of our packaged code.

Thirdly, objects which are not defined as "entry points" to the package are hidden from the active workspace. This allows variables which would normally be global to the application as a whole to be hidden from view. The active workspace cannot update these objects except through the defined entry points.

Fourthly, packages make it very easy to distribute code to many people, and to ensure they are making use of any updates to the code which may be provided. This makes the packaged workspace an ideal delivery vehicle for utility functions and groups of utility functions which may need to be accessed many times and in many different circumstances.

What Should We Package?

Given the above characteristics of the package environment, we should be able to identify the best parts of an application for packaging. In order to match the above characteristics, we should look for:

- self contained units of code
- a relatively small number of objects involved
- conservative use of workspace
- a small number of "top level" functions

Utility groups normally display these characteristics and form ideal candidates for packaging. Consider the command driven QSAM file interface described in [PIP87]; this forms an ideal candidate for distribution as a package. About the only complexity in this facility is that some functions are fixed dynamically - this will only happen once for a given application. In all other ways it meets the above requirements.

Design Implications

In order to be able to make some statements about the design implications of packages, we first have to decide what a package is in terms of application design. The important unit of code in application design is the module. If we make the assumption that a package is a new form of module, then we also ought to check how good a module it is. To make the use of packages in our applications worthwhile, they must not force us to sacrifice quality.

Let us take a step back a moment and talk about module quality. Two major measures of module quality have been established - these are discussed in detail in many publications (for example [PAG88], [CAD90]); so I shall only describe them briefly here.

Module Coupling

Module coupling looks at the quality of a module in terms of how it links to other modules. A range of coupling types has been identified, from DATA coupling (best case) to CONTENT coupling (or an even better name PATHOLOGICAL coupling) - worst case. In summary, the types of coupling are as follows:

DATA	all data passed as simple items in parameters
STAMP	all data passed in parameters, but use of local data structures
CONTROL	use of flags to control execution
COMMON	use of global variables
CONTENT	one module modifying the code in another

DATA, STAMP and CONTROL coupling are more or less acceptable, although design problems may still result. COMMON and CONTENT coupling are much less acceptable, although many of the worst effects of common coupling can be ameliorated by careful design and documentation. CONTENT coupling should be avoided at all times.

Module Cohesion

Module cohesion looks at the quality of a module in terms of how its various components link together. As with coupling, a range of cohesion types has been

identified, ranging from FUNCTIONAL cohesion (best case) to COINCIDENTAL cohesion (worst case).

Cohesion can also be thought about in terms of colours - although we are dealing with a monochrome display adapter since it's all shades of grey! Functionally cohesive modules can be likened to black boxes, coincidentally cohesive modules are white (or transparent) boxes. The other grades take intermediate shades of grey. The types of cohesion are:

FUNCTIONAL	One complete task
SEQUENTIAL	Most of a task, data transformed by each stage
COMMUNICATIONAL	Most of a task, data reused by each stage
PROCEDURAL	Most of a task, not related by data but order dependent
TEMPORAL	Many tasks determined by time
LOGICAL	One task selected from a "subject" related list
COINCIDENTAL	Many tasks bundled together for the sake of it

The top three types of cohesion are widely acceptable with an application. In practice you will nearly always find modules with procedural and temporal cohesion too. These are less acceptable and should be eliminated where possible. Logically and coincidentally cohesive modules should be considered unacceptable.

In addition to the levels of cohesion mentioned above, there is another level which is only implementable in some languages. INFORMATIONAL cohesion describes a module with multiple entry points - that is to say a module which can be called by referencing addresses inside the module itself.

This is different from LOGICAL cohesion since there is no flag involved in the selection of the operation chosen - all the selection is achieved by name. Each entry point can be considered to be a module in its own right, the only reason for using such a module is the benefits bestowed by the bundling together of the various operations. Usually INFORMATIONAL cohesion is classed as lying between FUNCTIONAL and SEQUENTIAL cohesion.

Modules Within APL

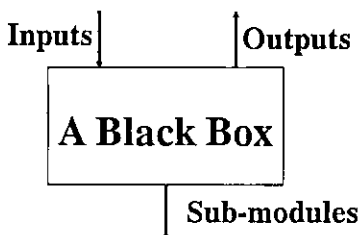
The main form of module within APL is the defined function. If correctly designed, defined functions should be of functional cohesion and should only use data coupling. Other types of module do exist in APL:

- diamonded lines of code
- use of the execute primitive function

The Module



The fundamental unit of design



Modules in APL:

Defined functions

Diamondized lines, Executed code

The package?

Both these forms usually show much less cohesion and much more coupling than the defined function. For example, a diamondized line of code can only be common coupled to other lines of code since there is no way of implementing local variables within a single line. Similarly, executed statements will frequently be content coupled with other lines of code - if part of the executed statement is decided by the value of local variable for example.

We are left to decide if a package is a suitable candidate for a module, and if so how good can it be in terms of module quality?

The Quality of Packages

To paraphrase the immortal bard: The quality of packages is not strained.... in implementation terms, there are two main ways to view a packaged workspace:

- as a series of modules bundled together
- as a single module with many entry points

The interpretation we employ will depend on how the package itself is defined. Consider a package which presents a series of low level utility functions to the user; these are unrelated to each other except by virtue of being in common use. In this case the package should be considered to be a series of modules simply bundled together for convenience.

Alternatively, the package may present a series of utilities all associated with the same aim (e.g. QSAM file processing under TSO). In this case the package is best considered to be a single module with several entry point functions. The important conclusion is that the quality of the package as a module depends on how well we design the package itself.

Package Coupling

The coupling between packages and the outside world can be as strong or as weak as we wish to make it. By failing to name entry points to the package, we make every object potentially available for modification - this will typically lead to a COMMON coupled package (or even worse, content coupling). If we make use of the facilities provided to name specific entry points to the package, we can ensure a lower level of coupling.

In the best case we can, by avoiding all references to variables during the packaging process, ensure that the module is at worst CONTROL coupled. This will avoid all the problems normally introduced by the use of COMMON coupling.

Package Cohesion

As with coupling, we can embody any level of cohesion we like into a package. For a loose assemblage of functions, the level of cohesion is likely to be either COINCIDENTAL or LOGICAL. In a well designed package, with several entry point functions, we should be able to ensure INFORMATIONAL cohesion - the separate packaged functions fulfilling the role of entry points in more traditional languages.

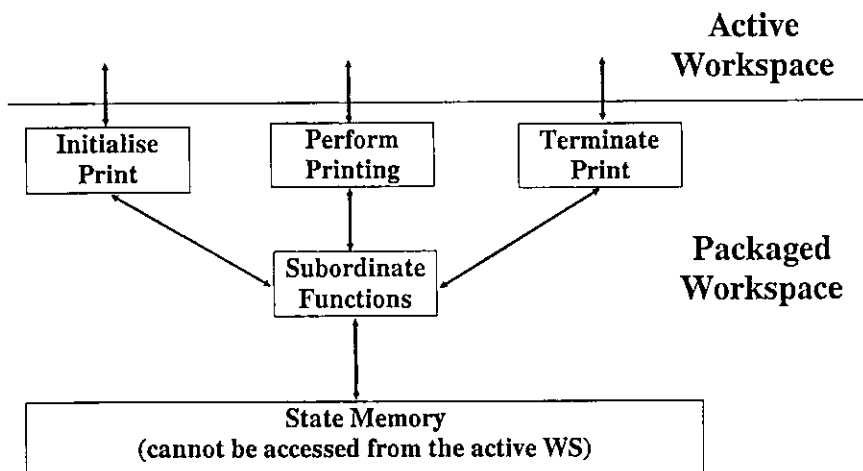
In the extreme, where our package only provides a single entry point function, FUNCTIONAL cohesion can be achieved. It may be desirable to package a single function if it makes use of significant numbers of global variables as "state memory" between invocations. Several sub-functions will probably have to be packaged also, these do not have to be entry point functions.

Packages and Informational Clusters

One problem that frequently has to be solved in an APL application is that of implementing an "informational cluster". This is simply the situation where a set of functions requires a "memory" between invocations. Each of the functions in the cluster address this common memory in some way, whilst those outside have no access to it.

A good example of an informational cluster is a printing utility which requires state memory to maintain a print buffer, information about page layout and so on.

Packages as Informational Clusters



This information is established by one utility function, but used by later invocations of the main print utility. The state memory may be purged by a terminating module.

In APL there is no simple way to implement an informational cluster. The only form of state memory available is a global variable but these can be accessed by all the functions in a workspace. Global variables are liable to "unauthorised" change or reference by functions outside the cluster.

Packaged workspaces provide an ideal facility for overcoming this problem. The functions and global variables are simply placed in a package, each of the "top level" functions are named as entry points. Once packaged, the global variable cannot be accessed by objects outside the package - it is protected from side effects derived from the rest of the objects in the active workspace.

Conclusions

The concept of the package enriches the range of facilities which we can use when designing APL applications. As with all aspects of application implementation, we should consider carefully how the package is going to be used before attempting any implementation. To identify a suitable target area for the packaging process we should look for:

- self contained units of code
- relatively small numbers of objects required
- small number of "entry point" functions
- conservative use of workspace

All these factors tend to be indicative of "informational clusters" within our application. We can partition the application into a series of independent units of code each with their own private state memory and workspace resources. Our application will gain from the independence of these various units - changes to one will not cause side effects in others and so on.

Packages are not a universal panacea for all the badly designed APL applications in existence. A badly designed package can do just as much damage to the implementation of an APL application as any other badly designed module. If we apply the same design techniques and quality measures to packages as we do to the rest of our APL code, we will be able to take advantage of packages without encountering any potential costs. **A package is simply another resource for the APL2 programmer, to be used with care and consideration at all times.**

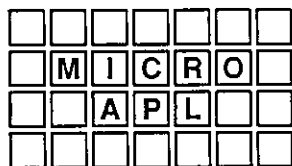
References

- [CAD90] Functional Design Seminar, Cocking & Drury Ltd, 1990
- [IBM89] *APL2 Release 3 Evaluation and Vector Performance*. IBM ZZ81-0197-00
- [PAG88] Page-Jones. *The Practical Guide to Structured Systems Design*.
Prentice Hall 0-13-690777-6
- [PIP87] Piper D.B. *Command Driven Interface for BDAM and QSAM*
3 3 118>Vector

MicroAPL for ... Consultancy and Training

MicroAPL has 10 years' experience with small-computer APL systems - stand alone, networked or multi-user - and using all the major versions of APL. We provide a full range of consultancy services including feasibility studies, project management, programming, documentation and support services. Our programming skills cover all areas of APL and systems programming. MicroAPL also offer a comprehensive range of APL training courses.

For more details, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866

APL 90 and the Ideal APL Conference

by Anthony Camacho

What is it that would make an ideal APL conference? No doubt everyone will have a different view. Here is mine with, I hope, a fair assessment of how well APL 90 lived up to it.

The ideal conference would help to make APL more widely used, teach me how to be a better APLer, give me code and other help in doing my job better, inform me about the state of the art (and what hardware and software I ought to want for my next system). It would also entertain me, give me time and opportunity to meet as many people as possible, and be held in an attractive place (where I would like to stay on and have a holiday). And if any conference ever did all of this I would be dead of exhaustion at the end of it.



High IQ in Helsingør (from the Helsingør Dagblad)

How did APL 90 measure up? It was well organised. All events ran smoothly. The conference bureau was friendly and efficient. The proceedings were posted to those who registered early so that a determined person could have read all the papers beforehand and made much better-informed decisions about which

presentations to attend than at any previous conference. I welcomed this and hope other conferences will do it. The selection of papers was broad and the quality seemed to me to be at least up to the standard of previous conferences. There were more invited papers than usual and they were most of them very good. The get-together on the opening night was a success; it was held in the Danish science museum at Helsingør - a very good location with interesting exhibits from early computing and electronics - and very enjoyable. The banquet too was good and held in one of the best restaurants in the Tivoli gardens. Wine was supplied and the main dish was reindeer - very tasty. We were able to walk round the gardens before and after the meal.



Restaurant Nimb on leaving the Banquet

The organisers deserve our thanks and congratulations on an excellent job. They had bad luck too: the original venue, the Marienlyst Hotel, went out of business at the critical moment and the conference had to be moved in haste to the Falkoner centre. The consequence was that people were not so well collected together. They were spread among many hotels and although the Falkoner centre has plenty of space and places to sit and talk, I fancy APL 90 was less of a social occasion than it would have been if the original venue had been possible. But it is fair to say that if I had not known that the conference was originally to have been elsewhere, I don't think I could have told. It was undoubtedly well done. Among the 'free' items given to every delegate was a pass for all central Copenhagen buses and trains and a pass giving entry to the Tivoli Gardens on two days.

Even if the conference had been held where originally planned, I feel it would not have done much to help spread the use of APL. In a way, of course, by

encouraging people who work at this, it did help, but the complaint about New York, that it probably didn't recruit ten new people to use the notation, is also valid for Copenhagen. It was, of course, slightly more expensive than the general run of conferences. Not too expensive for companies, but too expensive for most other people. Perhaps it was this that reduced the size of the audience; perhaps not. It is a pity that the organisers were not rewarded by a bigger audience; there were about under 300 paying delegates, I am told. I hope that it was not a financial failure because it was certainly a success as an event. The APL world has not discovered how to ensure there are enough delegates at conferences; I hope that the APL 91 committee, who are making such determined efforts to publicise Stanford, strike upon a more productive way to obtain delegates.

The good planning extended to the software library which had collected most of its offerings well before the conference and was ready with complete sets of copies from Tuesday onwards. Indeed some contributions were delivered too late to get into the early sets. A first-rate job was done here with little fuss.

In contrast to the excellent software library, there were no books on sale. Perhaps there is no Danish APL bookseller. We would nowadays have difficulty in providing one in the UK. Probably the delegates to APL 90 already had all the APL books they wanted. Nevertheless I would like to see books at such a conference. I am sure that most of the delegates had not seen Norman Thomson's book "APL for the Mathematics Classroom" and many would have been interested. Perhaps it would be carping to point out there was no education day, only three papers in the education stream, and as far as I could see practically no delegates from schools.

One of the events worth mentioning was the FinnAPL dinner in the Danish Royal yacht club building close to the little mermaid. They gave the best meal of the conference to the Russian visitors Alexei Miroshnikov from Leningrad and Andrei Kondrashev from Moscow, and to everyone they thought might be able to help spread the use of APL in the USSR. Apart from the meal, the speeches were excellent and sensible and there was ample time for everyone who wanted to buttonhole the principal guests to do so. FinnAPL are to be congratulated for what may be a trailblazing effort. SigAPL even mentioned that they are considering Leningrad for APL 92! This contact may encourage the richer APL organisations to support the Russians in spreading APL further through the USSR.

After the get-together in the Science museum, some of us were invited by Henri Brudzewsky to drinks at his house "twenty minutes walk away through the woods". This 'midnight hike' became one of the jokes of the conference. Either

Henri can walk faster than the rest of us or the party lost its way in the dark, probably the latter, because we measured it later on a large scale map and it was 2.50 kilometres. I'm glad I had a lift: the rest of them took over an hour! They saw quite a lot of the local wild life on the way I hear. But everyone including Ken Iverson himself survived the journey and some people enjoyed it. Now poor Henri gets teased whenever he says something will take twenty minutes - "You mean as in '20 minutes walk'!". They were quite good drinks too.



Plenary Audience: Henri Brudzewsky, Gustav Tollet et al

The conference ended with two plenary sessions. In the first a magnificent series of images was shown by Hans-Peter Meinzer. His software (all written in APL) can create sections through and views of parts of the human anatomy. He was able to show us the head of a girl, first complete, then with the skin removed and gradually layer by layer distinguishing between tissues of only slightly different densities, to the bare bones. We were shown images mainly of heads. One could even view the inside of a head which was, in real life complete. One could view the brain as a cloud with the cerebellum visibly different and a tumour extremely distinct, and rotate the image to see from any angle exactly how the tumour lay with regard to the rest of it. Here is an application for a massively parallel APL machine if ever there was one; each image can take hours to calculate.

After lunch the final session by Baird Smith was not at all to my taste. First it had nothing to do with APL. Second it was much too loud, both in the sense of the loudspeaker volume and in taste. I have to admit that he is a showman and that he had quite a number of good jokes in his presentation, but for me this was bathos; after a good conference, with lots of good APL, to have a non-APL clown getting laughs from fairly hackneyed data-processing jokes.

A Personal View on APL 90

by Ray Cannon

A week to the day after the end of APL 90, I received my copy of APL Quote Quad, volume 20, number 2, December 1989, in which an invitation to attend APL 90 was included! Luckily, the Danish end of the Conference was better organised, as I received the conference proceedings (APL Quote Quad volume 20 number 4) in July, 2 weeks BEFORE the conference started!



Eric Friis and Stanley Jordan

This was the fourth International Conference on APL I have attended, and the one I have enjoyed the most. The standard of the papers was good, the organization excellent, the food superb, the location wonderful, and probably the best lager in the world!

The main difficulty I have with all such conferences is deciding which of the venues to attend. (APL90 as is customary, had 3 streams.) So, when I talk about the papers I found outstanding, at best I have missed two thirds of the conference. Since the theme of this conference was "For the Future" I decided to try to attend those talks about compilers, parallel computers, and APL linking to other

languages with a secondary theme of windows, mice and CUA. I'll bypass the vendors' forums, the time allotted to them was a bit too short, and I felt that the ones I attended were not as good as the BAA's forum held earlier this year.

APL90 was exceptional for another reason. For the first time, delegates from the USSR attended an International APL conference. They gave two papers but I missed the first (it conflicted with one on my pet interest, parallel computers). However I managed to squeeze into the second "The Family of Soviet APL Systems" given by Andrew Kondrashev. This talk was a "sell out", some people were even listening from outside! I for one found it fascinating, for instance, (I hope I noted it down correctly), 37,000 copies of a Russian translation of "APL an Interactive Approach" by Gilman and Rose have been printed in the USSR.

АПЛ in The Soviet Union

by John Searle

I learnt APL from a Russian man, Michael Szudrich, whose family had emigrated to Australia after World War II. Thus it was no surprise to me that APL was used in Russia and other Soviet Republics, and was finding some acceptance. Russians have been using an expressive, descriptive language and a complex alphabet for a very long time.



Olga & Andrei Kondrashev, Erkki Juvenen & Alexei Miroshnikov

During the APL90 conference it was a pleasure to talk with Alexei Miroshnikov of Leningrad, and Andrew & Olga Kondrashev of Moscow. Andrew sees APL as "a unique opportunity to eliminate language barriers between computers and users". They were intrigued as to why they were receiving so much attention from other conference delegates. Perhaps "flavour of the month" has no translation into Russian? They were very polite to their hosts and Finnish minders, and very diplomatic in their avoidance of political discussion.

APL is in use from the Neva to Novosibirsk, and from Tbilisi to Kazakhstan. the first Soviet APL conference in 1986 had 20 delegates, but annual meetings have since been held with ever-growing numbers. Andrew estimates that there are now between 300 and 400 users in 20 cities.

A sense of humour is very much a part of the Russian culture. "My English is not as good as it seems," Alexey began, using very good English. He told a

funny/sad story about "Ilefthander", a brilliant scientist who could hammer golden nails into the boots worn by the Tsar's pet mechanical flea, but who died a poor and lonely man. His Moscow jokes are recorded elsewhere (he is, of course, from Leningrad).

Some political discussion was unavoidable, and both speakers were asked if perestroika had changed their work. Alexei said that perestroika had actually made it a little more difficult for his work at the institute, because less money was now available for centrally-funded planning and modelling activity. Andrew initially dismissed the question, and suggested that "you should ask Mr Gorbachev". Then, after some careful consideration, he delighted the audience with his belief that "all political problems cannot be solved by merely using computers".

Historical Perspective

Eastern European presence at APL conferences is not new. The APL75 conference had delegates from Czechoslovakia, Hungary and Latvia. (Thanks to Henri Brudzewsky for having such a good APL library.) APL first appeared in the Soviet Union around 1977, although most of the early interpreters were not fully implemented. In 1979, the Gilman & Rose book "APL an Interactive Approach" was translated into Russian. It is still in print and is believed to have sold over 37,000 copies.

Ken Iverson and Bob Bernecky, both then at I.P. Sharp, visited Moscow after APL84 in Helsinki, and an IPSA node has been available there since that time. The recent APL interpreters are based on early APL standards work by Falkoff and Orth. The most recent standards work does not appear to have flowed through. Apparently, the channel of communication through ISO to the USSR is open, but as yet there is no data in the channel. The APL/85 interpreter, documented in Andrew's paper, is the most powerful. It splits the available memory into 2 areas, one for functions and one for data, and the interpreter always runs in data mode. Sounds a bit like DOS?

One of my fondest memories is of the bus trip to Leningrad after APL84. We were an array of about 40 APLers and families from 15 countries, nested together for 4 days, and each of us was amazed by Leningrad's disclosures. It is a city that combines architecture and geography from Paris, Venice, Florence and Prague. It was midsummer and it was warm. I have a picture of the sun shining brightly on the town digital clock as it "struck" 2300 hours. If an APL conference is to be held in Leningrad in must be held in June. The sun is always over the yardarm, so don't expect to get much sleep.

Reports on the Presentations

The People, the Information and the Corporation

Mads Øvlisen of Novo Nordisk A/S

reported by Sylvia Camacho

Mads' talk was professional, well structured and well timed with good slide illustrations. The interest for me was the insight he gave into a Company with a high research and development commitment (25% of employees). This R&D serviced the 44% of employees engaged in production and 21% engaged in marketing the products. The traditional business is a restricted range of pharmaceuticals, particularly insulin. Most R&D is concerned with biotechnology and disease control and other aspects of environmental care. The Company sees its edge over its competitors to be matching its products to the exact requirements of customers. For instance the development of insulin delivery hypodermics convenient for patient use.

There is some tension within Marketing because the direct customers are doctors which means that contact with the actual users, the patients, is mediated by the doctors who do not necessarily have the same priorities. One possible use of Information Technology is therefore to obtain feedback from patients.

Øvlisen made the point that management concern for the interests of the shareholders does not offer the Company a plan of action. This is an obviously true but perhaps not sufficiently realized part of free enterprise ideology. Øvlisen made two other points; that employees usually under-perform and that Companies become conservative because of management anxiety. His remedy for some of these problems is increased dispersion of information among employees and encouragement of their individual differences. This he sees as incompatible with a tight central control of computer systems by DP management.

Øvlisen gave as the major uses of IT within his Company:

1. Mailbox facilities
2. Medical Representative System
3. R&D including molecular modelling
4. IFCA

The mailbox was prominent among his interests because of the flexible and fast exchange of information that it provided to for a geographically widely dispersed sales force. The Sales Representative System he hopes to base on lap

top computers to give management feedback from the grass roots. He made the point that there is no right way to manage R&D. The most that can be done is to match the risks and benefits over a portfolio of research projects and that this has to be directed from the top of the management hierarchy.

Mads' final comment was that the IFCA project which has a concept of adding information to products based on patient needs is a good but costly study by outside consultants and has the difficulty that the technical language used is not familiar to the management. This he said was his own problem and he feels that IT requires too much learning and encourages excessive emphasis on perfection whereas managers need to use less than perfect data. The problem is how to get a dialogue going between IT professionals and management so that if they are offered a better mousetrap they are able to decide whether they need to catch a mouse.

APL\? (or J) by Roger Hui and Ken Iverson

reported by Anthony Camacho

Since this paper was submitted APL\? has been named 'J'. Apparently even Ken Iverson does not know the reason why that particular name was chosen, although he must have assented to the name. Roger Hui chose it and keeps his reasons secret.

Ken Iverson told us that J is a dialect of APL, but it does not offer compatibility with any other dialect. The features it contains have been chosen without regard to any predecessor. This is interesting; it allows us to see what Ken Iverson and his collaborators like and dislike about their previous APLs.

J does away with the special APL symbols; all tokens in J are ASCII symbols or combinations of them. (This caused some dismay among the Scandinavians, who use the ASCII codes for the less used symbols for their special Scandinavian letters.) J preserves some things from APL: it has no reserved word, does not need spaces to punctuate its tokens, the tokens have mnemonic value but are not linked to any natural language, so it is truly international, but its tokens may be one symbol or two, whereas APL tokens are single. J simplifies and is more general than APL. Whereas APL has two 'parts of speech' the function (verb) and the operator (adverb), J has three (verb, adverb and conjunction) and whereas in ISO APL there are limitations which prevent adverbs modifying user-defined verbs, or even the existence of user-defined adverbs, in J there are no such restrictions. User defined conjunctions can join user-defined verbs as modified with user-defined adverbs.

In a brief review of the history of APL development he mentioned that Gene McDonnell had spoken about the 'sociotechnical' origins of APL at the tenth anniversary conference which was also in Copenhagen, and that APL\360 had been able to support 24 users when other time sharing systems had difficulty with three or four or five users. Many great improvements had been made over the years: Sharp had introduced: fast sum-over-booleans, fast indexing, membership, reverse, transpose, general arrays, complex arithmetic, fuzzy residue, dyadic grade, replicate. In 1982 Rationalised APL was circulated as an IPSA internal paper. In 1986 A Dictionary of APL was published and now in 1990 J.

The trends of Dictionary APL are:

- 1 Simplification of grammar and syntax
- 2 Operators significantly beyond APL\360
- 3 A uniform method for associating identifier with object
- 4 A strong emphasis on function rank, leading axes, major cells
- 5 The removal of dependence on such parameters as index origin
- 6 Exclude anything not functional
 - strands
 - bracket semicolon indexing
 - indexed assignment
 - bracket axis
- 7 A preference for functional rather than shape identities

Roger Hui told us something about the implementations of J so far. It is available on the AT&T 3B1, the Macintosh, the PC, the R6000, the MIPS R3000, the NeXt and a Philips machine. The implementations are written in C; the parser is a single page of code and all objects are held in a single C form, which contains the type, the reference count, the number of atoms, the rank, the shape and each atom. Most of the C functions accept arguments in this structure and return such results and, of course, all verbs, adverbs and conjunctions are held in this form as well as the nouns and pronouns.

J is to be freeware and the registration charge which brings a manual is to be \$24. Copies of the PC implementation (which is not quite complete) were issued through the software library. The address of Iverson Software Inc. is in the product guide.

Parallel Computing with APL

reported by Ray Cannon

I have an interest in Transputers, so I attended the paper "An APL-Simulator of Non-Von Neumann Computer Architectures" written by A. Geyer-Schulz, Alfred Taudes and J. Mitlohner (who gave the talk). This paper showed how APL could simulate various parallel computer architectures. Unfortunately (for me at least) the paper did not describe transputer array simulation, but the simulation of a theoretical model of a PRAM-machine described by Parberry (*Parallel Complexity Theory*, New York, John Wiley & Sons, 1987). That apart, I found the paper well presented and most stimulating.



Parallel APLers (in front of the Tycho Brahe Planetarium)
Jonathan Martin, Panagiotis Pantziarka, David Toop & Maurice Jordan

The next paper was "Automatic Parallelization of APL-style Programs" written and given by Wai-Mee Ching. (Some of you may remember Ching from Manchester where he gave a paper on an APL/370 compiler.) This paper talked about extending the compiler so that it produces 370 machine code to run applications in parallel mode on IBM 3090 multi-processors.

I talked to Ching about his work afterwards. He has written two compilers, one producing 370 machine code and the other generating C output. Both compilers by the way are written in APL and compile IBM VS APL!

Ching said that "Parallel computing is only interesting if it is transparent to the programmer". Some other gems from his talk I noted down include: "the compiler can find the parallelism in existing programs"; "data is parallel...use nodes not do-loops"; "to efficiently program on a parallel machine, write good sequential APL code".

The next paper in this vein was given by Robert Bernecky, "ACORN, APL to C on Real Numbers". This paper described how the authors wrote an APL to C

compiler, in Sharp APL/PC, and had a prototype working within a week! The resulting C code was compared against hand-coded Fortran and benchmarked on a Cray super-computer. The final results show that APL should be able to be compiled to C to run at least as fast as hand-coded Fortran, but there is still a long way to go before we get there.

After giving his paper, Bernecky then chaired a session with an invited speaker Alan Huang, of AT&T-Bell Laboratories on "Massively Parallel Computing". This talk was all about "light computers", "micro-lasers" and "life after electronics". The units Huang is creating are so fast that standard electronic equipment cannot be used to measure their speed, and so small that his laboratory has equalled the world production of lasers, by producing micro-lasers by the thousand on single silicon wafers.

They can now produce "designer crystals" where each atomic layer can be laid down individually. However, Huang said that his physicists only understood HP Basic, his computer scientists Lisp but his machines required A Parallel Language.

High Quality User Interfaces and fast Data Filing using a PC

by Jonny Österman

reported by Ray Cannon

Jonny Österman of A-System gave a very good talk on "High Quality User Interfaces and fast Data Filing using a PC". This he backed up with a demonstration, using some of the tools from his APL toolbox (commercially available as "ABOX").

Talking to Jonny on the A-System stand, I discovered that we have both been working on very similar airline charter costing systems for different airlines, he for SAS and myself for BA. These are the only two charter costing systems either of us had come across, and both were written quite independently in APL. Jonny was using a cut down version of his to demonstrate his ABOX utilities, and I was very impressed.

Compiled Language Interface in APL*PLUS II by James Wheeler

reported by Ray Cannon

In his talk, James Wheeler described how, under DOS, (which does NOT have dynamic linking) STSC's APL is able to call non-APL programs. With some background knowledge of 80386 assembler, it all made good sense, and was very revealing.

Tutorials

reported by Anthony Camacho

The tutorials given by Eugene McDonnell and Philip Benkard were, for me, the highlight of the conference. Both were excellent presentations; both gave new insights into the differences between the main streams of enhanced versions of APL; both contained much too much material for even the best reporter to capture. I hope that the following brief impressions will be superseded by complete accounts prepared with the help of the presenters.

Tutorial in Dictionary APL by Eugene McDonnell

Gene pitched this tutorial to suit people who have some knowledge of Dictionary APL, boxed (and nested) arrays and the ISO standard. It was not a programming lesson but a review of fundamentals, concentrating on identities, symmetries and analogies.



Dick Bowman, John Searle, Jonathan Martin, Maurice Jordan, & Lee Dickey relaxing in the Falconer Centre

APL was contrasted with FBAPP (FORTRAN BASIC Algol Pascal PL/1) languages: in APL arrays are primitive objects whereas in the others generally a scalar is the primitive object. It is the way a language is parsed that determines its consistency. Gene explained a parsing table, against which expressions were compared from right to left 'until the binding drops' - that is until a match between the rightmost part of the line being parsed and a complete row of the

table is found. Finding a match establishes that the part of the line being matched can be executed and the result substituted for the matched phrase before further parsing need be done. Incidentally it was pointed out how valuable it is that in APL it is not necessary to separate tokens by blanks. An analysis of the forms of expressions showed where data, function and operator strands would be.

Gene offered some snappy definitions of syntax and semantics.

Syntax is what goes with what.

Syntax is what the user really learns.

Syntax is context dependent and of short scope.

Semantics is where the action is.

Dictionary BOX (<) was compared with APL2 ENCLOSE (c). Everyone knows the difference for shallow scalars, but how many of us can state the differences for other arrays? For example:

```
3+<1 5
DOMAIN ERROR
3+c 1 5
4 8
```

So Gene suggested that < is semantics and c is structure.

Tutorial in APL2 by J. P. Benkard

Phil Benkard's tutorial was given at a faster pace. By using two overhead projectors a great deal could be shown and compared. Phil produced a fascinating series of insights based on analysis of the simple function to remove duplicates: $((\omega \downarrow \omega) = 1 \rho \omega) / \omega$.

I regret that I was unable both to follow and write down what Phil said and showed us; my notes contain enough pepper to cook a cauldron of soup, but they are too incoherent to offer to Vector. We have asked Phil if we can publish the full tutorial.



J. Philip Benkard

Education Stream: Wednesday Morning

reported by Anthony Camacho

APL Trivia by Ed Cherlin

Reading to Write by Ray Polivka

APL elegance by Norman Thomson

These three sessions formed the Education Stream. They were a delight to attend. Three expert presenters provided an object lesson in exposition. Your reporter was also the chairman, but his lack of notes is not serious since the presentations did not wander far from the papers in the proceedings.

Ed showed how APL's inner consistency can be revealed in apparently trivial ways. He produced a nice paradox: beginning with a definition of *trivial* which seemed to leave little scope for anything of importance to creep in, he showed how trivial functions and operators can be valuable both for what they do and for the improved clarity of the code in which they are properly used. Surely what is valuable is important.

Ray suggested that as in natural languages one does not normally write anything until one has progressed quite a long way towards learning to read, APL could be taught by getting the pupil to read code before writing it. He gave some lovely examples, many of which are in the appendix to his paper.

Norman highlighted a common problem of newcomers to APL2 - namely that they find they have enclosed their variables without realising it. His subtitle was 'The Art of Staying within One's Depth'. He gave many useful prescriptions and hints of great value in reaching a more intuitive understanding of APL2 facilities.

This was the education stream, yet there were several questioners who objected to some of the elegant and brief expressions shown on the grounds that they were not the fastest to execute. It was almost as if the only kind of education with APL they could imagine was teaching professional programmers how to earn their living with APL. The APL world ought to want APL to be used in the teaching of other subjects, where its simplicity, consistency and elegance are such a powerful advantage. Surely it is unreasonable to expect any implementation to be most efficient in its use of the machine on the briefest and most elegant expressions. Even if it is not unreasonable surely it is irrelevant. Nowadays machines are very cheap by comparison with the cost of the programmers' time; when the purpose is education the efficiency of the code really can almost be

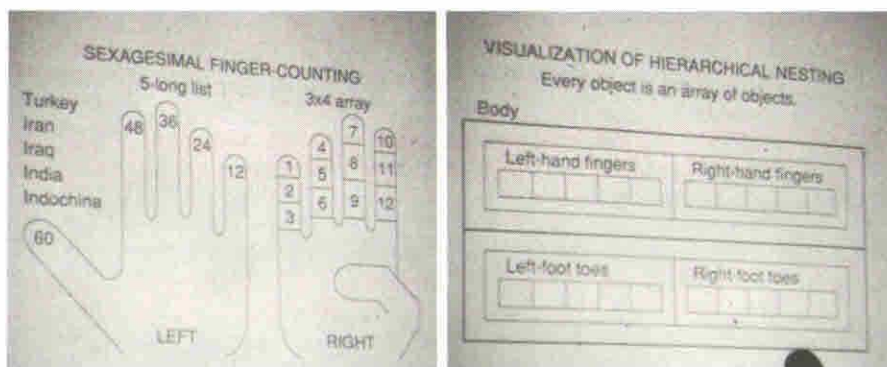
ignored. While so many APLers have their sense of values wrong about this, how are we to persuade the rest of the world?

Looking Beyond APL2 and NIAL

Invited Speaker: Trenchard More Jr.

reported by Anthony Camacho

This session was a tour de force on Array theory. It started simply by using the hands and feet as examples of hierarchy and nesting. Considerations of consistency and how to handle the degenerative cases of empty arguments according to their shapes and nestedness were dealt with in a breathtaking sequence of slides. Here are two of them. We hope to be able to persuade the author to publish an article about this in Vector.



Two of Trenchard More's slides

The K. E. Iverson Award for Outstanding Achievement

reported by Anthony Camacho

SigAPL announced that the Outstanding Achievement award has been renamed 'The K. E. Iverson Award for Outstanding Achievement' and that the first renamed award goes to Ray Polivka.

Jon McGrew did the presentation and had prepared a splendid testimonial to Ray's work over the years. There were photographs of Ray with his classes, of him teaching, of the books he had written or collaborated in and even of him asleep in an armchair. (Ray later told us that Jon had insisted on Ray's company when he went to buy himself a suit and while he had it fitted Ray, tired after the day's work, went to sleep in an armchair in the shop waiting for him.) Jon's

account of Ray's career was fascinating and beautifully illustrated. Ray's work in teaching and writing about APL must be well known to every APLer and the audience obviously was both delighted by the award and felt it to be well deserved.

And the award came as a complete surprise to Ray - he had not suspected anything when he was photographed teaching because it just so happened that no-one else was teaching when the photographer needed some urgent photos of IBM education - neither he nor Jo were even suspicious at the stratagems the organisers used to make sure that they would be present at the session when the award was announced. It was a heart-warming occasion.

ACM SigAPL report: Lynne Shaw

reported by Anthony Camacho

Lynne was pressed for time and the report this year was shorter than, for example, at New York.

Quote-quad: Prof L. J. Dickey has resigned and the new editors of Quote-quad are Adam Kertesz and Jon McGrew. The March issue of Quote-quad and the proceedings will be sent to members soon.

Membership: The membership trend is UP at the latest membership roster as of mid-June.



Charles Schulz with the APL91 Palm Tree

Conferences: APL 91 is to be at Stanford. Charles Schulz made some announcements about it. The aim is to make attendance as easy as possible for people paying their own way or whose employers have little to spend on conferences. There will be cheap university accommodation as well as luxurious hotels for those who want them. It is hoped to encourage far more educators to attend without reducing the conference's value to the professional APL programmer. The APL 91 committee are eager to receive people's suggestions and ideas. They are making great efforts to circulate the call for participation as widely as possible.

APL 92 is not yet allocated. One suggestion is to hold it in Leningrad.

A new award: SigAPL intend to establish a new award for extending awareness of APL. Anyone who wants to suggest a recipient should contact SigAPL for details.

Quotable Quotes and APL90 Jokes

by John Searle and Ray Cannon

Quotable Quotes from APL90

"A great truth is a truth for which the opposite is also a truth" - Trenchard More

"Life is too short to write Cray assembler code" - Bob Bernecky

"We were going to go through this in a sequential order, until we dropped the foils about 10 minutes ago" - Morten Kromberg, Denmark

"My English is not as good as it seems" - Alexey Miroshnikov, Leningrad

"There is too much religion in systems ... (too many of you) exhibit rejected prophet syndrome" - Mats Øvlisen, Novo Nordisk

"I hope there is no one in the room to whom that is not completely clear" - Norman Thomson, IBM

"I plead Alzheimer's defence - I didn't forget that, I just never knew it" - Ken Iverson to Adin Falkoff

"God was only able to invent the universe in a week because there was no installed user base" - Bob Bernecky

"There are live musicians on my records" - Stanley Jordan

"A window is something you open for air, not double-click on" - Baird Smith, IBM

"All political problems cannot be solved by merely using computers" - Andrew Kondrashev, Moscow

"Yea, from the table of my memory I'll wipe away all trivial fond records" - Hamlet, Denmark, ca 900, by Shakespeare, ca 1600, discussing the first computer virus

... and some unattributed contributions recorded by Ray Cannon:

"the machine's called a GF-11 after its 11 Giga-flop performance"

"parallel code running on a parallel machine"

"all other languages should be thrown away"

"I'm Swedish, but I'm not prejudiced"

"5000 lines of code with no comments"

"There's nothing new in computing."

"You get several sorts of Num Pricks"

"That's our problem, were Danish."

"It's exactly the same only different."

"What can you build with one LEGO block?"

"No language is an island."

APL90 Jokes

Q: How many APL*PLUS programmers does it take to change a light bulb?

A: None. STSC have already written a system function for that.

Q: What is the definition of fast food in Moscow?

A: Only having to queue for 2 hours to get into Macdonalds.

Q: What do you call an empty array in Moscow?

A: A shop.

APL90 Reflections

by John Searle

The conference was very well organized and APLers are definitely improving in their standard of presentation and use of high-tech audio visual aids. Nearly all of the authors spoke to or about their papers rather than just summarised them, and thus made actual attendance at the conference worthwhile. The organisers are to be congratulated in getting the proceedings out in advance.

Conference Highlights

First ever delegates from Moscow and Leningrad. First time the proceedings were distributed to early registrants in advance. Premiere of Iverson's new "J" notation, with free software. Public showing of Dyalog APL/X for HP and RS/6000, and new DOS version 6.1. Public showing of IBM's APL2 for RS/6000 and MicroAPL's APL.68000 level II. Stanley Jordan's APL music and impromptu guitar performance.

Conference Lowlights

STSC's reluctance to display any experimental APL version under X windows. Rainstorm timed to coincide with conclusion of fireworks in Tivoli gardens. The price of beer in Copenhagen (although the Swedes come over to drink it).

Exhibition Wars

There was healthy competition in the exhibition, and Dyadic systems were clear winners with their readiness to show the pre-release Dyalog APL/X and upcoming new version 6.1 of the DOS product. STSC gave a paper about APL under X but did not show any software. They appear to be hamstrung by the size of their installed user base and are very careful about new features. They have no immediate plans to call APL from C (they say no one has asked for it), but they hope to release a port of APL*PLUS II to Unix by the end of 1990.

Future of APL

As always, there was much discussion about what sort of future, if any, APL has in conventional data processing. I usually leave a conference feeling much more optimistic than at the start, having seen what can be done, and this conference

was no exception. Although conference numbers were down to about 300, the depth and variety of APL applications were as broad as ever. APL continues to be a competitive option for programming language choice.

Optimistic points:

- in Europe, APL continues to be one of the few environments which support different national languages yet provide international compatibility.
- Jim ("APL2") Brown of IBM has almost convinced the US Dept of Defense that APL2 is the best basis for a common prototyping language for DARPA (the defense advanced research projects agency). If this comes off it will be big.
- APL is used in the forefront of technical developments in medicine, music and space science, as well as computer processor research and language theory.

Pessimistic points:

- Reuters have "stabilised" Sharp APL so that there will be no new language developments, and support for the mainframe product could be reduced. There were a lot of ex-Sharp people at the conference looking for work.
- IBM have not included APL2 in the SAA because there is no AS/400 version. This means that a lot of "true-blue" IBM sites will not consider APL.
- APL has been around for so long that it has no appeal for managers or users who insist that their systems be leading edge.

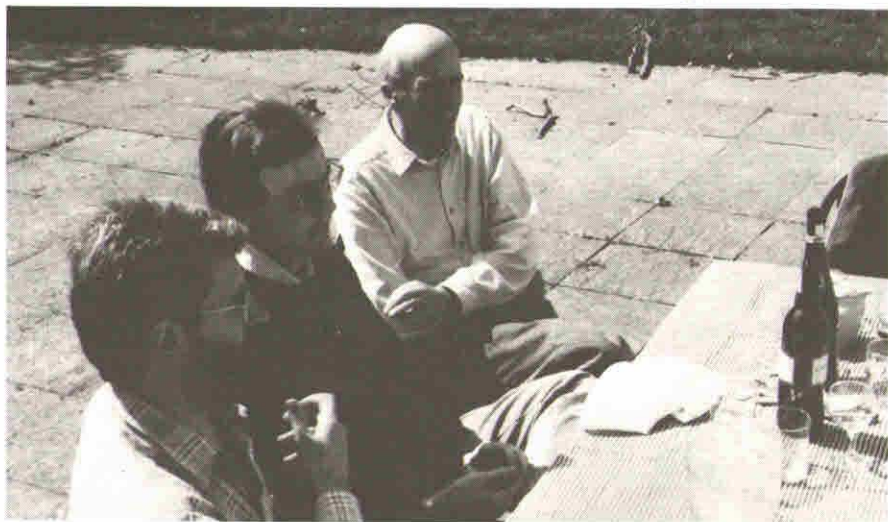
Whither APL Conferences?

The family of APL languages has reached quite a mature stage. The conferences now include developments in ISO APL, APL2, A, J, NIAL and even Mathematica. Through the conferences, attendees learn of the latest developments in many related fields, such as OOPS, neural networks, parallel processing, and other buzzwords, mainly because someone can explain them or prototype them very clearly using an APL model or notation. Despite too much navel-gazing, the conferences serve a good purpose. Long may they continue.

The next conference is planned for Stanford University, Palo Alto CA, in August 1991. The one after that may possibly be in Leningrad, June 1992 or 1993, so start saving your roubles for some white nights.

See you in Palo Alto!

Pictures at an Exhibition (I)



**The Vector Working Group plan their Campaign:
Ray Cannon, John Searle, Jonathan Barman,
and an excellent bottle of Australian Wine!**



Dave Selby, Ron Wilkes & Jim Brown on the IBM Stand

Pictures at an Exhibition (II)



John Sullivan, Dick Bowman and Maurice Jordan
at the Dyalog APL Stand



Richard Nabavi with David Crossley
at the MicroAPL Stand

A
DICTIONARY
OF

J

by
Kenneth E. Iverson

THE ISI DICTIONARY OF J

Kenneth E. Iverson

J is a dialect of APL, a formal imperative language. Because it is imperative, a sentence in APL may also be called an *instruction*, and may be *executed* to produce a *result*. Because it is formal and unambiguous it can be executed mechanically by a computer, and is therefore called a *programming language*, a phrase from which its name derives. Because it shares the analytic properties of mathematical notation, it is also called an *analytic language*.

APL originated in an attempt to provide consistent notation for the teaching and analysis of topics related to the application of computers, and developed through its use in a variety of topics, and through its implementation in computer systems. Discussions of the language may be found in *The Design of APL* and *The Evolution of APL* [1]. Reference 2 discusses a freeware implementation of the language defined here, as well as its relation to the earlier version [3], which also provides information on other dialects.

A dictionary should not be read as an introduction to a language, but should rather be consulted in conjunction with other material (such as Reference 4) that uses the language in some context of interest. On the other hand, a dictionary should be used not only to find the meanings of individual words, but should also be studied to gain an overall view of the language.

I: ALPHABET AND WORDS

The alphabet used is that of ASCII, and the words used to denote facilities in the language are spelled with either one ASCII character or with one character followed by a full stop (.) or a stop (:). Spaces between such words are permitted but not required, except that a number beginning with a decimal point must be prefaced by a space or a zero. The entire spelling scheme is shown in the Language Summary of Table 1.

Pronouns and proverbs begin with a letter of the English alphabet and continue with further letters or digits. Such a name followed by a colon is a *distinguished* or *non-standard* word that may, in a particular implementation, be used to supplement the standard words.

Quotes behave much as in English: everything between an opening quote and a closing quote is treated as a single word, except that a pair of adjacent quotes following an opening quote represents the quote character itself; 'can't' is the five-character abbreviation of the six-character word 'cannot'.

The negative sign is denoted by $_$, as in $_3$ and $_3.14$. The $_$ alone denotes infinity, and $___$ denotes negative infinity. A *real* number may also be denoted in *exponential form*, as in $3.25e4$ to signify 32500. A *complex* number is denoted by a *j* between two real numbers, as in $_3e4j5.6$. Numbers separated by spaces denote a *list* or *vector*, as in *prices* = 3.12 4.6 2e3.

II: GRAMMAR

The following sentences illustrate the six parts of speech:

```
fahrenheit=. 50
(fahrenheit-32)*5%9
10
prices=. 3 1 4 2
orders=. 2 0 2 1
orders*prices
6 0 8 2
+/orders*prices
16
+/\1 2 3 4 5
1 3 6 10 15
bump=. +#1
bump prices
4 2 5 3
```

PARTS OF SPEECH

50	fahrenheit	Nouns/Pronouns
+	- * % bump	Verbs/Proverbs
/ \		Adverbs
&		Conjunction
()		Punctuation
=		Copula

Verbs act upon nouns to produce noun results;

2

the nouns to which a particular verb apply are called its *arguments*. A verb may have two distinct (but usually related) meanings according to whether it is applied to one argument (to its right), or to two arguments (left and right). For example, 2%5 yields .4, and %5 yields .2.

An adverb acts upon a single noun or verb (to its left) to produce a verb. For example, +/ is a (*derived*) verb called *plus over* that sums an argument list to which it is applied, and */ yields the product of a list. A conjunction applies to two arguments (either nouns or verbs) to produce a verb.

Punctuation is provided by paired parentheses that specify the sequence of execution as in elementary algebra. The word =. behaves like the copulas "is" and "are" and is read as such, as in "area is 3 times 4" for *area* =. 3*4. The name *area* thus assigned is a *pronoun* and, as in English, it plays the role of a noun. Similar remarks apply to names assigned to verbs, adverbs, and conjunctions.

A. NOUNS

Nouns are classified in three ways: numeric or literal; open or boxed; arrays of various ranks, called (for ranks 0, 1, and 2) *atom*, *list*, and *table*, or (in mathematics) *scalar*, *vector* and *matrix*. Numbers and literals are represented as stated in Part I.

Arrays. A single entity such as 'A' or '+' or 2.3 or _2.3j5 is called an atom. The verb denoted by a comma chains its arguments to form a list whose *shape* (given by the verb \$) is equal to the number of atoms combined. For example:

```
date=. 1,7,7,6
$ date
4
word=. 's','a','w'
|. word      |. date
was          6 7 7 1
```

The verb |. used above is called *reverse*. The phrase *s \$ b* produces an array of shape *s* from the list *b*. For example:

```
(3,4) $ date,1,8,6,7,1,9,1,7
1 7 7 6
1 8 6 7
```

```
1 9 1 7
table=. 2 3 $ word,'bat'
$ table      table
2 3          saw
           bat
```

The number of atoms in the shape of a noun is called its *rank*. Each position of the shape is called an *axis* of the array, and axes are referred to by indices 0, 1, 2, etc. For example, axis 0 of *table* has length 2 and axis 1 has length 3. The last *k* axes of an array *b* determine *rank-k cells* or *k-cells* of *b*. For example, if:

```
a=. 'abcdefghijklnopqrst'
b=. 2 3 4 $ a, 'uvwxyz'
```

```
b
abcd
efgh
ijkl
```

```
mnop
qrst
uvwx
```

then the list 'abcd' is a 1-cell of *b*, the two 3 by 4 tables are each 2-cells of *b*, and the letters are each 0-cells of *b*.

The rest of the shape vector is called the *frame* of *b* relative to the cells of rank *k*. For example, if \$ *c* is 2 3 4 5, then *c* has the frame 2 3 relative to the cells of rank 2, the frame 2 3 4 relative to 5-element rank-1 cells, the frame 2 3 4 5 relative to rank-0 cells (atoms), and an empty frame relative to 4-cells.

A cell of rank one less than the rank of *b* is called an *item* of *b*, and items play an important role in the discussion of nouns and the application of verbs to them. For example, the verb *from* (denoted by {) selects items from its right argument, as in:

```
0{b      1{b      0{0{b
abcd      mnop      abcd
efgh      qrst
ijkl      uvwx
2 1{0{b      1{2{0{b
ijkl      j
efgh
```

Moreover, the verb *grade* (denoted by :) provides indices to { that bring items to "lexical" or "row-major" order. For example:

```
n=. 4 3 $ 3 1 4 2 7 9 3 2 0
```

3

$g = ./:n$
 n g $g(n$
 3 1 4 1 0 3 2 2 7 9
 2 7 9 3 1 4
 3 2 0 3 1 4
 3 1 4 3 2 0

Negative numbers, as in $_2$ -cell and $_1$ -cell (an item), are also used to refer to cells whose corresponding frames are of the length indicated by the magnitude of the number. For example, the list *abcd* may be referred to either as a $_2$ -cell or as a 1-cell of *b*.

Open and Boxed. The nouns discussed thus far are called *open*, to distinguish them from *boxed* nouns produced by the verb *box* denoted by $_$. The result of box is an atom, and boxed nouns are displayed in boxes. Box allows one to treat any array (such as the list of letters that represent a word) as a single entity, or atom. For example:

$w = . words = . (<'I'), (<'was')$
 $letters = . 'I was'$
 $\$ letters$ $|. letters$
 5 saw I
 $\$ words$ 2 3 \$w, |.w
 2 $|. words$

was	I
-----	---

I	was	was
I	I	was

B. VERBS

Monads and Dyads. Most verbs have two definitions, one for the *monadic* case (one argument), and one for the *dyadic* case (two arguments). In any sentence, the dyadic definition of a verb applies if it is preceded by a suitable left argument, that is, any noun that is not itself an argument of a conjunction; otherwise the monadic definition applies. The monadic case of a verb is also called a *monad*, and we speak of the *monad %* used in the phrase $\%4$, and of the *dyad %* used in $\%4$.

Ranks of verbs. The notion of verb rank is closely related to that of noun rank: a verb of rank *k* applies to each of the *k*-cells of its argument. For example (using the array *b* from Section A):

b
abcdefghijklmnpqrstuvwz
 $_2 b$ $_1 b$
abcdefghijkl *abcdefghijkl*
mnpqrstuvwz *mnpqrstuvwz*
 Since the verb *ravel* (denoted by $_$) applies to its entire argument, its rank is said to be infinite or unbounded. The rank conjunction $_$ used in the phrase $_2$ produces a related verb of rank 2 that ravel each of the 2-cells of *b* to produce a result of shape 2 by 12.

The shape of a result is the frame (relative to the cells to which the verb applies) catenated with the shape produced by applying the verb to the individual cells. Commonly these individual shapes agree, but if not, they are first brought to a common rank by adding leading unit axes to any of lower rank and are then brought to a common shape by applying the expression $ms \{. r$ to each result *r*, where *ms* is the maximum over the individual shapes. For example:

$_2 _0 (2 \ 3 \ 4) > 'I'; 'was'; 'it'$
 0 1 0 0 I
 0 1 2 0 was
 0 1 2 3 it

The dyadic case of a verb has two ranks, governing the cell ranks of the left and right arguments. For example:

$p = . 'abc'$
 $q = . 3 \ 5 \$ 'wake read lamp'$
 $p, _0 \ 1 \ q$

awake
bread
clamp

Finally, each verb has three intrinsic ranks: monadic, left, and right. The definition of any verb need therefore specify only its behaviour on cells of the specified ranks, and the extension to arguments of higher rank occurs systematically. The ranks of a verb merely place upper limits on the ranks of the cells to which it applies, and its domain may include arguments of lower rank. For example, matrix inverse ($_.$) has monadic rank 2, but applies to degenerate cases of vector and scalar arguments by treating them as one-column matrices.

Agreement. In the phrase $p \ v \ q$, the argu-

4

ments of v must *agree* in the sense that their frames (relative to the ranks of v) must either match, or the shorter (in terms of the number of axes) must match the suffix of the longer. Formally, if the frames of p and q are a, b and b , respectively, then $p \vee q$ is equivalent to $p \vee a \< q$. For example:

$p, "1 \ 1 \ q$
abcwake
abcread
abclamp

C. ADVERBS AND CONJUNCTIONS

Unlike verbs, adverbs and conjunctions have fixed valence; an adverb is monadic (applying to a single argument to its *left*), and a conjunction is dyadic.

Each argument of a conjunction may be either a noun or a verb, and it may therefore produce as many as four distinct classes of results. For example, $u \& v$ produces the *composition* of the verbs u and v ; $\wedge \& 2$ produces the *square* by combining the power function with the right argument 2; and $2 \& \wedge$ produces the function *2-to-the-power*. The conjunction $\&$ may therefore be referred to by different names for the different cases, or it may be referred to by the single term *and* (or *with*), which roughly covers all cases.

D. COMPARATIVES

The comparison $x=y$ is treated like the everyday use of equality (that is, with a reasonable relative tolerance), yielding 1 if the difference $x-y$ falls relatively close to zero. Tolerant comparison also applies to other relations and to *floor* and *ceiling* ($<$ and $>$); a precise definition is given in Part III under the comparative *less-than-or-equal* ($<:$). An arbitrary tolerance t can be specified by using the *custom* conjunction ($\&:$), as in $x = \&:t \ y$.

E. PARSING AND EXECUTION

A sentence is evaluated by executing phrases in a sequence determined by the parsing rules of the language. For example, in the sentence $10\%3+2$, the phrase $3+2$ is evaluated first to obtain a result that is then used to divide 10. In summary, the parsing rules are:

1. Execution proceeds from right to left, except that when a right parenthesis is encountered, the segment enclosed by it and its matching left parenthesis is executed, and its result replaces the entire segment and its enclosing parentheses.

2. Adverbs and conjunctions are executed before verbs; the phrase $"2-\alpha$ is equivalent to $(, "2)-\alpha$, not to $"(2-\alpha)$. Moreover, the left argument of an adverb or conjunction is the entire verb phrase that precedes it. For example, in the phrase $\alpha+/. \ . \ . \ /b$, the rightmost adverb $/$ applies to the verb derived from the phrase $\alpha+/. \ . \ . \ *$, not to the verb $*$.

3. Verbs are applied dyadically if possible. For example, $\alpha-b$ signifies subtraction, but $\alpha*-b$ signifies multiplication of α by the negative of b .

4. Certain phrases that would be invalid under these rules are treated as idioms, as described in Section F.

One important consequence of these rules is that in an unparenthesized expression the right argument of any verb is the result of the entire phrase to the right of it. The sentence $3 * p \% q \wedge |x-5$ can therefore be *read* from left to right: the overall result is 3 times the result of the remaining phrase, which is the quotient of p and the part following the $\%$, and so on.

Parsing proceeds by moving successive elements (or their *values* in the case of pronouns and other *names*) from the tail end of a *queue* (initially the original sentence prefixed by a left marker $[$) to the front of a *stack*, and eventually executing some eligible portion of the stack and replacing it by the result of the execution.

For example, if $\alpha = . \ 1 \ 2 \ 3$, and if $\emptyset$ separates the queue and stack, then $b = . + / 2 * \alpha$ would be parsed and executed as follows:

```
[ b = . + / 2 * α 0
[ b = . + / 2 * 0 1 2 3
[ b = . + / 2 0 * 1 2 3
[ b = . + / 0 2 * 1 2 3
[ b = . + 0 / 2 * 1 2 3
[ b = . + 0 / 2 4 6
[ b = . 0 + / 2 4 6
[ b 0 = . + / 2 4 6
```

```

=
[ b 0 =. 12
[ 0 b =. 12
[ 0 12
0 [ 12

```

The foregoing illustrates two major points: 1) Execution of the phrase $2*1\ 2\ 3$ is deferred until the next element (the $/$) is transferred; had it been a conjunction, the 2 would have been *its* argument, and the monad $*$ would have applied to $1\ 2\ 3$; and 2) Whereas the value of the name α is moved to the stack, the name b (because it precedes a copula) moves unchanged. Finally, a side effect is that the pronoun b is assigned the value 12.

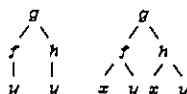
The executions in the stack are confined to the first four elements only, and eligibility for execution is determined only by the class of each of these elements (noun, verb, etc.). The parsing rules are prescribed more formally in Table 2.

The classes of the first four elements of the stack are compared with the first four columns of the parse table, and the first row that agrees in all four columns is selected. The bold elements in the row are then subjected to the action prescribed in column 5, and are replaced by its single result.

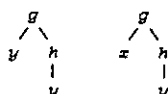
F. IDIOMS

Any phrase, such as $(+*-)$, which cannot be evaluated according to the normal grammatical rules, can, without contradiction, be assigned any idiomatic significance desired. The idioms *fork* and *hook* are defined by the following diagrams, using f , g , and h to denote functions (i.e., verbs):

FORK



HOOK



For example, $5(+*-)3$ is $(5+3)*(5-3)$.

The sixth line of Table 2 shows the conditions for a fork: three verbs in the stack preceded by anything but a conjunction (which would, of course, seize the leading verb as *its* argument).

5

<

III. DEFINITIONS

Each main entry begins with a one-line header such as:

```

< Ranks: _ 0 0 Box ; LessThan
u/. Ranks: _ _ _ Oblique ; Key
u@v Ranks: mv lv rv Atop

```

The header begins with the word being defined, accompanied (in the case of adverbs and conjunctions) by letters that specify the case under consideration: n and v for noun and verb right arguments; m and u for left arguments.

The ranks of the verb or derived verb are shown with the monadic rank followed by the left and right ranks. An infinite rank is denoted by $_$, and ranks dependent on the ranks of argument verbs and nouns are shown as mv , lv , rv (for rank of n), etc.

The header concludes with suggested names for the entry. Except for minor re-ordering due to the grouping of related verbs (such as $+ *$ - $\%$), the entries are presented in the order of the language summary of Table 1, an order also shown at the foot of each page. Arguments in the definitions are denoted by x and y .

```

= Ranks: _ 0 0 Classify ; Equal
MONAD. =y classifies the items of the nub of
y according to equality with the items of y,
producing a boolean table of shape #~.y by
#y. For example:

```

y	~.y	=y
abc	abc	1 0 1
def	def	0 1 0
abc		

DVAD. $x=y$ yields 1 if x is (tolerantly) equal to y . See $<:$.

```

=, =: Is(Local)/Is(Global)

```

Copulas, for use as in $vol=. 5*area=. 3*4$ and $SUM=. +/$. The $=$ is *local*, and $=:$ is *global*, as discussed under $=:$. Copulas may also be used *indirectly*. For example, if $x=. 'abc'$; $'de'$ and $(x)=. 3\ 4; 5\ 6\ 7$ then the values 3 4 and 5 6 7 are assigned to *abc* and *de*. Similarly, $'ab'=. i.2$ assigns 0 to a and 1 to b and $(2\ 3\ 5\ 'abcdef')=. i.2\ 3$ assigns 4 to e etc.

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

>

< Ranks: _ 0 0 Box ; LessThan
 MONAD. < y is an *atomic encoding* of y in the sense that it has rank 0 and can be decoded (by >). See boxed nouns in Section II.A.
 DYAD. $x < y$ is 1 if x is (tolerantly) less than y . See <:.

> Ranks: 0 0 0 Open ; LargerThan
 MONAD. Open (>) is the inverse of box (<), that is, $y > x$ is $> < y$. When applied to an open array (that has no boxed elements), open has no effect. Opened atoms are brought to a common shape as discussed in Section II.B.
 DYAD. $x > y$ is 1 if x is (tolerantly) larger than y . See <:.

<. >. Ranks: 0 0 0 Floor/Ceiling ; Min (LesserOf)/Max
 MONAD. <. y gives the *floor* or *integer part* of y ; thus, <. 3.1 _3.1 is 3 _4. The implied comparison with integers is tolerant (see <:), and <. y is therefore the largest integer such that (<. y) <: y . The *ceiling* >. y is defined by <. - y . See <: for tolerance, and McDonnell[5] for the case of complex arguments.
 DYAD. $x < . y$ yields the lesser of x and y , and $x > . y$ yields the larger. For example, 3 <. 4 _4 is 3 _4.

<: >: Ranks: 0 0 0 Inc(rement)/Dec ; LsOrEqual/LgOrEqual
 MONAD. <: y is $y-1$, and >: y is $y+1$. Also see -..
 DYAD. The result of $x <: y$ is 1 if x is less than or equal to y , and is otherwise 0. However, the comparison is made with a certain *tolerance* t normally set to 2 to the power _44 but also controlled by the *custom* conjunction δ ; as in $x \delta : 0.125 y$. Formally, $x <: y$ is 1 if $|x-y|$ does not exceed t times the larger of the magnitudes of x and y . Tolerance applies similarly to other relations, in particular to greater than or equal to (>:), to match and floor (-: and <.), but not to grade (/:).

The _ is not a verb, but denotes a negative number (as in _3.4), infinity (when used alone), or negative infinity (in _).

6

%.

+ * - % Ranks: 0 0 0 ; Plus/Times/Minus/Divide
 MONAD. The following definitions and examples apply:

Conjugate + y is (y^*) y +3j5 is 3j_5
 Signum * y is $y \div |y|$ *_3 0 5 is _1 0 1
 Minus - y is 0- y -7 is _7
 Reciprocal % y is $1 \div y$ %4 is .25
 DYAD. These are defined as in elementary arithmetic except that 0%0 is 0. See McDonnell[6].

+ . * . + : * : Ranks: * 0 0; Gcd(Or)/Lcm(And)/Nor/Nand
 DYAD. $x * . y$ is the least common multiple of x and y , and $x + . y$ is the greatest common divisor. For boolean (0 and 1) arguments, +. is equivalent to *or*, and *. to *and*. Thus 0 0 1 1 +. 0 1 0 1 is 0 1 1 1, and 0 0 1 1 *. 0 1 0 1 is 0 0 0 1. Finally, $x + : y$ is $-x + . y$, and $x * : y$ is $-x * . y$.

- . Ranks: 0 _ _ Not(1-); Less
 MONAD. - y is equivalent to $1-y$ and, in the case of a boolean argument, to logical negation.
 DYAD. $x - . y$ are all the items of x except for those that occur as cells of y .

- : Ranks: * _ _ ; Match
 DYAD. -: yields 1 if its arguments match completely, in shapes, boxing, and elements; but using tolerant comparisons. See <:.

% . Ranks: 2 _ 2 MatrixInv ; MatrixDiv
 MONAD. If y is a non-singular matrix, then %. y is the inverse of y , that is, (%. y) + / . . * y is the identity matrix $id = . = i . \# y$.

More generally, %. y is defined in terms of the dyadic case as $id \% . y$, or, equivalently, by the relation $(x \% . y) - : (% . y) + / . . * x$. The shape of %. y is $|. \# y$. The degenerate vector and scalar cases are defined by using ; y , but the shape of the result is $\$ y$. For a non-zero vector y , the result of %. y is a vector collinear with y whose length is the reciprocal of the length of y .

DYAD. If the columns of y are linearly independent, and if # x and # y agree, then $x \% . y$

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

%

is defined to minimize the elements of $r = x + d \cdot x + d = x - y + \dots * x \% y$. If y is square, it is necessarily non-singular (since its columns are independent), the elements of r are all zero, and $y + \dots * x \% y$ matches x . As in the monadic case, degenerate cases of y are treated as y .

Geometrically, $y + \dots * x \% y$ is the projection of the vector x on the column space of y , the point in the space spanned by the columns that is nearest to x . Common uses of $\%$ are in the solution of linear equations, and in the approximation of functions by polynomials (as in $c = (f \ x) \% x \wedge / z.4$).

%: Ranks: 0 0 0 Square Root; Root

The x -root of y (denoted by $x \% y$) is equivalent to $y \wedge x$. Thus, $3 \% y$ is the cube root of y . The *square root* (denoted by $\% y$) is equivalent to $2 \% y$.

^: Ranks: 0 0 0 Exponential/Natural Log; Power/Log

MONAD. The *exponential* ($\wedge y$) is equivalent to $e \wedge y$, where e is Euler's number $\wedge 1$ (approximately 2.71828). The *natural logarithm* ($\wedge .$) is inverse to $\wedge$, that is, $y = \wedge . \wedge y$, and $y = \wedge \wedge . y$.

DYAD. $y \wedge 2$ and $y \wedge 3$ and $y \wedge 0.5$ are the *square*, *cube*, and *square root* of y . The general definition of $x \wedge y$ is $\wedge y * \wedge . x$, and applies for complex numbers as well as real. For a non-negative integer right argument it is equivalent to $* / y \# x$; in particular, $* /$ applied to an empty list is 1, and $x \wedge 0$ is 1 for any x , including the case where $x = 0$.

The *base-x logarithm* $x \wedge . y$ is the inverse of power in the sense that $y = x \wedge . x \wedge y$ and $y = x \wedge x \wedge . y$.

u^:n Ranks: mu lu ru Power; Chain

MONAD. The verb u is applied n times. For example, $- / \wedge : 2 \ y$ is $- / - / y$. A negative value of n denotes $|n|$ applications of the obverse (normally the inverse) of u . Infinite n produces the limit of the application of u . Thus if $y = 1$, then $2 \& o. \wedge : y$ is .73908, the solution to the equation $y = \cos y$.

7

~:

DYAD. $x \ u \wedge : n \ y$ is defined as:

$(x \ u \wedge : (n-2) y) \ u \ (x \ u \wedge : (n-1) y)$

where $x \ u \wedge : 0 \ y$ and $x \ u \wedge : 1 \ y$ are defined as y and $x \ u \ y$. Thus, $0 + \wedge : (i.8) 1$ yields fibonacci numbers. See Tu and Perlis [7].

u^:v Ranks: mu lu ru Power; Chain

$u \wedge : v \ y$ is equivalent to $u \wedge : (v \ y) \ y$, and $x \ u \wedge : v \ y$ to $x \ u \wedge : (x \ v \ y) \ y$.

\$: Ranks: _ 1 _ ShapeOf; Shape

MONAD. $\$y$ yields the shape of y , as defined in Section II.A.

DYAD. The shape of $x \$ y$ is $x, s i y$, where $s i y$ is the shape of an item of y . For example:

y	$2 \ 2 \$ y$
abcd	abcd
efgh	efgh
ijkl	
	ijkl
	abcd

These show that the result is formed from the items of y , the last 1-cell ($abcd$) showing that the selection of items is cyclic.

\$. \$: Suite/SelfReference

See :: (General Definition).

u~ Ranks: lv>.rv rv lv Both; Cross

The adverb $\sim$ commutes or crosses the connections to the argument(s). Thus, $(x \ u \sim y) = (y \ u \ x)$ and $(u \sim y) = (y \ u \ y)$.

~. Ranks: _ * * Nub;

MONAD. $\sim . y$ selects the nub of y , that is, all of its distinct items:

y	$\sim . y$	$\sim . 3$
ABC	ABC	3
ABC	DEF	$\$ \sim . 3$
DEF		1

More precisely, the nub is obtained by selecting the leading item, suppressing from the argument all items (tolerantly) equal to it, selecting the leading remaining item, and so on.

~: Ranks: _ 0 0 NubSieve; NotEqual

MONAD. The result of $\sim : y$ is a boolean list b

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

such that $b \# y$ yields the nub of y , that is, $\sim y$.
 DYAD. $x \sim y$ is 1 if x is (tolerantly) unequal to y . See <:.

| Ranks: 0 0 0 Magnitude ; Residue
 MONAD. The magnitude of y is $\%y + y$. For example, $|_6 3j4$ is 6 5.

DYAD. The familiar use of residue is in determining the remainder in dividing a non-negative integer by a positive integer. Thus:

3 | 0 1 2 3 4 5 6 7 8
 0 1 2 0 1 2 0 1 2

The definition :

$$y - x * < . \% x + 0 = x$$

extends this notion to a zero left argument (which yields the right unchanged), and to negative and fractional left arguments. For a negative left argument, the result ranges between the left argument and zero, as it does for a positive left argument. For example:

$\begin{array}{r} .3 | .4 .3 .2 .1 0 1 2 3 4 \\ .1 0 .2 .1 0 .2 .1 0 .2 \\ .1 | 2.5 3.6 4 2 .1 .6 \\ 0.5 0.6 4 0 0 4 \end{array}$

However, to produce a true zero for cases such as $(\%3) | (2\%3)$, the residue is made tolerant by the following definition: $x | y$ is $y - x * < . s$ if $(x \sim 0) * . (> . s) \sim : < . s = . \% x + x = 0$ and is otherwise $y * x = 0$.

For example, $.1 | 2.5 3.6 4 2 .1 .6$ is 0 0.04 0 0. The definition also applies to complex numbers, using the properties of the floor on complex arguments.

| . Ranks: _ 0 _ Reverse ; Rotate

$|.y$ reverses the order of the items of y , and $k|.y$ rotates them:

$\begin{array}{cccc} y & |.y & 1|.y & \sim 1|.y \\ abcd & ijkl & efgh & ijkl \\ efgh & efgh & ijkl & abcd \\ ijkl & abcd & abcd & efgh \end{array}$

| : Ranks: _ 1 _ Transpose

MONAD. This verb reverses the order of the axes of its argument, and is equivalent to $(i. - \$\$ y) | : y$.

DYAD. $x | : y$ moves axes x of y to the tail end. If x is boxed, the axes in each box are "run

8

::

together" to produce a single axis in the result. For example:

$\begin{array}{ccc} y & 2 1 | : y & (< 2 1) | : y \\ abcd & aei & afk \\ efgh & bfj & mrw \\ ijkl & cgk & \\ & dhl & | : i. 3 4 \\ mnop & & 0 4 8 \\qrst & mqu & 1 5 9 \\ uvwx & nrw & 2 6 10 \\ & osw & 3 7 11 \\ & ptx & \end{array}$

$u. . v$ Ranks: 2 _ _ Determinant;
 DotProduct

MONAD. The expressions $-/.*$ and $+/.*$ are the determinant and permanent of square matrix arguments. More generally, $u. . v$ is defined in terms of a recursive expansion by minors:

$(\$ = . 1 + 0 = \# y . ' ; b ; ' y . ') :: ' '$

where b is

$'(0(|:y)u. . v\$(< > > \{0; \sim i. \# y. \}) \{y. '$

DYAD. For vectors and matrices, $x + / . * y$ is equivalent to the dot, inner, or matrix product of mathematics, and other rank-0 functions such as $< . + .$ and $*$ are treated analogously. Thus, $x u. . v y$ is defined by $u x v" (i v, \sim 1) " (1 + i v, \sim 1) y$.

In other words, u is applied to each of the items produced by applying v to each item of the list of "left-argument cells" and the items of y . The shapes of x and y must be such that the number of items in these left and right arguments agree. For example, if the ranks of v are 2 and 3, the shape of the result of u is sr , and the shapes of x and y are 2 3 4 5 6 and 4 7 8 9 10 11, then the shape of the result is 2 3 7 8, sr .

$u : . v$ Obverse

The result of $u : . v$ is the verb u but with an assigned obverse v (used as the "inverse" under the conjunctions $"$ and $\wedge$).

$m : : n$ Ranks: _ _ _ General
 Definition

$= < > _ + * - \% \wedge \$ \sim | . : , ; \# @ / \setminus \{ \} " \& ! ? ()$

::

9

;

If m is non-numeric, the conjunction $::$ produces a verb whose monadic and dyadic cases are determined by m and n , respectively. If $f = '2\$y.'$ $:: 'y.^x.'$ then f 64 is the square root and $3 f$ 64 is the cube root of 64. As illustrated by the foregoing, x and y denote the arguments and $\$$ denotes the function being defined, thus providing the "self-reference" that is also useful in recursive definition. Also see $x0$: and $y0$:

In general, m and n are boxed lists, the sentences in the boxes being executed in a sequence determined by the suite $\$$. An open list (as illustrated above) is treated as a single box, and the items of an open table are treated as a list of boxes. The following example shows the definition of an adverb g :

```

b = 'x.' :$ = . y. #1'
c = 'x.' :$ = (x., 0)+0, x.'
g = '1$y.' :: (b;c)
  1 g 4          g 4
  1 4 6 4 1      1 4 6 4 1

```

The suite $\$$ determines the sequence in which the sentences are executed. It is initially set to $i.ns$, where ns is the number of sentences; in the present example, it is reset to execute sentence 1 the number of times specified by the right argument. The explicit result of the verb is the result of the sentence executed last.

Any name assigned by the copula $=$ is made strictly local on its first assignment; that is, values assigned to the name have no effect on the use of the name outside of the verb or within other verbs invoked by it. The names x , y , $\$$, $\$$ are also local.

If sentence k begins with a name followed by a right parenthesis, the name is made local and set to k . $i.ns$. Such a label is useful in setting the suite to effect branching.

Adverbs. If m is 1, then $m::v$ yields an adverb. If $r = '1::\{.\&x.\&\}.'$, then $\wedge r$ 1 2 3 4 yields 2 3 1 4.

Conjunctions. If m is 2, then $m::n$ produces a conjunction. For example, if:

```

p = '$ = 1, y. #2'
q = 'a = { : ' ; ' a = x. &a'
pow = 2 :: (p;q)

```

then $o.pow$ 2 (1) is the square of pi .

$m::v$ $u::n$ $u::v$ Definition

The first argument specifies the monadic case, and the second argument the dyadic case, using in m and n the conventions defined under the case $m::n$ above.

Ranks: $_{-} \quad _{-} \quad _{-}$ Ravel ; Append
 MONAD. y gives a list of the atoms of y in "normal" order; the result is ordered by items, by items within items, etc. The resulting shape is $1\$*/\y . Thus, $(i. 2 3 4)-: i.* / 2 3 4$.
 DYAD. x, y appends items of y to items of x after 1) Reshaping one atomic argument to the shape of the items of the other, 2) Bringing the items to a common rank (of at least 1) by repeatedly itemizing $(, :)$ any of lower rank, and 3) Bringing them to a common shape by using *overtake* $(\{.)$. Compare with the "permissive frame-building" in Section II.B. For example, if $y = i. 2 3$:

'abc', 'd'	5 6 7, y	y, 7
abcd	5 6 7	0 1 2
	0 1 2	3 4 5
	3 4 5	7 7 7

Ranks: $_{-} \quad * \quad *$ Raze;
 MONAD. y assembles along a leading axis the opened elements of the ravel of y . For example:

y		
i	sing	of

i, y
is ing of

Ranks: $_{-} \quad _{-} \quad _{-}$ Itemize ; Laminate
 MONAD. y adds a single unit axis to y , making the shape $1, \$y$.

DYAD. An atomic argument in x, y is first reshaped to the shape of the other; the arguments are then itemized and appended, as in $(, :x), (, :y)$.

Ranks: $_{-} \quad _{-} \quad _{-}$ Table ; Link
 MONAD. y is a table formed by ravelling the items of y . In particular, the result has shape 1 1 if y is an atom.

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

; .

10

.

DYAD. $x;y$ is $\langle x \rangle, y$ if y is boxed, and $\langle x \rangle, < y$ if y is open.

$u; .n$ Ranks: $_ \ 1/2 _ \text{Cut}$

Cases $; .0$ and $; .3$ have left rank 2. Others have left rank 1.

MONAD. The phrase $u; .1 y$ applies u to each of a set of intervals of items of y to produce the items of the result. Each interval begins at an occurrence of the *delimiter* $0\{y$. For example:

$<; .1 y = \text{' worlds on worlds'}$

	worlds	on	worlds
$b = .5 \ 3\$i.9$			
$\$; .1 y$	b		$(+/.; .1)b$
7	0 1 2		9 12 15
3	3 4 5		3 5 7
7	6 7 8		
	0 1 2		
	3 4 5		

The phrase $u; .1 y$ differs only in that the delimiters are excluded from the intervals. In the cases $u; .2$ and $u; .2$ the delimiter is the *last* item, and marks the ends of intervals.

The phrase $u; .0 y$ applies u to y after reversing y along each axis, and is equivalent to $(0 _1 * / \$y)u; .0 y$.

The monads $u; .3$ and $u; .3$ apply u to tessellation by "maximal cubes", that is, they are defined by their dyadic cases using the left argument $(\$ \$y)\$ < . / \y .

DYAD. The dyads $u; .1$ and $u; .1$ and $u; .2$ and $u; .2$ differ from the monads in that the intervals are delimited by the *ones* in the boolean argument x . For example, if $x = .0 \ 1 \ 0 \ 0 \ 1$ and $y = .i.5 \ 3$:

y	$x + / .; .1 y$
0 1 2	18 21 24 7
3 4 5	12 13 14
6 7 8	$x + / .; .2 y$
9 10 11	3 5 7
12 13 14	27 30 33

The phrase $x \ u; .0 y$ applies u to a "rectangle" or "parallelepiped" of y with one vertex at the point in y indexed by $v = .0\{x$, and with the opposite determined as follows: the

dimension is $|1\{x$, but the rectangle extends back from v along any axis k for which the index $k\{0\{x$ is negative. Finally, the order of the selected elements is reversed along each axis k for which $k\{1\{x$ is negative. For example, $(1 _2, _2 \ 3)+; .0 \ i.4 \ 5$ is 11 12 13; 6 7 8.

The cases $u; .3$ and $u; .3$ provide (possibly overlapping) tessellations. The phrase $x \ u; .3 y$ applies u to each *complete* rectangle of size $|1\{x$ obtained by beginning at all positions obtained as integer multiples of (each element of) the *movement* vector $0\{x$. As in $u; .0$, reversal of each piece occurs along an axis for which the "window" $1\{x$ is negative.

The degenerate case of a list x is equivalent to the left argument $1, :x$, and therefore provides a complete tessellation of size x .

The case $u; .3$ differs from $u; .3$ only in that any shards of sizes less than $|1\{x$ are included. For example:

y	$(3 \ 2; 2 \ 3)<; .3 y$
abcde f	
ghijkl	
mnpq r	
stuvw x	
yzABCD	

abc	cde	ef
ghi	ijk	kl
stu	uvw	wx
yzA	ABC	CD

$;;$ Ranks: 1 ** Words

MONAD. $;; y$ is the list of boxed words in the list y according to the rhematic rules of Part I.

Ranks: $_ \ 1 _ \text{Tally; Copy}$

MONAD. $\#y$ is 1 if y is an atom, and is otherwise the number of items in y .

DYAD. If the arguments have an equal number of items, then $x\#y$ copies $+x$ items from y , with $i\{x$ repetitions of item $i\{y$; if one is an atom, it is repeated to make the item count of the arguments equal; if both are atoms, they are treated as one-element lists.

#. Ranks: 1 1 1 Base2; Base-x

MONAD. The base-2 value of y , that is, $2\#y$.

DYAD. A weighted sum of the items of y ; that is, $+ / w * y$, where $w = . * \{ .x, 1$. For example,

$= < > _ + * - \% \wedge \$ \sim | . : , ; \# @ / \{ \} " \& ! ? ()$

#:

11

10 #. 1 2 3 is 123, and 24 60 60 #.
1 2 3 is 3723.

#: Ranks: _ 1 0 Antibase2; Antibase
MONAD. #: y is the representation of y in base
2, equivalent to $(m\#2)\#y$, where m is the
maximum of the number of digits needed to
represent the elements of y in base 2. For
example:

```

|: #: i.8
0 0 0 0 1 1 1 1
0 0 1 1 0 0 1 1
0 1 0 1 0 1 0 1

```

DYAD. For simple cases, #: is inverse to #.
and, in general, $x\#.\#y$ is $(*/x)y$.

$u@v$ Ranks: mv lv rv Atop

The monad $u@v$ y is equivalent to $with(u@v$
 $y)$. The dyad $x u@v y$ is defined as $x x v$
 y . For example, $3|@-7$ is 4.

@. Ranks: 1 1 _ Permute

MONAD and DYAD. If p is a permutation of the
atoms of $i.n$, then p is said to be a permutation
vector of order n , and if $n=\#b$, then $p\{b$
is a permutation of the items of b .

@. p yields a list of boxed lists of the atoms of
 $i.\#p$ called the *standard cycle representation*
of the permutation p . For example, if $p=. 4$
 $5 2 1 0 3$, then $@.p$ is $(,2);4 0;5 3$
1 because the permutation p moves to position
2 the item 2, to 4 the item 0, to 0 the item 4,
to 5 the item 3, to 3 the item 1, and to 1 the
item 5. The monad @. is self-inverse; applied
to a standard cycle it gives the corresponding
direct representation.

A given permutation could be represented by
cycles in a variety of ways; the standard form
is made unique by the following restrictions:
the cycles are disjoint and exhaustive (i.e., the
atoms of the boxed elements together form a
permutation vector); each boxed cycle is ro-
tated to begin with its largest element; and
the boxed cycles are put in ascending order
on their leading elements.

Non-Standard Representations. If p and c
are standard and cycle representations of order
 $\#b$, then $p@.b$ and $c@.b$ produce the corre-

sponding permutations of items of b . More
generally, since the tally of b determines the
order of the permutation, the arguments p and
 c may be non-standard in ways to be defined.
In particular, negative integers down to $-\#b$
may be used, and are treated as their residues
modulo $\#b$.

If q is not boxed, and if the elements of
 $(\#b)|q$ are distinct, then $q@.b$ is equivalent
to $p@.b$, where p is the standard form of q
given by $p=. ((i.n)-.n|q),n|q$, where
 $n=. \#b$. In other words, positions occurring
in q are moved to the tail end.

If q is boxed, the elements of $(\#b)|>f\{q$
must be distinct for each f , and the boxes
are applied in succession. Thus, $(2 1;3 0$
 $1)@.i.5$ is $(<2 1)@.(<3 0 1)@.i.5$,
and the result of either is the standard direct
permutation 1 2 3 0 4.

The transformation provided by the monad
@. is extended to non-negative non-standard
cases by treating any argument q as a represen-
tation of a permutation of order $1+>./q$.

@: Ranks: 1 0 _ Atom;AtomPermute

MONAD and DYAD. If T is the table of all $1:n$ per-
mutations of order n arranged in lexical or-
der (that is, $/:T$ is $i.n$), then k is said to
be the *atom representation* of the permutation
 $k\{T$. Moreover, $k@.b$ permutes items of b by
the permutation of order $\#b$ whose atom rep-
resentation is $(\#b)|k$. For example, $1@:b$
transposes the last two items of b and $1@:b$
reverses all items, and $3@:b$ and $4@:b$ rotate
the last three items of b . Finally, $(i.n)$
 $@:i.n$ produces the ordered table of all per-
mutations of order n , as does $(i.@:i.n)n$.

The monad @: applied to any cycle or direct
permutation yields its atom representation. For
example, @: 0 3 2 1 is 5, as are @: 3 2 1
and @: 0;2;3 1 and @: <3 1.

$u/$ Ranks: _ _ _ Insert; Function Table
MONAD. u/y applies the dyad u between the
items of y . Thus:

```

i. 3 2 +/i. 3 2 */1 2 3
0 1 6 9 6

```

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

/.

2 3

4 5

If y has no items (that is, $0=\#y$), the result of u/y is the *identity element* of the verb u . An identity element of a function u is a value e such that either $x:-x u e$ or $x:-e u x$ for every x in the domain (or perhaps some significant sub-domain such as boolean) of u . This definition of insertion over an argument having zero items, extends partitioning identities such as $(+/y) -: (+/k\{.y\})+(+/k\{.y\})$ to the cases $k=.0$ and $k=.#y$.

The *identity function* of a verb u is a function ifu such that $(ifu y) -: (u/y)$ if $0=\#y$. Denoting v inverse by iv , the identity functions are:

```

$&0$}.&$< > + - +.~:|
$&1$}.&$ = <:;> * % *.% ^ !
$&_&}.&$< .
$&_&_&}.&$> .
i.&(0$,.&(&2$).&$ ,
i.&(1$&}.&$.&$ @. {
=&i.&(1$&}.&$.&$ %./...*
ifu&$ u/
$&(iv iv $0$&}.&$ u"v

```

DYAD. If x and y are numeric lists, then $x * y$ is their multiplication table. For example:

```

1 2 3 * / 4 5 6 7
4 5 6 7
8 10 12 14
12 15 18 21

```

In general, each cell of x is applied to the entire y , that is, $x u/y$ is $x u"(\text{lu},_)y$.

$u/$. Ranks: _ _ Oblique ; Key

MONAD. If $a=.i.3 4$, then $</.a$ is $(,0);1 4;2 5 8;3 6 9;7 10;11$, that is, the $(3+4)-1$ elements of the result are obtained by applying $<$ to each of the oblique lines of y . Hence, if p and q are the coefficients of two polynomials, $+//.p*/q$ yields the coefficients of the product polynomial.

More generally, $u/$. y is the result of applying u to the oblique lines of $_2$ -cells of y . If the rank of y is less than two, y is treated as the table; y .

DYAD. $x u/$. y is $(=x)u\&\#y$, that is, items

12

\.

of x specify keys for the corresponding items of y , and u is applied to each of the collections of y having identical keys. For example, $1 2 3 1 3 2 1 </.'abcdefg'$ yields $'adg';'bf';'ce'$.

/: \: Ranks: _ 1 _ GradeUp/Grade Down;Sort

MONAD. $/: grades$ its argument, yielding a permutation vector such that $(/:y)\{y$ sorts y in ascending order. For example:

```

{:g=. /:y=. 3 1 4 2 1 3 3
1 4 3 0 5 6 2
g\y
1 1 2 3 3 3 4

```

As illustrated above, elements of g that select equal elements of y occur in ascending order.

If y is a table, $/:y$ grades the base value of the rows, using a base larger than the magnitude of any of the elements. A y of higher rank is treated as $;y$, that is, as if its items were each ravelled.

If y is literal, $/:y$ grades according to the collating sequence specified by the alphabet a ; any other collating sequence cs can be imposed by grading the result of $cs i. y$.

Downgrade is like upgrade, except that the items of $(\backslash:y)\{y$ are in descending order.

DYAD. $x /: y$ is defined by $(/:y)\{x$; that is, x is sorted into an order specified by y . In particular, $y/:y$ (or $/:~y$) sorts y .

$u\backslash u\backslash$. Ranks: _ 0 Scans:Prefix/Suffix;Infix/Outfix

MONAD. $u\backslash y$ has $\#y$ items resulting from applying u to each of the prefixes $k\{.y$, for k from 1 to $\#y$. Thus, $<\backslash'abc'$ yields $(, 'a')$; $'ab'$; $'abc'$, and $+/\backslash 1 2 3 4$ is $1 3 6 10$.

$u\backslash$. y has $\#y$ items resulting from applying u to suffixes of y , beginning with one of length $\#y$ (that is, y itself), and continuing through to a suffix of length 1.

DYAD. If a is positive, then the items of $a u\backslash y$ result from applying u to each infix of length a . If a is negative, then u is applied to the non-overlapping infixes of length $|a|$, including a possible final shard. Thus, $2<\backslash q=. 'abcde'$

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

{ 13 }

is 'ab'; 'bc'; 'cd'; 'de', and $_2 \backslash q$ is 'ab'; 'cd'; 'e'.

If α is positive in the expression $\alpha \backslash y$, then α applies to the outfixes of y obtained by suppressing successive infixes of length α . If α is negative, the outfixes are determined by suppressing non-overlapping infixes, the last of which may again be a shard. For example, $2 \backslash .q$ is 'cde'; 'ade'; 'abe'; 'abc'.

{ Ranks: 1 0 _ Catalog;From

MONAD. $\{y\}$ forms a catalog from the atoms of its arguments, its shape being the chain of the shapes of the opened items of y , and the common shape of the boxed results is $\$y$. The case $\{a; b\}$ is called the *cartesian product* of a and b . Thus:

{ 'ht'; 'ao'; 'gtw'			>{10 11; 2.2 2
hag	hat	haw	10 0
hog	hot	how	10 1
tag	tat	taw	10 2
tog	tot	tow	10 3
			11 0
			11 1
			11 2
			11 3

DYAD. If x is an integer in the range from $-#y$ to $<: #y$, then $x\{y$ selects item $(#y) | x$ from y . More generally, x may be a boxed list, whose successive elements are (possibly) boxed lists that specify selection along successive axes of y . For example:

y	$2 \ 0\{y$	$(<2 \ 0)\{y$
abcdef	mnpqr	m
ghijkl	abcdef	$(<2 \ 0; 1 \ 3)\{y$
mnpqr	np	
	bd	

Finally, if any $r = .>f\{x$ used in the selection is itself boxed, selection is made by the indices along that axis $(i.f\{\$y)$ that do not occur in $>r$. Thus, $(<(<2 \ 0), (<1 \ 3))\{y$ is $gikl$.

$m\} \ u\}$ Ranks: _ _ Merge

The function $m\}$ is equivalent to $m\{\}$.

MONAD. $u\}y$ is a merge of the items of y whose shape is the shape of an item of y . Each atom in the merge is selected as a corresponding atom in an item, the particular item being specified by the index in the corresponding position of $u \ i.\} \$y$ (whose shape must be the shape of an item of y). For example, if $m = .0 \ 1 \ 3 \ 1 \ 2$, and $u = m\{\}$, then:

y	$u \ i.\} \$y$	$u\}y$
abcde	0 1 3 1 2	agrio
fghij		mly
klmno		agrio
pqrst		

DYAD. If x has the same shape as the index array $j = .u \ i.\$y$, then $x \ u\}y$ produces a merge of x and y by inserting the atoms of x in the selected positions of y . More generally, the shape of x may be a suffix of the shape of j , and the result is a merge of $(\$j)\$, x$ and y . For example, if $x = .AGMS'$ and $u = (<0 \ 1)\&|$: (the diagonal function), then:

y	$u \ i.\$y$	$x \ u\}y$	$'Z' \ u\}y$
abcde	0 6 12 18	Abode	Zbode
fghij		fghij	fzhij
klmno		klmno	klnzo
pqrst		pqrst	pqrzt

{ . Ranks: _ 1 _ First;Take

MONAD. $\{ . y$ selects the leading item of y ; that is, y if y is an atom, and $0\{y$ if it is not.

DYAD. If x is an atom, $x\{ . y$ takes from y an interval of $|x|$ items; beginning at the front if $x > 0$, and ending at the tail if $x < 0$. For example:

y	$2\{ . y$	$4\{ . y$	$_4\{ . y$
0 1	0 1	0 1	0 0
2 3	2 3	2 3	0 1
4 5		4 5	2 3
	0 0	4 5	

In an *overtake* (as in $4\{ . y$ and $_4\{ . y$ above), extra items consist of *fills*; zeros if y is numeric, boxed empties if it is boxed, and spaces otherwise. If y is an atom, the result of $1\{ . y$ is a one-element list. Finally, if $\$y$ is 0, s , then the fill items are $s\$0$.

In general, x may be a vector of length no more than $\$y$; the effect of element k is $(k\{x)\} . "((\$y) - k)\}y$.

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

{ :

14

& :

}. Ranks: _ 0 _ Rest;Drop
 MONAD. **}**. y selects the rest of the items of y after the first, and is equivalent to $1 \cdot y$.
 DYAD. $x \cdot y$ drops (at most) x items from y in a manner similar to $\{ \cdot \cdot$

{ : **}**. Ranks: _ _ _ Right/Left

Both monads are nilpotent, yielding their arguments. The dyad **{ :** yields its right argument, and **}**: the left.

m"n Ranks: |.3\$ |.n Constant

The ranks of the derived verb are specified directly by n if it has three elements, and otherwise by $|.3$ |.n$. Thus a single element specifies all ranks, and the last of two elements specifies the monadic and right ranks. The derived verb **m"n** produces the constant result m for each cell to which it applies.

u"n Ranks: |.3\$ |.n Rank

The function **u"n** is identical to **u** except that its ranks are determined by n as specified under **m"n**.

u"v Ranks: mv lv rv Rank

The function **u"v** is equivalent to **u** except that its ranks are the ranks of v .

". Ranks: 1 1 1 Execute

MONAD. **"**. y is the result of executing the character list y . Thus, **"**. 'a=.3+4' assigns the value 7 to the name a and yields the explicit result 7. The result of **"**. '' is the empty list.
 DYAD. $x \cdot y$ is the result of **"**. y if y is a valid sentence (whose execution finishes properly). Otherwise it is the result of **"**. x , which may invoke an error report in the normal manner.

" : Ranks: _ 1 1 Format

MONAD. **" :** y is equivalent to $x \cdot y$, where x is chosen to provide a minimum of one space between columns. Default output is identical to this monadic case.

DYAD. $x \cdot y$ produces characters from the alphabet α . that represent y in a format specified by x . Each element e of x controls the representation of the corresponding element of

y as follows:

$w = \cdot < \cdot |e$ specifies the total width allocated; if this space is inadequate, the entire space is filled with asterisks. If e is zero, enough space is allocated.

$d = \cdot < \cdot 10 * (|e) - w$ specifies the number of digits following the decimal point (which is itself included only if d is not zero).

Any negative sign is placed just before the leading digit.

If $e > 0$, the result is right justified in the space w .

If $e < 0$, the result is put in exponential form (with one digit before the decimal point) and is left-justified except for two fixed spaces allowed on the left (including one for a possible negative sign).

Boxed elements are displayed in boxes, as shown in Section II.A.

The custom conjunction specifies the number of digits of precision for real numbers. For example, **" : 4 (5\$3)** yields 1.667.

m&v u&n Ranks: (rm>.rv)/(lu>.m) **
 With

$m \& v \ y$ is defined by $m \ v \ y$, and $u \& n \ y$ by $y \ u \ n$.

u&v Ranks: mv mv mv With

MONAD. **u&v** y is defined by $u \ v \ y$, applying individually to the cells of rank mv .

DYAD. $x \ u \& v \ y$ is defined by $(v \ x) \ u \ (v \ y)$.

u&.v Ranks: mv mv mv Under

The function **u&.v** is equivalent to the composition **u&v** except that the function obverse to v is applied to the result for each cell. The obverse pairs are specified by the conjunction **:** or by:

+ - x |. | : | : < x. -. . / : < | x. . ^ x :
 + - x |. | : | : > x. -. . / : > | x. . ^ x :
 For example, the phrase $x + \& . \wedge y$ yields the product of x and y , and $|. \& . > y$ reverses each of the boxed elements of y .

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

!

! : Custom

Unlike other conjunctions, this one applies in a manner specific to each verb. For the relations, its right argument specifies the tolerance to be used, as in $< \delta : (2 \wedge 4)$. For other verbs (such as π), the effects are discussed in their definitions.

! Ranks: 0 0 0 Factorial; Outof

MONAD. For a non-negative integer y , the definition of $!y$ is $\ast / \ast : i.y$. In general, $!y$ is $\text{gamma} > y$.

DYAD. For non-negative arguments $x!y$ is the number of ways that x things can be chosen out of y . In general it is defined as $(!y)\% (!x) \ast (!y-x)$, with the understanding that infinities (occasioned by $!$ on a negative integer) cancel if they occur in both numerator and denominator. For example:

```
! / 3 ~ i . 7
1 2 1 0 0 0 0
0 1 1 0 0 0 0
0 0 1 0 0 0 0
1 1 1 1 1 1 1
-3 2 -1 0 1 2 3
-6 3 1 0 0 1 3
-10 -4 -1 0 0 0 1
```

! : External

This conjunction is used to communicate with the operating system as well as with the keyboard (as an input file) and with the screen (as an output file). See Appendix B.

? Ranks: 0 0 0 Roll; Deal

MONAD. $?y$ yields a uniform random choice from $i.y$.

DYAD. $x?y$ yields a list of x items chosen at random without repetition from the list $i.y$.

) Label-Command

This parenthesis sets off a *label* in function definition (See $:$), and introduces a *command* to change certain parameters as detailed in Appendix A.

 α . Alphabet

α . is a list of the elements of the alphabet; it determines the *collating sequence* used in

15

sorting ($/$: and $\backslash$:).

e. Ranks: 0 0 RazeIn; In

MONAD. $e.y$ produces a boolean result that determines for each atom of y whether its open contains each item of the raze of y . Thus:

$y = 'abc'; 'dc'; 'a'$

```
..y      e.y
abcdca    1 1 1 0 1 1
           0 0 1 1 1 0
           1 0 0 0 0 1
```

DYAD. If x has the shape of an item of y , then $xe.y$ is 1 if x matches an item of y . In general, $xe.y$ is $(\#y) > y i.x$. For example:

```
'cat' e. 'abcd'
1 1 0
'cat' e. 2 3 $'catdog'
1
```

E. Ranks: * - - ; In

DYAD. If $b = .x E. y$, then the *ones* in b indicate the beginning points of occurrences of the pattern x in y . For example:

```
'co'E.'cocoa'
1 0 1 0 0
(0 1 + / i . 3) E . 4 | + / ~ i . 5
1 0 0 0 0
0 0 0 0 0
0 0 1 0 0
0 1 0 0 0
0 0 0 0 0
```

i. Ranks: 1 - - Integers; IndexOf

MONAD. The shape of $i.y$ is $|y$, and its atoms are the first $\ast / y$ non-negative integers. For example:

```
i . 2 3      i . 2 _3      i . ' i . _4
0 1 2        2 1 0        0   3 2 1 0
3 4 5        5 4 3
```

As shown in the examples, a negative element in y causes reversal of the atoms along the corresponding axis.

DYAD. If $ri.x$ is the rank of an item of x , then the shape of the result of $xi.y$ is $(-ri.x) . \$y$. Each atom of the result is either $\# x$, or the index of the first occurrence among the items of x of the corresponding $ri.x$ -cell of y .

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

16

O. Ranks: 0 0 0

MONAD. o. *y* yields *pi* times *y*. For example, *o.1* is 3.14159.

DYAD. k o. *y* is one of the circular, hyperbolic, or pythagorean functions, according to the following table:

<i>k</i>	<i>function</i>	<i>k</i>	<i>function</i>
0	%:1- <i>y</i> ²	7	Tanh <i>y</i>
1	Sine <i>y</i>	8	%:1- <i>y</i> ²
2	Cosine <i>y</i>	9	(<i>y</i> + <i>y</i>)%2
3	Tangent <i>y</i>	10	<i>y</i>
4	%:1+ <i>y</i> ²	11	(<i>y</i> + <i>y</i>)%0f2
5	Sinh <i>y</i>	12	(^.* <i>y</i>)%0f1
6	Cosh <i>y</i>		

The function **(-k)o.y** is inverse to **ko.y**. In particular, the cases **_1**, **_4**, **_9**, and **_10**, are *Arcsine y*, *%:1-*y*²*, *y*, and *+y*. The arguments of Sine, Cosine, and Tangent (and the results of their inverses) are in radians.

x. y.

These denote the arguments in a definition as defined for under General Definition (:):

REFERENCES

1. Falkoff, A.D., and K.E. Iverson, *The Design of APL*, IBM Journal of R&D, July 1973 and *The Evolution of APL*, ACM Sigplan Notices, August 1978.
2. Hui, et al., *APL?*, APL90, ACM.
3. Iverson, K.E., *A Dictionary of APL*, ACM APL Quote-Quad, September 1987.
4. Iverson, K.E., *J*, Vector (British APL Assoc.), August 1990.
5. McDonnell, E.E., *Complex Floor*, APL73, North Holland.
6. McDonnell, E.E., *Zero Divided by Zero*, APL76, ACM.
7. Tu, Hai-Chen, and A.J. Perlis, IEEE Software, January 1986.

ACKNOWLEDGMENTS

I am indebted to Mr. Roger Hui for his rapid development of a flexible, reliable, and highly portable implementation of *J*, for his close collaboration in the design of the language, for critical reading of successive drafts of the dic-

tionary, and for significant contributions to the treatments of certain topics, in particular, the identity functions.

Appendix A: SYSTEM COMMANDS

The following commands may be entered at any point in a session; the leading right parenthesis distinguishes them from normal sentences. Brackets indicate that the argument *y* is optional; when omitted, the present value of the parameter is printed.

A right parenthesis may also be used to specify a *label*, as described under the conjunction :: (definition).

)box [y] The eleven-element integer list *y* specifies the indices of the ASCII characters to be used in displaying boxed arrays, in the order:

┌	┐	└	┘	┌	┐	└	┘	┌	┐	└	┘
---	---	---	---	---	---	---	---	---	---	---	---

)clear Restore to initial condition, erasing all names.

)off Terminate session.

)prompt [y] The prompt for immediate input is specified by *y*. The default value is ' ' (that is, three spaces).

)reset Terminate all suspended functions.

)r1 [y] The "random link" used as a "seed" by the random number generator (?) is specified by *y*. The default value is 16807.

)script [y] The argument is one (quoted) file name for input or two (as in '*abc*' '*defg*') for input and output. Except that system commands are ignored, the entire input file is executed as a script before further input is accepted from the keyboard. An unquoted dash (-) for the input file denotes keyboard input. The output is directed to the screen as well as to any designated output file. An output file so generated is suitable as the input in a subsequent command.

)esscript [y] Behaves like *)script*, except that output does not appear on the screen.

)si Stack Indicator, showing functions currently suspended.

= < > _ + * - % ^ \$ % ~ | . : , ; # @ / \ { } " & ! ? ()

|

17

::

Appendix B: EXTERNAL CONJUNCTION

0!:0 y The list y is executed by the host system, and the result is returned.

1!:0 y Directory.

1!:1 y File read: y designates a file to be read, and the result is a string of the file contents; y may be a boxed name suited to the host file system, or it may be the number 1, designating the keyboard as the source file.

x 1!:2 y File write: x is a string, and y is a boxed file name, or the number 2 designating screen output.

x 1!:3 y File append (like x 1!:2 y , but appends rather than replaces).

1!:4 y File size: y is a boxed file name.

1!:55 y File erase: y is a boxed file name.

3!:0 y Storage type of the noun y , encoded as 2^ ϵ .6 for boolean, literal, integer, floating, complex, boxed.

3!:1 y Convert to internal representation (of a noun y).

3!:2 y Convert from internal representation.

4!:0 y Name class of boxed name, encoded as ϵ .7 for undefined (but valid), label, noun, verb, adverb, conjunction, other.

[x] 4!:1 y Name list: result is a list of boxed names belonging to the classes 1 to 5 (as shown in 4!:0) in y . The optional left argument lists the initial letters of names to be included.

4!:55 y The boxed name y is erased.

5!:0 y Fix: (Inverse of 5!:1 and 5!:2).

5!:1 y Canonical representation.

5!:2 y The boxed representation of a boxed name y .

6!:0 y Time stamp: the current time as a list in the order YMD HMS. The last element may be fractional.

6!:1 y Time since start of session.

6!:2 y Time required to execute the sentence y .

6!:3 y Delay for y seconds.

7!:0 y Space currently in use.

7!:1 y Space used since start of session.

7!:2 y Space required to execute the sentence y .

128!:0 y QR decomposition of y in the domain of $\mathbb{R}$, giving a boxed Hermitian matrix and a boxed upper triangle.

128!:1 Invert square upper triangle.

= < > _ + * - % ^ \$ ~ | . : , ; # @ / \ { } " & ! ? ()

			Page
=	Classify ; Equal	Is (Local)	5
<	Box ; LessThan	Floor ; LesserOf	
>	Open ; LargerThan	Ceiling ; LargerOf	6
-	NegSign/Infinity		
+	Conjugate ; Plus	; GCD (Or)	
*	Signum ; Times	; LCM (And)	
-	Negate ; Minus	Not (1-) ; Less	
%	Recip ; Divide	MatrixInv ; MDiv	7
^	Exp ; Power	NaturalLog ; Log	
\$	ShapeOf ; Shape	Suite	
~	Both ; Cross	Nub ;	
	Mag ; Residue	Reverse ; Rotate	8
.		Determ ; DotProd	
:		Obverse	
,	Ravel ; Append	Definition	9
;	RavellItems ; Link	Itemize ; Laminate	
		Cut	10
#	Tally ; Copy	Base2 ; Base	11
@	Atop	Dir-Cyc ; Permute	
/	Insert ; Table	Oblique ; Key	12
\	Prefix ; Infix	Suffix ; Outfix	
{	Catalog ; From	First ; Take	13
}	Merge	Rest ; Drop	
"	Rank	Execute ; Execute	14
&	With	Under	
!	Factorial ; OutOf	External	15
?	Roll ; Deal		
)	Label-Cmd (Appendix A)		
a		Alphabet	
e		Razein ; Member	
i		Integers ; IndexOf	
o		PiTimes ; Circular	
x		First Argument	16
y		Last Argument	
E		; MemberOfInterval	

Table 1: LANGUAGE SUMMARY

[=(av	v	n	d	Monad	LEGEND
c	n	v	n	Monad	a Adv
[=(avm	n	v	n	Dyad	o Conj
[=(avm	nv	a	d	Adverb	n Noun
[=(avm	nv	c	nv	Conjunction	p Pronoun
					v Verb
[=(avm	v	v	v	Fork	[Lmark
[=(	v	v	d	Hook	= Is
np	=	cavn=	d	Is	(Lparen
(	cavn	)	d	Punctuation	) Rparen
d	d	d	d	Get next	d Any

Table 2: PARSING RULES

Densities Well Suited For Statistics Problems

by Dietrich Trenkler, University of Osnabrück, West-Germany

Introduction

Devising examinations in Statistics is a tedious thing to do. For instance, understanding the meaning of the Central Limit Theorem definitely is a goal, but finding continuous distributions for which the student can easily compute expectations and variances is hard to do by trial and error. So why not let the computer do the job. In this paper we introduce several APL programs written in APL*PLUS/PC to help in these circumstances. We hold that this is a further step in the direction of generating examinations which at the same time cover a broad range of topics, are less prone to errors and minimize the efforts on behalf of the instructor.

The Problem

Let us be given a function f defined by

$$f(x) = \begin{cases} a_2 x^2 + a_1 x + a_0, & \alpha \leq x \leq \beta, \\ 0 & \text{else.} \end{cases}$$

Our aim is to give some guidelines for choosing the coefficients a_i such that

- f is a probability density function (p.d.f) and
- they are sufficiently smooth to ease further computations.

There are two sufficient and necessary conditions for f to be a p.d.f.:

- (i) $f(x) \geq 0$ for all x , and
- (ii) $\int_{\alpha}^{\beta} f(t) dt = 1$.

Condition (ii) is equivalent to

$$2a_2(\beta^3 - \alpha^3) + 3a_1(\beta^2 - \alpha^2) + 6a_0(\beta - \alpha) = 6. \quad (1)$$

To guarantee that f meets condition (i) it is necessary that

$$f(\alpha) \geq 0 \quad \text{and} \quad f(\beta) \geq 0. \quad (2)$$

But this is not sufficient if f has a minimum in (α, β) . It is easy to see that f has a global minimum (maximum) in $x_0 = -\frac{a_1}{2a_2}$ if $a_2 > 0$ ($a_2 < 0$). So we impose the restriction $f(x_0) \geq 0$ which is equivalent to

$$4a_0a_2 \geq a_1^2 \quad (3)$$

whenever $a_2 > 0$ and $-2a_2\beta < a_1 < -2a_2\alpha$.

Having chosen α, β, a_2, a_1 and a_0 one can compute expectations associated with the distribution given by f from

$$\begin{aligned} E[X^n] &= \frac{a_2}{n+3}(\beta^{n+3} - \alpha^{n+3}) + \frac{a_1}{n+2}(\beta^{n+2} - \alpha^{n+2}) + \frac{a_0}{n+1}(\beta^{n+1} - \alpha^{n+1}) \\ &= \frac{A}{(n+3)(n+2)(n+1)} \end{aligned} \quad (4)$$

where

$$A = a_2(n+2)(n+1)(\beta^{n+3} - \alpha^{n+3}) + a_1(n+3)(n+1)(\beta^{n+2} - \alpha^{n+2}) + a_0(n+3)(n+2)(\beta^{n+1} - \alpha^{n+1}).$$

The distribution function F is given by

$$F(x) = \begin{cases} 0, & x < \alpha, \\ \frac{a_2}{3}(x^3 - \alpha^3) + \frac{a_1}{2}(x^2 - \alpha^2) + a_0(x - \alpha), & \alpha \leq x \leq \beta, \\ 1, & \beta < x. \end{cases}$$

These formulas can be used to compute variances and percentiles. However, computing percentiles is hard unless $a_2 \neq 0$ and $a_1 = a_0 = 0$ or $a_2 = 0$. Then we get for $0 \leq p \leq 1$

$$F^{-1}(p) = \begin{cases} \sqrt[3]{\alpha^3 + \frac{3p}{a_2}}, & a_2 \neq 0, a_1 = a_0 = 0, \\ -\frac{a_0}{a_1} \pm \sqrt{\left(\frac{a_0}{a_1} + \alpha\right)^2 + \frac{2p}{a_1}}, & a_2 = 0, a_1 \neq 0, \\ \alpha + \frac{p}{a_0}, & a_2 = a_1 = 0. \end{cases} \quad (5)$$

The Programs

Starting point in program *GENERDENSITY* is criterion (1). Using *SUNUP2* we first find a matrix of integers $z_2 \leq z_1 \leq z_0$ such that $z \leq z_i$ for some integer

z fixed in advance and $z_2 + z_1 + z_0 = 6$. This number z has been set to -24 since experience has shown that in this way *WS FULL* problems can be avoided on the author's PC. Then using *PERM* all $3! = 6$ permutations of its columns are generated. This way, there may be some redundancy but *DEDOUBLEROWS* at the end of the program truncates all duplicate lines. The numbers

$$a_2 = \frac{z_2}{2(\beta^3 - \alpha^3)}, \quad a_1 = \frac{z_1}{3(\beta^2 - \alpha^2)}, \quad a_0 = \frac{z_0}{6(\beta - \alpha)}$$

are potential coefficients. To guarantee smooth numbers it is checked if ma_i is integer for some positive integer m . Thus, setting $m = 4$ yields numbers like -2.25, 3.5 or -3 for instance. Finally, those rows of the matrix are extracted which fulfill criteria (2) and (3).

Programs *EXPECTATION* and *DISTRIBUTION* can be used to determine $E[X^n]$ and F .

An Example

Suppose we wish to determine smooth densities based on a polynomial in $[0, 1]$ such that a_2, a_1 and a_0 are integers. Then

□-A-GENERDENSITY 1 0 1

```

-3  2  1
 0  0  1
 6 -8  3
 3 -6  3
 6 -6  2
 3 -4  2
 0 -2  2
 3 -2  1
-6  6  0
-3  4  0
 0  2  0
 9 -8  2
 9 -6  1
 6 -4  1
 3  0  0

```

The right argument of *GENERDENSITY* consists of the numbers m, α and β . For instance, the third line of this matrix consisting of the numbers 6, -8 and 3 indicates that

$$f(x) = \begin{cases} 6x^2 - 8x + 3, & 0 \leq x \leq 1, \\ 0 & \text{else} \end{cases}$$

is a p.d.f.

Using α as the right argument the first four columns of the following expression show the coefficients

$$\begin{aligned} b_3 &= \frac{a_2}{3}, \\ b_2 &= \frac{a_1}{2}, \\ b_1 &= a_0, \quad \text{and} \\ b_0 &= -\alpha\left(\frac{a_2}{3}\alpha^2 + \frac{a_1}{2}\alpha + a_0\right) \end{aligned}$$

of F in $[\alpha, \beta]$:

O DISTRIBUTION A

-1	1	1	0	0
0	0	1	0	3
2	-4	3	0	0
1	-3	3	0	0
2	-3	2	0	0
1	-2	2	0	0
0	-1	2	0	2
1	-1	1	0	0
-2	3	0	0	0
-1	2	0	0	0
0	1	0	0	2
3	-4	2	0	0
3	-3	1	0	0
2	-2	1	0	0
1	0	0	0	1

The fifth column tells which of the formulas in (5) can be used to determine percentiles.

Finally, to compute $E[X^n]$ we can use the program *EXPECTATION*. There are two ways to specify its left argument. If it consists of four entries the first must be 0 or 1. If it is equal to 1 all numbers are displayed to compute the numerator A and denominator in (4). In this way we can gain insight into the complexity of the computations involved. If it is equal to 0, these numbers are truncated and only the expectations are computed. If only three numbers are used then this is equivalent to using four with the first equal to 0. The other numbers are α , β and n , respectively. To compute $E[X]$ we get

1 0 1 1 EXPECTATION A

-18	16	12	10	24	0.4167
0	0	12	12	24	0.5000
36	-64	36	8	24	0.3333
18	-48	36	6	24	0.2500
36	-48	24	12	24	0.5000
18	-32	24	10	24	0.4167
0	-16	24	8	24	0.3333
18	-16	12	14	24	0.5833
-36	48	0	12	24	0.5000
-18	32	0	14	24	0.5833
0	16	0	16	24	0.6667
54	-64	24	14	24	0.5833
54	-48	12	18	24	0.7500
36	-32	12	16	24	0.6667
18	0	0	18	24	0.7500

For instance to compute $E[X]$ of the distribution determined by the p.d.f. mentioned above we find that

$$E[X] = \frac{36 - 64 + 36}{24} = \frac{8}{24} = 0.3333.$$

Finally, to compute the variances $E[X^2] - E^2[X]$ the following expressions can be used.

E2*0 1 2 EXPECTATION A

E1*0 1 1 EXPECTATION A

A, E1, E2, E2-E1*2

-3	2	1	0.4167	0.2333	0.0597
0	0	1	0.5000	0.3333	0.0833
6	-8	3	0.3333	0.2000	0.0889
3	-6	3	0.2500	0.1000	0.0375
6	-6	2	0.5000	0.3667	0.1167
3	-4	2	0.4167	0.2667	0.0931
0	-2	2	0.3333	0.1667	0.0556
3	-2	1	0.5833	0.4333	0.0931
-6	6	0	0.5000	0.3000	0.0500
-3	4	0	0.5833	0.4000	0.0597
0	2	0	0.6667	0.5000	0.0556
9	-8	2	0.5833	0.4667	0.1264
9	-6	1	0.7500	0.6333	0.0708
6	-4	1	0.6667	0.5333	0.0889
3	0	0	0.7500	0.6000	0.0375

***** The Programs *****

```

▽ R←DEDOUBLEROWS MAT;A;B
[1] R←B←⊖((82=ODR MAT)/'DAV'), 'MAT'
[2] R←(1,1↓R≠1⊖R)/R←R[1R]
[3] R←(1+[/,R)1⊖R←R1B
[4] R←(1,1↓R≠1⊖R←R[A])/A←A R
[5] R←((11↑⊖B)←R)/MAT
[6] A Removes all duplicate rows of MAT.
[7] A Considerably faster and less WS FULL prone than
[8] A Finn-APL idioms 70, 206 and 208.
▽

▽ R←ALPHA DISTRIBUTION A;B
[1] R←(A[;1]÷3),(0.5×A[;2]),A[;,3]
[2] R←R,-R+.×ALPHA○.* 3 2 1
[3] B←11↑⊖R←R,0
[4] R[[(A[;1]≠0)^(A[;2]=0)^(A[;3]=0)/B;5]←1
[5] R[[(A[;1]=0)^(A[;2]≠0)/B;5]←2
[6] R[[(A[;1]=0)^(A[;2]=0)/B;5]←3
▽

▽ R←ABN EXPECTATION C;A;B;C;D;E;N
[1] ABN←-4↑ABN
[2] A←ABN[2] ◇ B←ABN[3] ◇ N←ABN[4]
[3] R←C[;1]×(N+2)×(N+1)×(B+N+3)-A*N+3
[4] R←R,C[;,2]×(N+3)×(N+1)×(B+N+2)-A*N+2
[5] R←R,C[;3]×(N+3)×(N+2)×(B+N+1)-A*N+1
[6] R←((⊖R)×1,ABN[1])⊖R←R,A,[1.1]B),(A←+/R)÷B←(N+3)×(N+2)×N+1
▽

```

```

      V R←SUMUP2 abc;A;I;a;b;c
[1]  a←abc[1] ◊ b←abc[2] ◊ c←abc[3]
[2]  R←c ◊ →0×1b=1
[3]  R←(0,b)ρI←a
[4]  M1:→0×1c<+ /bρI
[5]  A←I,SUMUP2 I,(b-1),c-I
[6]  R←R;A
[7]  I←I+1 ◊ →M1
[8]  A All integers  $x(1) \leq x(2) \leq \dots \leq x(n)$  with  $x(1) + \dots + x(n) = S$ ,  $a \leq x(1)$ .
[9]  A abc[1] ↔ a, abc[2] ↔ n, abc ↔ S.
      V

```

```

      V R←PERM N;I;J;A;B;C;M
[1]  M←I+2 ◊ R← 1 1 ρ1 ◊ B←0
[2]  OLOOP:B←1,B ◊ C←(0,1+1↓ρR)ρJ←0
[3]  ILOOP:A←(JΦB)\R
[4]  A[;M-J]+I ◊ C←C;A
[5]  →(I>J+1)/ILOOP
[6]  M←1+1↓ρR←C
[7]  →(N≥I+I+1)/OLOOP
[8]  A All permutations of 1N.
      V

```

```

      V R←GENERDENSITY MAB;A;B;C;D;E;I;S;M;N;V
[1]  M←MAB[1] ◊ A←MAB[2] ◊ B←MAB[3]
[2]  C←(E←C#0)/C←(2×(B*3)-A*3),(3×(B×B)-A×A),6×B-A
[3]  S←SUMUP2 ~24,(ρC),6
[4]  D←PERMpC
[5]  R←(0,1↓ρD)ρI←1
[6]  LOOP:R←R;S[;D[I;]]
[7]  →((1↓ρD)≥I+I+1)/LOOP
[8]  R←(Λ/0=1IM×R)÷R←R+(ρR)ρC
[9]  R←(0≤R+.×E/(A×A),A,1)÷R
[10] R←(0≤R+.×E/(B×B),B,1)÷R
[11] R←(1,(2#1↓ρR),1)\R
[12] E←R[;1]>0
[13] E←E^(R[;2]>-R[;1]×B+B)^(R[;2]<-R[;1]×A+A
[14] E←~E^(4×x/R[; 1 3])<R[;2]*2
[15] R←DEDOUBLEROWS E÷R
      V

```

What's Wrong with Parentheses?

by Joseph L.F. De Kerf

Abstract

There is a tendency to avoid the use of parentheses in APL. Some renowned specialists even claim they never use them. Excluding parentheses however may, in a lot of cases, decrease readability and as such reduce maintainability. In some cases, the use of unnecessary parentheses even increases readability. So, "What's wrong with parentheses?"

In Vector Vol.5 No.3 page 31, Alan Sykes (1) presented the defined function *COUNTALL* which returns as explicit result *R* the frequency table of the right argument vector *X*, unique values being placed in ascending order within the first column:

```

      ▽TAB←COUNTALL X;I;N
[1]  X←X[⊆X] ⋄ R← 1 2 ρ(1+X),ρX
[2]  +(X[1]=X[ρX])/0
[3]  I←(1⊆X)*X ⋄ N←I/1ρI
[4]  TAB←(I/X),[1.5]N-0,~1+N
      ▽

```

In Vector Vol.6 No.2 page 23, Gerard Langlet (2) establishes this function works perfectly except when *X* is not a vector, index origin is set to 0, or *X* is an empty vector (array). He introduces the following version *COUNTALLV*:

```

      ▽R←COUNTALLV X
[1]  X←,X ⋄ R←~1+1+X+X[⊆X] ⋄ R←X*1⊆X+R,X ⋄ X←1+R/X ⋄
      R←X,[⊆IO+.5]1+R-~1⊆R+R/1ρR
      ▽

```

which runs correctly for any numeric array in either origin $\square IO$.

Both functions still produce *DOMAIN ERROR* when *X* is a character vector (or array) (?).

It is true that *COUNTALLV* is index origin independent, but for numeric arrays it is still - just like *COUNTALL* - comparison tolerance dependent. Why are we so

anxious to make defined functions and eventually defined operators index origin independent, while nobody cares about comparison tolerance dependency?

The statement that both functions do not work for character vectors (or arrays) however, is only partly true.

In VAX APL and Dyalog APL e.g., the monadic form of grade up and grade down of character vectors and matrices/arrays is supported, the collating sequence being implied by the ordering in the atomic vector $\square AV$. In WATCOM APL too, the monadic form of grade up and grade down of character vectors and matrices is supported, but the collating sequence is determined by the system variable $\square CS$ (the default value being $\square AV$).

This means that with such implementations, the function *COUNTALL* works perfectly for non-empty character strings and index origin 1, while *COUNTALLV* doesn't work at all: The statement

```
R←~1+1+X←X[AX]
```

contains a mishmash of types and causes a *DOMAIN ERROR*.

But what disturbs me at most is: **What are we talking about?** Are we talking about teaching of and with APL? Or are we talking about the use of ingenious but sometimes irritating tricks which in most cases decrease readability and as such reduce maintainability?

When in my courses I tackle the problem of producing such a frequency table, I first describe how we would do it with paper and pencil: (1) sorting of the data in ascending order; (2) eliminating of the duplications in the ordered set; and (3) counting the frequency of the elements of the cleaned set in the original but ordered set. Translating this algorithm literally into APL gives the defined function *COUNTALL1*:

```

      V R←COUNTALL1 X
[1]   X←X[AX←,X]
[2]   R←((X←X)=1ρX)/X
[3]   R←R,[[IO+0.5]+/Rρ.=X
      V

```

which runs correctly for any numeric or alphabetic array in either origin $\square IO$. In addition, it is an impressive demonstration of the beauty and power of the outer product generator. That the use of the outer product may be avoided is not

relevant - at that stage of the course at least. Most important is that, in comparison with *COUNTALL* and *COUNTALLV*, the function *COUNTALL1* is extremely clear and perfectly readable.

Of course we may avoid the use of parentheses, as is done e.g. in the defined function *COUNTALL2*:

```

      ▽ R←COUNTALL2 X
[1]   X←X[⌊X⌋,X]
[2]   R←X1X
[3]   R←R=1ρX
[4]   R←R/X
[5]   R←R, [⌊IO+0.5⌋]+/R○.=X
      ▽

```

and we may write the sequence of statements on one line, using the diamond separator, which results in the defined functions *COUNTALL3* and *COUNTALL4* respectively:

```

      ▽ R←COUNTALL3 X
[1]   X←X[⌊X⌋,X] ♦ R←((X1X)=1ρX)/X ♦ R←R, [⌊IO+0.5⌋]+/R○.=X
      ▽

```

```

      ▽ R←COUNTALL2 X
[1]   X←X[⌊X⌋,X] ♦ R←X1X ♦ R←R=1ρX ♦ R←R/X ♦ R←R, [⌊IO+0.5⌋]+/R○.=X
      ▽

```

But *COUNTALL1* is much more readable than *COUNTALL2* and when I try to decipher one-liners of statements separated by diamonds such as in *COUNTALL3* and *COUNTALL4*, the first thing I do is - like most of my students - to write the separated statements beneath each other, from top to bottom. In addition, for this example, the advantage in memory space and cpu time of excluding parentheses and/or using the diamond separator is quasi negligible.

Another even more impressive example is the calculation of the length of the hypotenuse of a right triangle from its legs A and B. This may be done with the defined functions *HYP1* or *HYP2*:

```

      ▽ R←A HYP1 B
[1]   R←((A*2)+(B*2))*0.5
      ▽

```

```

      ▽ R←A HYP2 B
[1]   R←((A*2)+B*2)*0.5
      ▽

```

... showing how, due to the so-called right-to-left rule, the second pair of internal parentheses is superfluous (Whether this improves readability is discussable!).

The functions may be used in the higher-dimension case by evaluating $A \text{ HYPX } B \text{ HYPXC } \dots$ or - if the implementation supports the use of defined functions as arguments of operators - by evaluating $\text{HYPX}/A, B, C, \dots$. The extension to the higher-dimension case however, may be incorporated in the defined function itself such as is done for instance in the function HYP3 :

```

      ▽ R←HYP3 V
[1]  R←(+/V*2)*0.5
      ▽

```

where the right argument vector V is the cocatenation of the legs $A \ B \ C \ \dots$ ($V \leftarrow A, B, C, \dots$).

Once again, the use of parentheses may be excluded as is done in the defined functions HYP4 and HYP5 respectively:

```

      ▽ R←A HYP4 B
[1]  R←A*2
[2]  R←R+B*2
[3]  R←R*0.5
      ▽

      ▽ R←HYP5 V
[1]  R←+/V*2
[2]  R←R*0.5
      ▽

```

and we may write the sequence of statements on one line, using the diamond separator, as is done in the defined functions HYP6 and HYP7 respectively:

```

      ▽ R←A HYP6 B
[1]  R←A*2 ◊ R←R+B*2 ◊ R←R*0.5
      ▽

      ▽ R←HYP7 V
[1]  R←+/V*2 ◊ R←R*0.5
      ▽

```

But once again, the excluding of the parentheses and the use of the diamond separator decrease readability, while the difference in memory space and cpu time is quasi negligible. This does not mean that the diamond separator shouldn't be supported by the ISO APL Standard: most current non-IBM implementations of APL support the diamond separator and a lot of publications and software made available over the past ten to fifteen years make use of the separator (Whether we like it or not!) (3).

In "APL: An Interactive Approach" (4), L. Gilman and A.J. Rose advise: "As you begin to exercise your skills building expressions with many functions, don't hesitate to overparenthesise. When you are more at home in your understanding of the APL language, you can experiment with omitting, nonessential parentheses." More recently, in "APL2 at a Glance" (5), J.A. Brown et al. proclaim: "If parentheses help you understand what you write, use them." I should say: "If parentheses help you and the readers of your programs understand what you write, use them."

Opponents of APL claim that APL is not readable and as such is not maintainable. This is a myth. APL is one of the most readable and maintainable programming languages available. However, as a consequence of its conciseness and powerfulness, we may be tempted needlessly to complicate and obscure our programs. Please, let us keep our programs clear and lucid. So, "What's wrong with parentheses?"

Note: In SHARP APL and APL90, using the *lev* function, the hypotenuse may be calculated through the single statement without parentheses:

$$R \leftarrow R * 0.5 - R \div V * 2$$

But in my opinion, this too is much less readable than the statement:

$$R \leftarrow (+ / V * 2) * 0.5$$

References

- [1] Sykes A. *Counting in APL - Part Two*. Vector, Vol.5, No.3, January 1989, pp.26-32.
- [2] Langlet G.A.. *APL "RISC" Programming Style*. Vector, Vol.6, No.2, October 1989, pp.23-24.
- [3] De Kerf J.L.F. *A Note on Separators in APL*. APL-CAM Journal, Vol.11, No. 3, 16 July 1989, pp.683-696.
See also Vector, Vol.6, No.2, October 1989, pp.123-139.
- [4] Gilman L. and Rose A.J. *APL: An Interactive Approach - Third Edition*. John Wiley and Sons, New York, New York, 1984.
- [5] Brown J.A., Pakin S., Polivka R.P. *APL2 at a Glance*. Prentice Hall, Englewood Cliffs, New Jersey, 1988. See also: Brown J.A., Pakin S., Polivka R.P. *APL2 - Ein erster Einblick*. Springer-Verlag, Berlin-Heidelberg, 1989.

dyalog APL

for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable $\square$ SM (Screen Map). Facilities include:*

Pop-up windows by localisation of $\square$ SM
Scrolling (vertical, horizontal, linked)
GENUINE numeric fields
Date fields
NESTED forms

- **Fully Integrated window-based Session Manager, Editor and Debugging Environment.**
- **Compatible with the Draft ISO standard for APL**
- **Huge Workspaces (16Mb and MORE!)**
- **Virtual Memory Support (using on-chip demand paging)**
- **Full 32-bit processing speed**
- *APL2 features including:*
 - Nested Arrays**
 - Defined Operators**
 - Selective Specification**
- *Commercial features including:*
 - $\square$ **FMT**
 - Component Files**
 - Error Handling**
 - "AP124-like" Screen Manager**
- **External Variables**
- **Interface to GSS*CGI for fast device-independent graphics**
- **C Interface**

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

For further information contact:

Dyadic Systems Limited

Riverside View, Basing Road, Old Basing, Basingstoke,
Hants, RG24 0AL Tel: 0256 811125 Fax: 0256 811130

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner	Ray Cannon	132
Technical Correspondence	Dave Piper	133
□CT Explained	Norman Thomson	135
Namespace Play	Jonathan Barman	138

Hackers' Corner: Unlocking Functions in Dyalog APL for MS DOS

by Ray Cannon

By comparing DYALOG APL workspaces as DOS files, it is easy to discover that locked functions differ from unlocked functions by a single byte. Unlocked functions have a marker of hex 23, but in locked functions, this marker is hex 33.

So here is how to unlock a locked function. (To simplify the explanation, I will use a function called *F00* and a temporary workspace named LOCK.)

First copy the function *F00* into a clear ws. Save this workspace as LOCK. (Don't name the ws the same as the locked function as this might cause difficulties later.) Exit back to DOS.

Change directory to the one holding your newly saved ws. Now using DEBUG modify the HEX 33 to HEX 23 viz:- Enter DEBUG thus: DEBUG LOCK.DWS<enter> Then after DEBUG's prompt (a minus sign) type: L 0<enter> This will load the file starting at offset 0. Then locate the locked function by searching for it thus:- S 0,700 46 4F 4F<enter> Where 46 4F 4F is *F00* (the function name) in hex. Debug will respond with its location thus:- 27F4:05C8 or some such numbers. Ask DEBUG to display its contents round there by typing: D 05C8<enter> DEBUG will respond with 8 lines of hex with text on the right. At the position 5C8 in the text will be the function name (*F00*). In the HEX the function name will be followed by 00. This will be followed about 8 HEX numbers and then the all-important HEX 33. (In the text, this will display as the number 3.) If the HEX 33 is at location 5D4 you change it to HEX 23 by typing:- E 5D4 23<enter> You then write the change back to disk by typing:- W 0<enter> Debug will say "Writing 062c bytes" or some such. Then exit from DEBUG by typing:- Q<enter>

Now reload DYALOG APL,)COPY LOCK F00 and check *F00* is unlocked. Simple!

Alternatively, you could write a function in APL to read LOCK as a DOS file into APL, modify the HEX 33 to 23, and write the data back out to the same DOS file. But I'll leave that up to you.

TECHNICAL CORRESPONDENCE

From: Dave Piper

12th Sept 1990

I feel I must follow up a couple of points raised in Peter Branson's letter to Vector (Vector[6;4;95]).

Firstly with regards to training people in nested array versions of APL. I suspect Peter is arguing from the point of view of someone who learnt nested arrays after learning "APL1". If introduced in a well structured course, the concept of depth is no more difficult to grasp than that of rank. Once depth is understood, most of the other bits to do with nested arrays follow quite easily.

In my experience, one of the first things students on APL fundamentals courses try is:

```
A←1 2 3
B←4 5 6
C←A B
```

I find the explanation of why you cannot put names next to each other is often quite difficult!

If you are training people to use nested array versions of APL, why introduce any unnecessary differentiation between simple and nested arrays? Peter's point about being able to maintain "APL1" code is valid I guess; however, getting students to work out how to do things without nested arrays (on a course which teaches nested arrays) can form a useful exercise. By teaching nested arrays from the first, their work can take advantage of nested arrays and their associated benefits immediately.

Secondly, Peter talks about a workspace search function. From his description it sounds as if the original uses "peppery" code e.g.:

```
LIST←(v/'',(STR)⊟⊞CR"[2]FNS)+FNS
```

Pepper-ised code (nothing to do with my surname), is one of the deadly sins of nested array versions of APL. The best technique for the application of EACH is to write a function that works on a single example, then generalise this by using each. We would solve the problem:

```

      ∇ R←STR SEARCH_FN NAME
[1]   R←∇/,STR∈□CR NAME
      ∇

```

and use this function:

```
LIST←((STR)SEARCH_FN"[2]FNS)+FNS
```

... thereby reducing the use of each to just one! It sounds from his letter as though this is just the approach Peter took - I'm not surprised it ran so much faster. The important point is that the first version may use "many APL2 features" (in reality no more than the second version) - but it uses them in an inappropriate manner. It is the appropriate use of facilities that is often the most important factor when designing code.

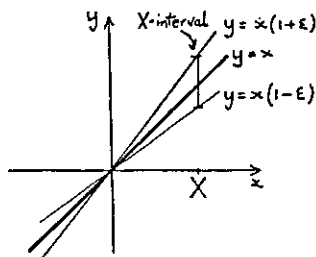
My only other comment is the idea of fixing *SEARCH_FN* or its equivalent as a local function to help with this is ridiculous. Far better to set up a copy list variable and use the indirect copy facility. Alternatively, with APL2 Version 1 Release 3, you can always set it up as a packaged workspace.

Dave Piper
41 Sandown Drive
Rainham
Kent ME8 9DT

Comparison Tolerance Explained

by Norman Thomson

I should like to add some further comments to the note on Comparison Tolerance by RG Williams and RG Green in Vector Vol.6 No.4. This is a topic about which I consider an unjustified mystique has evolved. A simple but complete picture is given by the following diagram which for every value on the x-axis defines on the y-axis an interval within which other numbers are deemed equal to it.



For ease of reading $\square CT$ is written as ϵ on the diagram. The interpretation is that for some computational purposes X means not a number, but an *interval*, viz. $[X(1-\epsilon), X(1+\epsilon)]$.

The above is the situation which pertains to a "perfect" computer, i.e. one which is not constrained by the bounds of finite arithmetic. Of course any real computer *is* so bounded, and thus quite separately from Comparison Tolerance there is for each type of computer a specific System Tolerance, *not* controllable by the user. In general the interval diagram describing System Tolerance has the same form as the Comparison Tolerance chart, but there is no necessity for it to do so, and the lines defining the bounds could in principle have completely arbitrary shapes. On machines with IBM type architectures the mantissa of a double precision floating point number is effectively represented by 53 bits. 2^{53} is approximately 10^{16} so System Tolerance is about $1E^{-16}$.

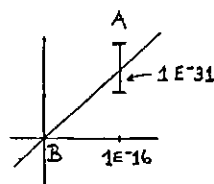
Returning to Comparison Tolerance, a complete understanding of $\square CT$ consists of appreciating that functions which involve comparisons are applied to *intervals* and not to numbers. Thus, to use Williams and Green's example, set

$$\begin{aligned} A &= 1E^{-16} \\ B &= 0 \\ \square CT &= 1E^{-15} \end{aligned}$$

Then

$$A=B$$

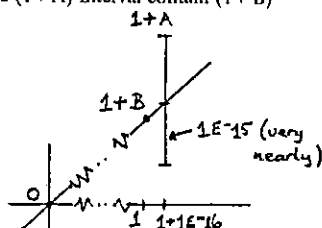
means does B lie in the A -interval where A is the number with the larger absolute value.



Clearly the answer is no. On the other hand

$$(1+A) = 1+B$$

means does the $(1+A)$ -interval contain $(1+B)$

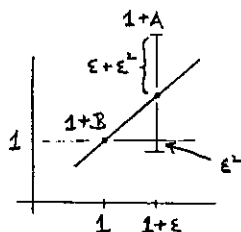


and the answer is yes. Provided you remember that you are doing *interval arithmetic* there are no problems and no contradictions ... or at least not on the perfect machine. Problems only arise in critical cases where System Tolerance interferes. Take for example the more interesting settings

$$A = \square CT + 1E^{-15}$$

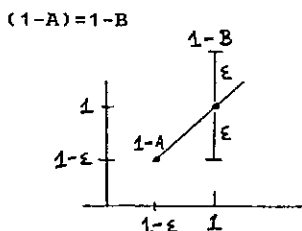
$$B = 0$$

Following the reasoning above the result of $(1+A) = 1+B$ can be predicted from the diagram



The answer on a perfect machine is undoubtedly yes. However once ϵ^2 comes into the area of System Tolerance, i.e. $\square CT$ is less than about $1E^{-8}$, the answer becomes unpredictable - on APL2/PC the answer is no. Setting $A = \square CT + 10^{-N}$ we find that $(1+A) = (1+B)$ is true for $N = 5$ to 8, 13, 14 and 16 and false for $N = 9, 10, 11, 12$ and 15. On mainframe APL2 it is true in all cases.

A further question is whether the intervals are open or closed. This should be answered for individual implementations by asking the question



On APL2/PC $(1-A) = (1-B)$ is true for $N = 5, 7, 9, 12, 14$ and 15 and false for $N = 6, 8, 10, 11, 13$ and 16. On mainframe APL2 it is true in all cases.

It should always be borne in mind that $\square$ CT applies only to functions that perform *comparisons* either implicitly or explicitly, and I thus question some of the functions in Williams and Green's list. For example how is $\square$ affected by Comparison Tolerance? - it isn't on IBM systems, neither is monadic $\square$ (wrongly called index) although *index of* (i.e. dyadic index) *does* involve comparisons and therefore *is* sensitive to $\square$ CT. The case for residue can be made either way - it appears to be sensitive to $\square$ CT on Dyalog but not on IBM.

Bracket indexing on the other hand should be added to the list for all systems since an index, e.g. 1 in $M[1]$, is in fact an *interval* which if it includes an integer identifies an item in an array. With $A \leftarrow \square CT * 10 * N$, $M[1-A]$ and $M[1+A]$ both give DOMAIN ERRORS on APL2/PC for $N = 5$ to 14 and $M[1]$ for $N > 14$. For mainframe the range giving DOMAIN ERRORS is $N = 5$ to 13.

If the list of functions sensitive to $\square$ CT is extended to IBM APL2 it will also include $\leq$ (find), $\equiv$ (match) and $\square$ (index).

As a final comment, I raise the question - how useful is $\square$ CT anyway? The popular response seems to be "not for me, but it's great for numerical analysts." My experience however is that not even they find it to be of very great value, and I am inclined to view $\square$ CT as something of an outgrown residual from the early brainstorming days of APL development. I would nevertheless be pleased to hear of practical examples of its usefulness.

Norman Thomson,
Mail Point 188,
IBM UK Laboratories Ltd.
Hursley Park,
WINCHESTER,
Hants SO21 2JN

Footnote: I should like to apologise for a minor error in my contribution to The Card Collecting Problem in Vector Vol.6 No. 4, namely the omission of the first α in the DRAW function which should begin `DRAW:  $\alpha$ DRAW ...` and not `DRAW: DRAW ...` as stated.

Namespace Play

by Jonathan Barman

I had some fun poking about the innards of packaged workspaces and namespaces, so I put together this short annotated APL2 session to show the sort of things I found out. It is not a tutorial session, but a very personal and possibly idiosyncratic view of Namespaces.

Creating a packaged workspace is rather boring, and it is much more interesting to look at one distributed with APL2. The obvious choice is `PACKAGE` workspace which packages up workspaces. The official way to get at the `PACKAGE` function is `3 11 ⍋NA 'PACKAGE'`, but if you look in the Names file which APL2 reads when presented with a `3 11 ⍋NA` you can see that under VM the real left argument to `⍋NA` should be:

```
'AP2VNRP2.AP2PPKG' 11 ⍋NA 'PACKAGE' 1
```

The 1 being returned shows that we hit bull's eye, and have got the correct fully qualified name. The 'AP2VNRP2' is the name of a VM simulated load library, and 'AP2PPKG' is the member name. Under TSO you would normally only need to give the member name 'AP2PPKG' as the APL start process would normally give automatic access to the various load libraries required.

Define a function to save typing out all that gobbledegook:

```
⍷ R←A_NA_P NAME
[1] R←'AP2VNRP2.AP2PPKG' 11 ⍋NA NAME
⍷
```

As we get going things can get confused as to which namespace a function runs in. Functions starting with `A_` are defined in, and run in, the Active workspace. `A_NA_P` will be accessed from other namespaces, but the body of the function will always run in the Active workspace and will `⍋NA` to a name in the `PACKAGE` namespace.

Now let's see what functions have been defined in the `PACKAGE` namespace. You can `⍋NA` to system functions as well as user functions, but you have to give a surrogate name otherwise it gets confused. Anyhow we want to show where the function runs with a `P_` for the `PACKAGE` namespace:

```

1      A_NA_P 'P_NL □NL'

```

The PACKAGE namespace is 'open', in that there is no restriction on the objects that may be accessed. It could have been packaged up with only the PACKAGE function available for Name Association, in which case the statement above would have returned a zero and none of the following would have been possible.

A list of functions in the PACKAGE namespace can be neatly displayed with:

```

      (= [2] P_NL 3) ~'' '
ATR CLOSE COIBM DAN . . . .

```

The sneaky way to get inside an open namespace is to fix a function that blows up inside it:

```

1      A_NA_P 'P_FX □FX'
      P_FX 'P_STOP' 'SΔP_STOP+L1' 'L1:'
P_STOP
)FNS
A_NA_P P_FX P_NL PACKAGE

```

The function *P_STOP* does not exist in our namespace, as the *P_FX* function used its right argument in the PACKAGE namespace, and fixed the function there. We will have to get at it before running it:

```

1      A_NA_P 'P_STOP'
      P_STOP
P_STOP[2]

```

Now we are in the PACKAGE namespace, and can muck about at leisure:

```

)EDITOR 2
VPACKAGE

```

Variables which have been □NA'ed are even easier than functions, because you don't have to worry about where they have been defined:

```

P_VAR+ 'THIS VARIABLE WAS ASSIGNED IN THE PACKAGE NAMESPACE'

```

Lets get at the *A_NA_P* function in the *PACKAGE* namespace so we can post things back into the Active workspace. In a packaged namespace *' ' 11* *□NA* allows one to get at objects in the Active workspace. As before, a little function saves typing and its name will make it clear where it runs:

```

)EDITOR 1
VR+P_NA_A NAME
[1] ' ' 11 □NA NAMEV

P_NA_A 'A_NA_P'
1
A_NA_P => 'P_VAR' 'P_NA_A'
1 1

```

Running *A_NA_P* causes an implicit switch back to the active workspace, and then the body of the function is executed. What one has to hold onto is that *A_NA_P* always runs in the Active Workspace and does a *□NA* to the *PACKAGE* namespace, regardless of from where it is called. *P_VAR* and *P_NA_A* should now appear back in the Active workspace:

```

)RESET
)FNS
A_NA_P P_FX P_NA_A P_NL P_STOP PACKAGE
P_VAR
THIS VARIABLE WAS ASSIGNED IN THE PACKAGE NAMESPACE

```

We have now got the functions *A_NA_P* and *P_NA_A* accessible in both namespaces which enable us to post objects back and forth. Now let's mess about with variables to show how one can change them in either namespace and get consistent results:

```

P_VAR+15
A_VAR+'ASSIGNED IN THE ACTIVE WORKSPACE'
P_NA_A 'A_VSUG A_VAR'
1

```

The Package namespace variable *P_VAR* has been reassigned. A new variable has been created in the Active workspace, and we have caused it to be available in the Package namespace by running the *P_NA_A* function, which flips into the Package namespace and does the *□NA* back to the Active Workspace. A surrogate name *A_VSUG* was given just to be different.

Get into the Package namespace and see what has happened:

```

        P_STOP
P_STOP[2]
        P_VAR
1 2 3 4 5
        A_VSUG
ASSIGNED IN THE ACTIVE WORKSPACE

```

Surrogate names make it possible for packaged workspace functions to work safely on a list of variable names without having to use execute. Alterations can be made in place so that big arrays do not have to be passed to and fro. However, `□NA` is quite expensive in CPU; doing `□NA` on 1000 variables can get rather boring.

Reassignment can be carried out in the Package namespace, and low and behold, the changes pop up in the Active namespace:

```

        ((' '=A_VSUG)/A_VSUG)+'.'
        )RESET
        A_VAR
ASSIGNED IN THE ACTIVE WORKSPACE

```

A `)SAVE` at this point keeps all the changes we have made as a sort of hidden part of our workspace, but does not touch the original AP2PPKG member of the Load Library.

```

        )SAVE PLAY
????????
        )CLEAR
CLEAR WS
        )LOAD PLAY
????????
        )FNS
A_NA_P P_FX P_NA_A P_NL P_STOP PACKAGE
        P_STOP
P_STOP[2]
        )FNS
A_NA_P ATR CLOSE COIBM DAN . . . . P_NA_A P_STOP

```

This feature could be useful for developing and debugging a packaged workspace. When development has finished `)OUT` can be used to write out all the required objects, and then the package can be re-created with the new code.

Two functions are supplied by IBM to help manage the packaged environment, called `EXP` and `QNS`, which we will get at in both namespaces:

```

0 11  $\square$ NA  $\triangleright$ 'EXP' 'QNS'
1 1
P_NA_A  $\triangleright$ 'EXP' 'QNS'
1 1

```

The second statement shows that we can $\square$ NA to a function indirectly, which is an unpleasantly convoluted way of going about things, but might be useful. The 0 11 means that we do not need to check whether the names are functions or variables. There are some decidedly arcane situations where 0 11 has to be used, but this is not one of them. QNS tells us how the namespace was entered:

```

P_STOP
P_STOP[2]
QNS 0
AP2VNRP2.AP2PPKG 11

```

The right argument is always 0. Could a future enhancement change this? Strictly speaking, QNS gives the left argument to $\square$ NA used to access the function which caused an explicit entry to the namespace. If PACKAGE had been entered the official way with 3 11 $\square$ NA 'PACKAGE', then QNS 0 would have returned 3 11. Also, the new Program Interface feature of APL2 allows packaged workspaces to be entered without calling an APL function, in which case QNS returns ' ' 11. Use QNS with care.

EXP executes a statement in the previous namespace, whatever that may be:

```

EXP  $\leftarrow$  'X+50'
X
VALUE ERROR
)RESET
X
50

```

We could have done the same job with X+50 and A_NA_P 'X', if we had known that the previous namespace was in fact the Active workspace. It may be better to think of ways of keeping track of namespaces so that explicit $\square$ NAs can be done instead of blindly using EXP everywhere. The IBM manual gives some examples of how this can be done.

Hopefully the session has given the flavour of Packaged workspaces and namespaces, which will enable you to get stuck in and develop useful applications.

Index to Advertisers

APL People	2
Cocking and Drury	8
Dyadic Systems Ltd	130
HMW Computing	6
MicroAPL	10,75
APL Booklist (Renaissance Data Systems)	32

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

Product Guide updates should continue to go to Alison Chatterton, as should requests for advertising space.

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL.
The membership year runs from 1st May to 30th April.

Name: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category (please tick box): 90/91

UK private membership	£10	☐
Overseas private membership	£18	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£85	☐
Corporate membership overseas	£140	☐
Sustaining membership	£360	☐
Non-voting student membership (UK only)	£5	☐

For student applicants:

Name of course: _____
Supervisor: _____
Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

Signature (and Expiry date) _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
Alison Chatterton, 9 Oak Grove,
HERTFORD SG13 8AT
England

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1990/91 Committee

Chairman:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Secretary:	Anthony Camacho 0727-860130	2 Blenheim Rd, ST ALBANS, Herts AL1 4NR.
Treasurer:	John Sullivan 081-462 3628	24 Mounthurst Road, HAYES, Kent BR2 7QN
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 0737-242950	Oaklands Cottage, Wray Common, REIGATE, Surrey RH2 0LE
Education:	Dr Alan Sykes 0792-295296	European Business Management School Swansea University, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 081-562 5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 071-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 081-948 6737	13A Mount Ararat Road, RICHMOND, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-860130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	David Ziemann (071-436 9481), John Searle (081-948 6737), Ray Cannon (0252-874697), Sylvia Camacho, Bridget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel: 0225 462602

Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483 302249
Fax: 0483 302279

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 071-353 4212
Fax: 071-353 3325

Cooking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel: 071-436 9481

REUTER FILE
1-4 Singer St.
LONDON EC2A 4BQ
Tel: 071-867 1166
Fax: 071-867 9792

Intelligent Programs Ltd
Unit 7, Hermitage Court
6-10 Sampson St
LONDON, E1 9NA
Tel: 071-481 4813

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 0256 811125
Fax: 0256 811130

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel: 0732-885126

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8866

Peter Cyriax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel: 071-624-7013
Mobile: 0860-377963



The British Computer Society, 13 Mansfield Street, London W1M 0BD.